Optimization with Optuna: A Practical and Theoretical Guide to Modern Black-Box Optimization

Advanced Mini-Course

Miguel Conde

2026-03-14

Table of contents

Preface 1
Who This Book Is For . . . 1
What You Will Learn . . . 2
How To Use The Book . . . 2
A Note On Scope . . . 2

Introduction 3
The Problem Optuna Solves . . . 3
Why This Book Exists . . . 3
How The Book Is Organized . . . 4
Part I: Foundations . . . 4
Part II: Core Usage . . . 4
Part III: Advanced Techniques . . . 4
Part IV: Applications . . . 4
What You Should Expect From The Examples . . . 4
A Working Mental Model . . . 5
What "Mastery" Looks Like Here . . . 5
Suggested Reading Path . . . 5

I. Part I: Foundations 7

1. Foundations of Modern Optimization 9
1.1. Why This Matters . . . 9
1.2. Formal Definition of Optimization . . . 9
1.2.1. Example . . . 10
1.3. Black-Box Optimization . . . 10
1.4. Curse of Dimensionality . . . 11
1.5. Grid Search vs Random Search . . . 11
1.5.1. Grid Search . . . 11
1.5.2. Random Search . . . 12
1.6. Sample Efficiency . . . 12
1.7. Exploration vs Exploitation . . . 12
1.7.1. Exploration . . . 12
1.7.2. Exploitation . . . 13
1.8. When Gradient-Based Methods Fail . . . 13
1.9. Why Bayesian Optimization Works . . . 14
1.10. Where Optuna Fits . . . 14
1.11. Practical Exercise: Implement Random Search . . . 15
1.12. Recap . . . 16

1.13. Decision Guide . . . 16

2. Bayesian Optimization Fundamentals **17**
2.1. Why This Matters . . . 17
2.2. A Fast First Win . . . 17
2.3. Why Surrogate Models Matter . . . 18
2.4. Gaussian Processes: The Classical View . . . 19
2.5. What Does a GP Predict? . . . 20
2.6. How the Acquisition Function Chooses the Next Point . . . 20
2.7. Probability of Improvement . . . 21
2.8. Expected Improvement . . . 21
2.9. Upper and Lower Confidence Bounds . . . 22
2.10. The Bayesian Optimization Loop . . . 22
2.11. Where Gaussian-Process Bayesian Optimization Struggles . . . 22
2.11.1. Computational Complexity . . . 23
2.11.2. High-Dimensional Search Spaces . . . 23
2.11.3. Categorical and Conditional Variables . . . 23
2.12. Why Optuna Transitions to TPE Instead . . . 23
2.13. Practical Exercise: Compare Mean and Improvement . . . 24
2.14. Recap . . . 24
2.15. Decision Guide . . . 24

3. The TPE Algorithm **27**
3.1. Why This Matters . . . 27
3.2. A Fast First Win . . . 27
3.3. Why TPE Flips the Bayesian Optimization View . . . 29
3.4. Splitting Observations into Good and Bad Trials . . . 29
3.5. The Two Density Models . . . 30
3.6. The Likelihood-Ratio Rule . . . 30
3.7. How Parzen Estimators Build the Densities . . . 31
3.8. Where Exploration Comes From in TPE . . . 31
3.8.1. Density Ratio . . . 31
3.8.2. Candidate Sampling . . . 31
3.8.3. Quantile Threshold . . . 31
3.9. The Full TPE Procedure . . . 32
3.10. Why TPE Works Well in Practice . . . 32
3.10.1. High-Dimensional Spaces . . . 32
3.10.2. Categorical Variables . . . 32
3.10.3. Tree-Structured Search Spaces . . . 33
3.10.4. Better Scalability . . . 33
3.11. Why Optuna Uses TPE by Default . . . 33
3.12. Practical Exercise: Implement a Simplified TPE Loop . . . 33
3.13. Recap . . . 34
3.14. Decision Guide . . . 34

4. Optuna Architecture **35**
4.1. Why This Matters . . . 35
4.2. A Fast First Win . . . 35

4.3. Architecture at a Glance . . . 36
4.4. The Study Object . . . 37
4.5. The Trial Object . . . 38
4.5.1. Trial lifecycle . . . 38
4.6. The Objective Function . . . 38
4.7. The Search Space and Define-by-Run . . . 39
4.8. Samplers . . . 39
4.9. Pruners . . . 40
4.10. Storage Backends . . . 41
4.10.1. In-memory storage . . . 41
4.10.2. SQLite storage . . . 41
4.10.3. Server databases . . . 42
4.11. The Internal Optimization Loop . . . 42
4.12. A Minimal End-to-End Example . . . 43
4.13. Practical Exercise: Inspect Trial Objects . . . 44
4.14. Recap . . . 45
4.15. Decision Guide . . . 45

II. Part II: Core Usage **47**

5. Basic Optuna Usage **49**
5.1. Why This Matters . . . 49
5.2. A Fast First Win . . . 49
5.3. Installation and Setup . . . 50
5.4. Creating a Study . . . 50
5.5. Defining the Objective Function . . . 51
5.6. Running the Optimization . . . 51
5.7. The Parameter Suggestion API . . . 52
5.7.1. `suggest_float` . . . 52
5.7.2. Log-scaled floats . . . 52
5.7.3. `suggest_int` . . . 53
5.7.4. `suggest_categorical` . . . 53
5.8. A Study with Multiple Parameters . . . 53
5.9. Retrieving Optimization Results . . . 53
5.9.1. Best value . . . 53
5.9.2. Best parameters . . . 54
5.9.3. Best trial . . . 54
5.10. Inspecting All Trials . . . 54
5.11. A Minimal End-to-End Example . . . 54
5.12. Practical Exercise: Optimize a Mathematical Function . . . 55
5.13. Recap . . . 56
5.14. Decision Guide . . . 57

6. Designing Search Spaces **59**
6.1. Why This Matters . . . 59
6.2. A Fast First Win . . . 59

6.3. Continuous, Discrete, and Categorical Parameters . . . 60
6.3.1. Continuous parameters . . . 60
6.3.2. Discrete parameters . . . 61
6.3.3. Categorical parameters . . . 61
6.4. Why Log-Scaled Parameters Matter . . . 61
6.5. Conditional Parameters . . . 62
6.6. Dynamic Search Spaces . . . 62
6.7. Hierarchical Search Spaces . . . 63
6.8. Example: Neural Architecture Search . . . 64
6.9. Example: Algorithm Configuration . . . 64
6.10. Best Practices for Search Space Design . . . 64
6.10.1. Use log scales for multiplicative parameters . . . 64
6.10.2. Avoid absurdly large ranges . . . 65
6.10.3. Use domain knowledge aggressively . . . 65
6.11. Practical Exercise: Conditional Hyperparameter Optimization . . . 65
6.12. Recap . . . 66
6.13. Decision Guide . . . 67

7. Samplers **69**
7.1. Why This Matters . . . 69
7.2. A Fast First Win . . . 69
7.3. What a Sampler Does . . . 70
7.4. TPE Sampler . . . 71
7.5. Random Sampler . . . 71
7.6. CMA-ES Sampler . . . 72
7.7. Grid Sampler . . . 73
7.8. NSGA-II for Multi-Objective Optimization . . . 74
7.9. Exploration Behavior Across Samplers . . . 74
7.10. Samplers in High-Dimensional Spaces . . . 75
7.11. Choosing the Right Sampler . . . 75
7.12. Practical Exercise: Compare Samplers . . . 76
7.13. Recap . . . 77
7.14. Decision Guide . . . 77

8. Pruning **79**
8.1. Why This Matters . . . 79
8.2. A Fast First Win . . . 79
8.3. Why Pruning Helps . . . 80
8.4. Intermediate Results . . . 81
8.5. The Pruning Decision . . . 81
8.6. Median Pruning . . . 82
8.7. Successive Halving . . . 83
8.8. Hyperband . . . 83
8.9. Asynchronous Pruning . . . 83
8.10. Pruners Available in Optuna . . . 84
8.11. Integration with Training Loops . . . 84
8.12. Example: Deep-Learning-Style Pruning Loop . . . 85
8.13. Practical Exercise: Simulate Pruning . . . 85

8.14. Recap . . . 86
8.15. Decision Guide . . . 87

III. Part III: Advanced Techniques **89**

9. Advanced Optimization Techniques **91**
9.1. Why This Matters . . . 91
9.2. Fast First Win: A Multi-Objective Study . . . 91
9.3. When Does the Search Space Become Part of the Algorithm? . . . 94
9.4. What Changes in Multi-Objective Optimization? . . . 95
9.5. How Should You Read a Pareto Front? . . . 96
9.6. How Do Constraints Enter the Search? . . . 96
9.6.1. Penalty-Based Constraints . . . 97
9.6.2. Constraint-Aware Multi-Objective Search . . . 97
9.7. When Should You Write a Custom Sampler? . . . 98
9.8. When Should You Write a Custom Pruner? . . . 98
9.9. Practical Exercise: Constrained Multi-Objective Optimization with NSGA-II . . . 99
9.10. Recap . . . 100
9.11. Decision Guide . . . 100

10. Visualization and Diagnostics **101**
10.1. Why This Matters . . . 101
10.2. Fast First Win: Build a Study Worth Diagnosing . . . 101
10.3. Which Tools Does Optuna Provide? . . . 105
10.4. How Should You Read Optimization History? . . . 105
10.5. What Does Parameter Importance Actually Mean? . . . 106
10.6. How Do Slice Plots Expose Useful Structure? . . . 107
10.7. When Do Parallel Coordinates Help More Than Slices? . . . 107
10.8. What Do Contour Plots Add? . . . 107
10.9. How Do Visuals Reveal a Bad Search Space? . . . 108
10.10. How Do You Diagnose Parameter Interactions? . . . 108
10.11. When Is the Dashboard Worth Using? . . . 109
10.12. Practical Exercise: Diagnose a Weak Study and Repair It . . . 109
10.13. Recap . . . 110
10.14. Decision Guide . . . 110

11. Distributed and Scalable Optimization **111**
11.1. Why This Matters . . . 111
11.2. Fast First Win: Local Parallel Optimization . . . 111
11.3. What Does `n_jobs` Actually Buy You? . . . 113
11.4. When Do You Need Shared Storage? . . . 113
11.5. Which Storage Backend Should You Use? . . . 114
11.5.1. In-Memory Storage . . . 114
11.5.2. SQLite Storage . . . 114
11.5.3. PostgreSQL or MySQL . . . 114
11.6. How Does Distributed Execution Work? . . . 115
11.7. What Happens When Workers Fail? . . . 115

11.8. How Should You Think About Scalability? . . . 116
11.9. What Does a Multi-Worker Setup Look Like? . . . 116
11.9.1. Step 1: Create or Load the Study . . . 116
11.9.2. Step 2: Define the Worker Objective . . . 116
11.9.3. Step 3: Run the Worker . . . 117
11.10How Do You Monitor a Large Study? . . . 117
11.11How Does Optuna Fit Into Larger Distributed Systems? . . . 117
11.12Practical Exercise: Turn a Local Study Into a Shared Study . . . 118
11.13Recap . . . 119
11.14Decision Guide . . . 119

IV. Part IV: Applications **121**

12. Optuna for Machine Learning **123**
12.1. Why This Matters . . . 123
12.2. Fast First Win: Tune a Real Classifier Safely . . . 123
12.3. Why Are Pipelines So Important? . . . 125
12.4. How Should You Structure a Scikit-Learn Objective? . . . 125
12.5. What Changes With Gradient Boosting Libraries? . . . 126
12.5.1. XGBoost Pattern . . . 126
12.5.2. LightGBM Pattern . . . 127
12.6. How Does Optuna Fit Deep Learning Loops? . . . 127
12.6.1. PyTorch Pattern . . . 127
12.6.2. TensorFlow / Keras Pattern . . . 128
12.7. Why Is Cross-Validation Still Central? . . . 128
12.8. When Do You Need Nested Cross-Validation? . . . 129
12.9. How Do You Avoid Data Leakage? . . . 129
12.10What Makes an ML Study Reproducible? . . . 130
12.11Practical Exercise: End-to-End Pipeline Optimization . . . 130
12.12Recap . . . 131
12.13Decision Guide . . . 132

13. Optuna for General Optimization **133**
13.1. Why This Matters . . . 133
13.2. Fast First Win: Optimize a Non-Convex Function . . . 133
13.3. What Makes a Problem a Good Fit for Optuna? . . . 136
13.4. How Does Mathematical Optimization Differ From ML Tuning? . . . 137
13.5. How Should You Think About Simulation Calibration? . . . 137
13.6. Why Are Physics and Engineering Objectives Good Examples? . . . 138
13.7. What About Algorithm Configuration? . . . 138
13.8. Why Is Reinforcement Learning a Special Case? . . . 139
13.9. How Do You Handle Expensive or Noisy Objectives? . . . 139
13.10Why Do Benchmark Functions Still Matter? . . . 140
13.11A Closer Look: The Rastrigin Function . . . 140
13.12Practical Exercise: Optimize a Benchmark Function . . . 141
13.13Recap . . . 148
13.14Decision Guide . . . 148

14. Production Workflows **151**
14.1. Why This Matters . . . 151
14.2. Fast First Win: Add Persistence and Logging . . . 151
14.3. Experiment Tracking . . . 153
14.4. Reproducibility . . . 153
14.4.1. Fix Random Seeds . . . 154
14.4.2. Version Control . . . 154
14.4.3. Environment Reproducibility . . . 154
14.5. Study Persistence . . . 155
14.6. Experiment Management . . . 155
14.6.1. Naming Studies . . . 156
14.7. Integration With ML Pipelines . . . 156
14.8. Logging Experiments . . . 157
14.9. Optuna Dashboard . . . 157
14.10 Monitoring Production Experiments . . . 158
14.11 Best Practices for Production Systems . . . 158
14.11.1 Use Persistent Storage . . . 158
14.11.2 Limit Trial Runtime . . . 158
14.11.3 Monitor Resource Usage . . . 158
14.11.4 Separate Optimization and Training . . . 159
14.11.5 Store Best Model Configuration . . . 159
14.12 Example Production Workflow . . . 159
14.13 Practical Exercise: Turn A Study Into A Production Artifact . . . 160
14.14 Recap . . . 160
14.15 Decision Guide . . . 161

15. Final Project **163**
15.1. Why This Matters . . . 163
15.2. How to Use This Chapter . . . 163
15.3. Fast First Win: A Small Capstone Preview . . . 164
15.4. Project A: ML Pipeline Optimization . . . 167
15.4.1. Objective . . . 167
15.4.2. Problem Setup . . . 167
15.4.3. What the Search Space Should Express . . . 167
15.4.4. Full Project Code . . . 168
15.4.5. What Project A Should Teach You . . . 170
15.4.6. Recommended Extensions . . . 170
15.4.7. Deliverables for Project A . . . 170
15.5. Project B: Complex Function Optimization . . . 170
15.5.1. Objective . . . 171
15.5.2. Benchmark Function Choices . . . 171
15.5.3. Benchmark Definitions . . . 171
15.5.4. Objective Wrapper . . . 172
15.5.5. Comparing Samplers . . . 172
15.5.6. Recommended Evaluation Protocol . . . 172
15.5.7. What Project B Should Teach You . . . 173
15.5.8. Deliverables for Project B . . . 173

15.6. Project C: Simulation Optimization . . . 174
15.6.1. Objective . . . 174
15.6.2. Example Problem: Inventory Optimization . . . 174
15.6.3. Simulation Model . . . 174
15.6.4. Objective Function . . . 175
15.6.5. Why Project C Is Valuable . . . 176
15.6.6. Possible Extensions . . . 176
15.6.7. Deliverables for Project C . . . 176
15.7. How the Three Projects Fit Together . . . 177
15.8. Recommended Capstone Order . . . 177
15.9. Final Report Structure . . . 177
15.10 Completion Checklist . . . 178
15.11 Recap . . . 178
15.12 Course Completion Summary . . . 178

Summary **179**
The Main Lessons To Keep . . . 179
How The Parts Fit Together . . . 180
What Competence Should Look Like Now . . . 180
Where To Go Next . . . 180
Final Takeaway . . . 181

References **183**

Appendices **185**

A. Optuna Expert Cheat Sheet **185**
A.1. How To Use This Appendix . . . 185
A.2. 1. Basic Optimization Workflow . . . 185
A.3. 2. Parameter Suggestion API . . . 186
A.3.1. Continuous parameters . . . 186
A.3.2. Log-scale parameters . . . 186
A.3.3. Integer parameters . . . 186
A.3.4. Categorical parameters . . . 186
A.4. 3. Conditional Search Spaces . . . 186
A.5. 4. Study Configuration . . . 187
A.5.1. Default study . . . 187
A.5.2. Custom sampler . . . 187
A.5.3. Persistent storage . . . 187
A.6. 5. Common Samplers . . . 187
A.7. 6. Pruning Pattern . . . 188
A.8. 7. Best Trial Retrieval . . . 188
A.9. 8. Access All Trials . . . 189
A.10. 9. Visualization Tools . . . 189
A.10.1. Optimization history . . . 189
A.10.2. Parameter importance . . . 189
A.10.3. Slice plot . . . 189

A.10.4. Parallel coordinate . . . 190
A.10.5. Contour plots . . . 190
A.11.10. Dashboard . . . 190
A.12.11. Distributed Optimization . . . 190
A.13.12. Parallel Execution . . . 191
A.14.13. Multi-objective Optimization . . . 191
A.15.14. Reproducibility . . . 191
A.16.15. Production Setup . . . 192
A.17.16. Common Best Practices . . . 192
A.17.1. Use log scales when scale matters . . . 192
A.17.2. Avoid huge ranges without justification . . . 192
A.17.3. Use cross-validation when appropriate . . . 192
A.17.4. Use pipelines to avoid leakage . . . 192
A.17.5. Use pruning for iterative workloads . . . 193
A.18.17. Common Optimization Pattern . . . 193
A.19.18. Debugging Optimization . . . 193
A.20.19. Typical Trial Budgets . . . 193
A.21.20. Mental Model Of Optuna . . . 194
A.22.One-Screen Summary . . . 194
A.23.Final Note . . . 194

Preface

This book is a practical and theoretical guide to using Optuna as a modern black-box optimization framework. It is written for readers who already know Python, machine learning, and standard hyperparameter-search ideas, but want a deeper and more systematic understanding of how Optuna works and where it fits beyond routine model tuning.

Most introductions to Optuna stop at the mechanics of `create_study`, `trial.suggest_*`, and `study.optimize`. Those tools matter, but they are only the surface. The more important questions are structural: what kind of optimization problem are you actually solving, why do some search spaces fail, when does TPE outperform simpler baselines, how should pruning be interpreted, and what changes when optimization becomes distributed, persistent, and production-facing.

This book is organized as a progression from foundations to applications. The early chapters explain the optimization setting Optuna was built for. The middle chapters cover search-space design, samplers, pruning, diagnostics, and distributed execution. The later chapters apply those ideas to machine learning, general black-box optimization, and production workflows, before ending with a capstone portfolio in Final Project.

The intended reading style is hands-on but critical. Most chapters include runnable examples, but the goal is not only to run code successfully. The goal is to understand why a workflow is well designed, when a result should be trusted, and what trade-offs appear once optimization is used in realistic systems.

Who This Book Is For

This book assumes that you already have:

- working knowledge of Python
- familiarity with machine-learning evaluation
- basic understanding of grid search and random search
- enough mathematical maturity to follow optimization arguments and probability-based intuition

It is not written as a first introduction to machine learning or Python. It is written for practitioners who want to move from "I can tune a model with Optuna" to "I can design and justify optimization workflows across several problem classes."

What You Will Learn

By the end of the book, you should be able to:

- explain the black-box optimization setting that motivates Optuna
- understand the role of Bayesian optimization ideas and the TPE algorithm
- design effective search spaces, including conditional and hierarchical ones
- choose and interpret samplers and pruners more deliberately
- diagnose optimization behavior using visual and statistical signals
- scale studies across persistent and distributed workflows
- apply Optuna to machine learning, benchmark optimization, and simulation-based objectives
- structure optimization work so that results are reproducible and operationally credible

How To Use The Book

There are two reasonable ways to read it.

- Read it sequentially if you want the full conceptual arc from foundations to capstone.
- Read by need if you already use Optuna and want targeted guidance on topics such as Designing Search Spaces, Visualization and Diagnostics, or Production Workflows.

If you plan to implement serious optimization workflows, the strongest path is sequential reading followed by the capstone chapter. The chapters are designed to build on one another, especially from the TPE discussion onward.

A Note On Scope

Optuna is often introduced as a hyperparameter optimization library, but this book deliberately uses a wider framing. Hyperparameter tuning is one major application, not the whole story. The same Optuna abstractions can be used for simulation calibration, benchmark-function optimization, algorithm configuration, and other black-box search problems where evaluation is expensive or noisy.

That broader perspective is the organizing idea of the book.

Introduction

Optuna is one of the most effective libraries available for modern black-box optimization in Python. Many users encounter it through hyperparameter tuning, but that entry point can hide its deeper value. Optuna is not merely a convenience wrapper around search loops. It is a flexible framework for making optimization decisions under limited budgets, expensive evaluations, mixed search spaces, and uncertain objective behavior.

This book takes that broader view seriously. Its central claim is that Optuna becomes far more useful once you understand the optimization setting it was designed for, the statistical logic behind its default sampler, and the engineering discipline required to evaluate and operationalize the results.

The Problem Optuna Solves

The class of problems Optuna handles well has a recognizable structure.

You have a function or system that can be evaluated, but not easily analyzed. Each evaluation may be expensive, noisy, or stochastic. The search space may include continuous, integer, categorical, or conditional decisions. Gradients may be unavailable or irrelevant. And the total trial budget may be small enough that brute-force search is wasteful.

That is the black-box optimization setting. In practical terms, it includes problems such as:

- tuning a machine-learning pipeline under cross-validation
- calibrating a simulation to match observed behavior
- comparing algorithmic configurations under runtime constraints
- optimizing a stochastic system where every evaluation includes randomness

Optuna gives you a way to search these spaces adaptively rather than blindly.

Why This Book Exists

There are many code examples online that show how to use Optuna, but fewer resources explain the reasoning behind effective usage. In practice, mistakes often happen at a higher level than syntax.

Common failures include:

- badly designed search spaces
- overconfident interpretation of noisy results
- leakage in machine-learning evaluation

- misuse of persistent storage or distributed workers
- premature confidence in one sampler without understanding the landscape

This book is meant to close that gap. It combines theory, implementation, and workflow design so that Optuna is treated as part of an optimization methodology, not just a library call.

How The Book Is Organized

The book is divided into four parts.

Part I: Foundations

The opening modules explain what black-box optimization is, why classical exhaustive search fails, how Bayesian optimization is framed conceptually, and why TPE matters so much in Optuna's design.

Part II: Core Usage

The next part covers the mechanics that determine whether Optuna is effective in practice: studies, trials, parameter suggestion APIs, search-space design, sampler behavior, and pruning.

Part III: Advanced Techniques

This part moves into multi-objective optimization, diagnostics, distributed execution, and the deeper operational patterns required once studies become more complex or costly.

Part IV: Applications

The final part applies the earlier ideas to machine learning, general black-box optimization, production workflows, and a capstone portfolio in Final Project.

The structure is intentionally cumulative. Later chapters assume you understand not just the API, but the optimization logic behind it.

What You Should Expect From The Examples

Most important chapters include runnable examples, but the examples are intentionally small. The goal is to make the logic visible rather than hide it in a large notebook or application scaffold.

You should expect examples that are:

- short enough to read in one sitting
- self-contained when practical
- designed to illustrate one main idea at a time

- connected to realistic trade-offs rather than only happy-path usage

Where a code block is not meant to execute during the book build, it is presented as a copyable pattern rather than as a runnable notebook cell. That distinction is deliberate. The book tries to balance correctness, readability, and real workflow relevance.

A Working Mental Model

One useful way to think about Optuna is as a coordination layer around repeated experiments.

```
define search space
    -> evaluate candidate
    -> record result
    -> update search strategy
    -> repeat under budget
```

Everything else in the book refines that loop.

- The theory chapters explain why the loop should be adaptive.
- The architecture chapters explain how Optuna represents the loop internally.
- The search-space and sampler chapters explain how to make the loop effective.
- The diagnostics and production chapters explain how to make the loop trustworthy.

What "Mastery" Looks Like Here

By the end of the book, mastery does not mean memorizing every API. It means being able to answer questions such as:

- Is this a good optimization problem for Optuna?
- Is the search space well designed for the actual decision structure?
- Does the evaluation protocol measure what we claim it measures?
- Are the results robust enough to trust?
- Which sampler or pruning strategy is justified here?
- Can this workflow be resumed, inspected, and operationalized later?

That is the standard this book is written toward.

Suggested Reading Path

If you are new to serious optimization work, read the book in order. If you already use Optuna, you can still benefit from the following path:

1. Start with the foundations to align the optimization framing.
2. Read the chapters on search spaces, samplers, and pruning carefully.

3. Use the diagnostics and production chapters to harden your workflows.
4. Finish with the capstone in Final Project.

The next chapter, Foundations of Modern Optimization, starts with the fundamental question behind everything that follows: what kind of optimization problem are we really trying to solve when we reach for a tool like Optuna?

Part I.

Part I: Foundations

1. Foundations of Modern Optimization

This chapter establishes why modern optimization matters for Optuna users. Once objective evaluations become expensive, noisy, high-dimensional, or non-differentiable, the search strategy matters as much as the model itself.

By the end of the chapter, you should be able to explain what optimization problem Optuna is designed to solve, why naive search strategies break down, and why Bayesian optimization is a practical response.

1.1. Why This Matters

Suppose you are tuning a machine learning pipeline with a learning rate, tree depth, regularization strength, and feature preprocessing choices. Even before selecting a model, you already face a search problem over a mixed space of continuous, discrete, and categorical variables.

That is the setting where Optuna becomes useful: not ordinary calculus-based optimization, but efficient search over expensive black-box objectives.

1.2. Formal Definition of Optimization

In mathematical terms, optimization is the process of finding the best input parameters for a function.

Typically, we want to solve:

$$x^* = \arg\min_{x \in \mathcal{X}} f(x)$$

where:

- $f(x)$ is the objective function.
- x is the parameter vector.
- $\mathcal{X}$ is the search space.
- x^* is the optimal parameter setting.

The problem can also be written as a maximization task:

$$x^* = \arg\max_{x \in \mathcal{X}} f(x)$$

1.2.1. Example

Consider the problem of minimizing:

$$f(x) = (x - 2)^2$$

The optimal solution is:

$$x^* = 2$$

This is a simple convex function. In realistic problems, however, the objective may be non-convex, noisy, or expensive to evaluate.

In machine learning, for example, we often optimize an objective such as:

$$f(x) = \text{validation error of a model}$$

where x represents hyperparameters.

1.3. Black-Box Optimization

Many real-world optimization problems are black-box functions. In these cases:

- We cannot compute gradients.
- We do not know the analytical form of the objective.
- We can only evaluate the function at selected points.

Operationally, this means we can query $f(x)$, observe the result, and use that observation to decide what to try next.

Examples include:

Problem	Why it is black box
ML training	Complex training dynamics
Simulations	Physics or system engines
A/B experiments	Real-world user behavior
Monte Carlo simulations	Stochastic outputs

In these settings, traditional methods such as gradient descent, Newton methods, or closed-form optimization are often unavailable. Instead, we rely on sampling-based optimization.

1.4. Curse of Dimensionality

Search spaces grow exponentially with the number of parameters.

If each parameter has k possible values and we have d parameters, the total number of combinations is:

$$k^d$$

For example:

Parameters	Values each	Total combinations
5	10	100,000
10	10	10,000,000,000

This makes exhaustive search infeasible very quickly. The consequence is not just that full search becomes slow, but that most of the space is never explored at all.

Important consequences:

- Sampling efficiency becomes critical.
- Intelligent search strategies matter more than brute force.
- A poor search space definition can dominate the final result.

1.5. Grid Search vs Random Search

1.5.1. Grid Search

Grid search evaluates every combination in a predefined grid.

Example:

learning_rate	depth
0.01	3
0.01	6
0.1	3
0.1	6

Advantages:

- Simple.
- Deterministic.
- Easy to parallelize.

Disadvantages:

- Scales exponentially.
- Becomes inefficient in high dimensions.
- Wastes evaluations on unimportant parameters.

1.5.2. Random Search

Random search samples parameters independently from the search space.

```
x1 = random()
x2 = random()
x3 = random()
...
```

The key practical insight is that random search often outperforms grid search when only a subset of parameters strongly affects performance. Grid search spends many evaluations covering dimensions that may not matter much, while random search covers more distinct combinations of the important ones.

1.6. Sample Efficiency

When evaluations are cheap, brute-force methods can be acceptable. But many real objectives are expensive:

Task	Evaluation cost
ML training	Minutes
Deep learning	Hours
Simulation	Minutes
Scientific experiment	Days

If we can only afford a small budget, each evaluation should be as informative as possible. This is the idea of sample efficiency: finding strong solutions with as few objective evaluations as possible.

Random search does not use the results of earlier trials. Modern optimizers do.

1.7. Exploration vs Exploitation

Every optimizer must balance two competing goals.

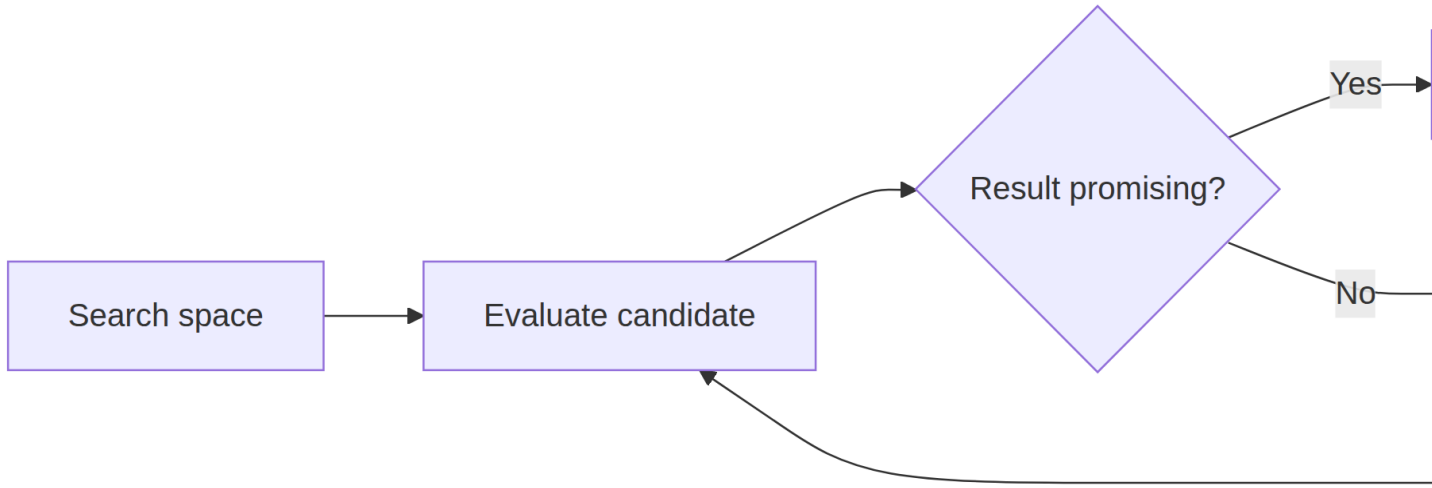
1.7.1. Exploration

Explore unknown regions of the search space to discover potentially better areas.

1.7.2. Exploitation

Search near known good solutions to refine promising regions.

Too much exploration slows convergence. Too much exploitation can trap the search in a local optimum. Practical optimization algorithms succeed by balancing these two behaviors dynamically.



1.8. When Gradient-Based Methods Fail

Gradient-based methods work well when the objective is differentiable, gradients are available, and the optimization landscape is well behaved. Those assumptions fail in many real applications.

Problem	Why gradients fail
Discrete parameters	Gradients are undefined
Simulation-based models	Gradients are unavailable
Non-differentiable functions	Discontinuities
Noisy objectives	Gradients become unstable

Examples of non-gradient-friendly choices include:

- Number of trees.
- Optimizer type.
- Activation function.

These are categorical or discrete decisions, so gradient descent is not the right tool.

> ⚠ Common Mistake
>
> Treating hyperparameter tuning as if the objective were smooth, convex, and differentiable leads to the wrong optimization strategy. In Optuna-style workloads, the objective is often mixed, noisy, and black box by construction.

1.9. Why Bayesian Optimization Works

Bayesian optimization is designed for expensive black-box functions. Instead of optimizing the true objective directly, it builds a surrogate model:

$$\hat{f}(x)$$

The surrogate approximates the true objective using past observations. An acquisition function then decides which point should be evaluated next.

At a high level, the process is:

1. Evaluate several candidate points.
2. Fit a probabilistic model from the observed results.
3. Estimate which regions look promising or uncertain.
4. Evaluate the next candidate selected by the acquisition rule.
5. Update the model.
6. Repeat.

Advantages:

- Uses information from previous trials.
- Balances exploration and exploitation.
- Reaches strong solutions in fewer evaluations.

That is why Bayesian optimization is widely used in hyperparameter tuning, robotics, engineering design, and reinforcement learning.

1.10. Where Optuna Fits

Optuna implements a modern Bayesian optimization workflow, with TPE (Tree-structured Parzen Estimator) as its default sampler.

Compared with Gaussian-process-based approaches, TPE is especially practical because it:

- Scales better to higher dimensions.
- Handles categorical variables naturally.
- Works well with conditional search spaces.

We will study this algorithm in more detail in The TPE Algorithm.

1.11. Practical Exercise: Implement Random Search

As a first exercise, implement a simple optimizer for the function:

$$f(x) = (x - 3)^2 + \sin(5x)$$

```
import math
import random


def objective(x):
    return (x - 3) ** 2 + math.sin(5 * x)


best_x = None
best_y = float("inf")

for _ in range(1000):
    x = random.uniform(-5, 5)
    y = objective(x)

    if y < best_y:
        best_y = y
        best_x = x

print(f"Best x: {best_x:.4f}")
print(f"Best value: {best_y:.4f}")
```

```
Best x: 3.4287
Best value: -0.8071
```

This optimizer does three things:

1. Samples random points.
2. Evaluates the objective.
3. Keeps the best solution found so far.

Its limitations are exactly what motivate Optuna: it ignores previous evaluations and does not adapt to what it learns.

Try extending the exercise in three steps:

1. Reduce the evaluation budget from 1000 to 50 and compare the result.
2. Change the search interval and observe how sensitive random search is to space design.
3. Rewrite the objective in two dimensions and see how quickly brute-force search degrades.

1.12. Recap

Modern optimization in machine learning and data science often involves:

- Expensive black-box functions.
- High-dimensional search spaces.
- Discrete or non-differentiable parameters.

That combination makes classical exhaustive or gradient-based methods a poor default. Bayesian optimization improves sample efficiency by learning from prior evaluations, and Optuna is designed to make that workflow practical.

1.13. Decision Guide

Use the following rule of thumb:

- Use grid search only for very small, fully discrete spaces.
- Use random search as a strong baseline when evaluations are cheap.
- Use Bayesian optimization when evaluations are expensive and each trial should teach the optimizer something.

Next, we will make those ideas precise in Bayesian Optimization Fundamentals.

2. Bayesian Optimization Fundamentals

This chapter introduces the core logic behind Bayesian optimization: build a probabilistic model of an expensive objective, estimate where information is most valuable, and decide where to evaluate next. That is the conceptual bridge between the search intuition in Foundations of Modern Optimization and Optuna's default TPE-based workflow.

By the end of the chapter, you should be able to explain the role of surrogate models, why acquisition functions are needed, and why classical Gaussian-process-based Bayesian optimization is both powerful and limited.

This chapter assumes you are already comfortable with the optimization vocabulary from Foundations of Modern Optimization, along with basic probability concepts such as mean, variance, and normal distributions.

2.1. Why This Matters

Suppose each hyperparameter trial for a large model takes two hours. You cannot afford to evaluate hundreds or thousands of random configurations just to discover where the promising region might be. You need a strategy that treats each evaluation as expensive and uses prior results to make better decisions.

That is the problem Bayesian optimization is built to solve.

2.2. A Fast First Win

Before discussing Gaussian processes formally, it helps to see what the optimizer is trying to do. In Bayesian optimization, each candidate point gets a predicted mean, a predicted uncertainty, and an acquisition score that decides whether it is worth evaluating.

The following example computes Expected Improvement for a few candidate points using only the Python standard library.

```
import math


def normal_pdf(z):
    return math.exp(-0.5 * z * z) / math.sqrt(2 * math.pi)


def normal_cdf(z):
```

```
    return 0.5 * (1 + math.erf(z / math.sqrt(2)))


def expected_improvement(mu, sigma, f_best):
    if sigma <= 1e-12:
        return max(f_best - mu, 0.0)
    z = (f_best - mu) / sigma
    return (f_best - mu) * normal_cdf(z) + sigma * normal_pdf(z)


f_best = 0.18
candidates = [
    {"x": 0.5, "mu": 0.30, "sigma": 0.05},
    {"x": 1.0, "mu": 0.24, "sigma": 0.12},
    {"x": 1.5, "mu": 0.28, "sigma": 0.30},
    {"x": 2.0, "mu": 0.16, "sigma": 0.02},
]

for candidate in candidates:
    candidate["ei"] = expected_improvement(
        candidate["mu"],
        candidate["sigma"],
        f_best,
    )

best_candidate = max(candidates, key=lambda candidate: candidate["ei"])

for candidate in candidates:
    print(candidate)

print("Next point to evaluate:", best_candidate)
```

```
{'x': 0.5, 'mu': 0.3, 'sigma': 0.05, 'ei': 0.00013602220379060653}
{'x': 1.0, 'mu': 0.24, 'sigma': 0.12, 'ei': 0.023735586888156728}
{'x': 1.5, 'mu': 0.28, 'sigma': 0.3, 'ei': 0.07627083428972159}
{'x': 2.0, 'mu': 0.16, 'sigma': 0.02, 'ei': 0.021666309411753716}
Next point to evaluate: {'x': 1.5, 'mu': 0.28, 'sigma': 0.3, 'ei': 0.07627083428972159}
```

The key observation is that the best next point is not always the one with the lowest predicted mean. A point with slightly worse predicted performance may still be more valuable if its uncertainty is high enough.

2.3. Why Surrogate Models Matter

Bayesian optimization starts from a simple idea: instead of optimizing the true objective function directly, we build an approximate probabilistic model from the observations we already have.

Let the true objective be:

$$f(x)$$

We build a surrogate model:

$$\hat{f}(x)$$

using observed data:

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

where:

$$y_i = f(x_i)$$

The purpose of the surrogate is not just to estimate the function value. It must also estimate uncertainty. In other words, for each candidate point x, we want a predictive distribution:

$$p(f(x) \mid D)$$

That uncertainty estimate is what makes exploration possible.

2.4. Gaussian Processes: The Classical View

The most classical surrogate model in Bayesian optimization is the Gaussian process, usually abbreviated GP.

A Gaussian process defines a distribution over functions rather than a single best-fit curve. Formally:

$$f(x) \sim \mathcal{GP}(m(x), k(x, x'))$$

where:

- $m(x)$ is the mean function.
- $k(x, x')$ is the covariance kernel.

The mean function captures the expected value of the objective, and the kernel encodes similarity between inputs. If two inputs are close according to the kernel, the GP expects their outputs to be similar as well.

Common kernels include:

- Radial basis function (RBF).

- Matern.
- Rational quadratic.

The intuition is more important than the formulas at this stage: a Gaussian process gives you both a prediction and a measure of confidence in that prediction.

2.5. What Does a GP Predict?

At a candidate point x, a GP provides:

- A predictive mean:

$$\mu(x)$$

- A predictive variance:

$$\sigma^2(x)$$

So the prediction can be summarized as:

$$f(x) \sim \mathcal{N}(\mu(x), \sigma^2(x))$$

This is the core output that drives Bayesian optimization. The optimizer now has two pieces of information:

- What value looks plausible at this point.
- How uncertain that estimate is.

2.6. How the Acquisition Function Chooses the Next Point

The surrogate model tells us what we currently believe about the objective. The acquisition function tells us what to do next.

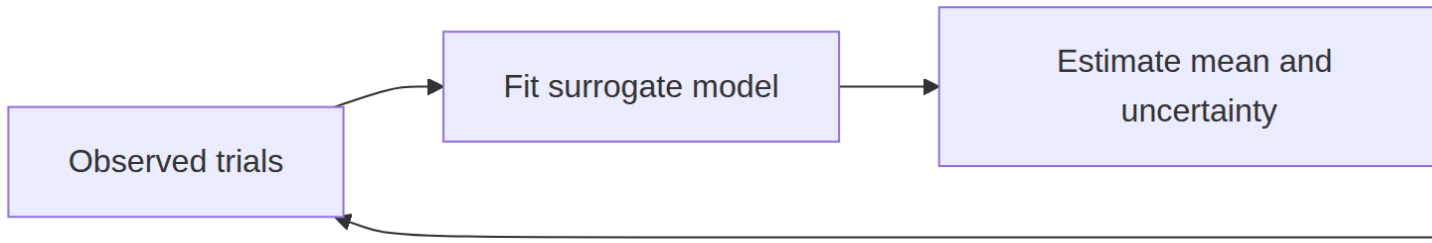
In general, the next evaluation point is chosen by solving:

$$x_{\text{next}} = \arg\max a(x)$$

where $a(x)$ is the acquisition function.

Acquisition functions encode the exploration-exploitation tradeoff. They prefer points that either:

- Look promising according to the predicted mean.
- Remain uncertain enough that evaluating them could reveal something better.



2.7. Probability of Improvement

Probability of Improvement, or PI, is the simplest acquisition function.

For minimization, let f^* denote the best observed value so far. Then PI measures the probability that a candidate point beats that value:

$$PI(x) = P(f(x) < f^*)$$

Its interpretation is straightforward: rank points by how likely they are to improve the current best.

The limitation is equally straightforward: PI often over-exploits. A point that is only marginally better but very certain can dominate a riskier point that could lead to a much larger improvement.

2.8. Expected Improvement

Expected Improvement, or EI, fixes the main weakness of PI by asking not only whether improvement is likely, but how large that improvement might be.

For minimization:

$$EI(x) = \mathbb{E}[\max(f^* - f(x), 0)]$$

This combines:

- Low predicted values.
- High uncertainty.

That balance is why EI became one of the standard acquisition rules in Bayesian optimization.

The fast example at the start of the chapter is an explicit EI calculation, and it shows the core idea: the next point is chosen for decision value, not just predicted objective value.

2.9. Upper and Lower Confidence Bounds

Another family of acquisition functions uses confidence bounds directly.

For maximization:

$$UCB(x) = \mu(x) + \kappa\sigma(x)$$

For minimization, the analogous lower confidence bound is:

$$LCB(x) = \mu(x) - \kappa\sigma(x)$$

where:

- $\mu(x)$ is the predicted mean.
- $\sigma(x)$ is the predictive standard deviation.
- κ controls how aggressively the optimizer explores.

Large κ favors exploration. Small κ favors exploitation. This makes confidence-bound strategies attractive when you want a direct knob for controlling search behavior.

2.10. The Bayesian Optimization Loop

Putting the pieces together, the classical Bayesian optimization loop is:

1. Sample a small set of initial points.
2. Fit a surrogate model such as a Gaussian process.
3. Compute an acquisition function over the search space.
4. Select the point that maximizes the acquisition score.
5. Evaluate the true objective.
6. Update the model with the new observation.
7. Repeat.

This loop works because the optimizer spends evaluations where they are expected to be most informative, not merely where the search space happens to be easy to enumerate.

> ⚠ Common Mistake
>
> Do not confuse optimizing the surrogate mean with performing Bayesian optimization. If you always choose the point with the best predicted mean, you remove the exploration term and the method can get trapped in regions that only look good under the current model.

2.11. Where Gaussian-Process Bayesian Optimization Struggles

Gaussian-process-based Bayesian optimization is elegant, but it has practical limits.

2.11.1. Computational Complexity

Training a Gaussian process typically scales as:

$$O(n^3)$$

where n is the number of observations. That becomes expensive once the number of evaluations grows.

2.11.2. High-Dimensional Search Spaces

Gaussian processes usually work best in relatively low-dimensional settings. Once the search space contains many interacting hyperparameters, the model quality and acquisition optimization can degrade quickly.

As a rough rule of thumb, performance often becomes harder to maintain once the effective search dimension grows well beyond the small-scale regime.

2.11.3. Categorical and Conditional Variables

Many machine learning search spaces include categorical or tree-structured choices such as:

```
optimizer = ["adam", "sgd", "rmsprop"]
```

This is awkward for classical GP kernels, which are more naturally defined over continuous spaces.

2.12. Why Optuna Transitions to TPE Instead

Optuna does not use Gaussian processes as its default strategy. Instead, it uses the Tree-structured Parzen Estimator, or TPE.

The conceptual shift is important. Classical Bayesian optimization often models:

$$p(y \mid x)$$

TPE instead models:

$$p(x \mid y)$$

It separates the observed trials into good and bad regions, estimates densities for each group, and searches for points that are more likely under the good-trial density than under the bad-trial density.

This leads to an optimization criterion of the form:

$$\arg\max \frac{\ell(x)}{g(x)}$$

where:

- $\ell(x)$ is the density of good observations.
- $g(x)$ is the density of bad observations.

This reformulation is one reason TPE is practical for Optuna. It handles categorical variables better, scales more gracefully, and works well in higher-dimensional or conditional search spaces.

We will unpack the full algorithm in The TPE Algorithm.

2.13. Practical Exercise: Compare Mean and Improvement

Use the earlier Expected Improvement example as a starting point and modify it in three steps:

1. Change `f_best` and observe how the acquisition ranking changes.
2. Increase the uncertainty of one candidate and check when exploration becomes dominant.
3. Add a new candidate with a very low mean and very low uncertainty, then compare it against a riskier candidate with higher uncertainty.

Your goal is to build intuition for why acquisition functions do not behave like plain objective minimization.

2.14. Recap

Bayesian optimization combines three components:

- A surrogate model that approximates the unknown objective.
- An uncertainty estimate that makes exploration possible.
- An acquisition function that selects the next evaluation point.

Gaussian processes provide the classical picture of this workflow, and they remain useful as a conceptual foundation. But the same chapter also explains why Optuna's default strategy moves away from GP-based optimization in practice.

2.15. Decision Guide

Use the following mental model:

- Use Bayesian optimization when each objective evaluation is expensive enough that learning from earlier trials matters.
- Use Gaussian-process intuition to understand surrogate modeling and acquisition functions.
- Expect GP-based methods to struggle once the space becomes highly dimensional, categorical, or heavily conditional.

- Expect Optuna's TPE-based approach to be a better practical default for many real hyperparameter-search workloads.

Next, The TPE Algorithm turns that transition into a concrete optimization method.

3. The TPE Algorithm

This chapter explains the algorithm Optuna uses by default: the Tree-structured Parzen Estimator, or TPE. Instead of fitting a regression model for the objective as in classical Gaussian-process Bayesian optimization, TPE models densities over parameter values conditioned on whether past trials were good or bad.

By the end of the chapter, you should be able to explain why TPE models $p(x \mid y)$ rather than $p(y \mid x)$, how the good-trial and bad-trial densities are constructed, and why that design works well for Optuna's flexible search spaces.

This chapter assumes you already understand surrogate models and acquisition functions from Bayesian Optimization Fundamentals.

3.1. Why This Matters

Suppose you are tuning a real machine-learning pipeline with continuous hyperparameters, categorical choices, and conditional branches such as model-specific parameters. A Gaussian-process surrogate becomes awkward quickly, both computationally and structurally.

TPE matters because it keeps the central idea of Bayesian optimization, learning from prior trials, while replacing the classical GP machinery with something that works much better in the messy search spaces common in Optuna workloads.

3.2. A Fast First Win

Before looking at the full algorithm, it helps to see the core ranking rule in action. TPE divides past observations into good and bad trials, estimates a density for each group, and then scores candidates by how much more they resemble good trials than bad ones.

The following pure-Python example implements a tiny one-dimensional TPE-like ranking loop using Gaussian kernels. It is not a production implementation, but it captures the mechanism that matters conceptually.

```
import math


def objective(x):
    return (x - 2.0) ** 2 + math.sin(3.0 * x)
```

```python
def quantile(values, q):
    ordered = sorted(values)
    index = int(q * (len(ordered) - 1))
    return ordered[index]


def gaussian_kernel(x, center, bandwidth):
    z = (x - center) / bandwidth
    return math.exp(-0.5 * z * z) / (bandwidth * math.sqrt(2.0 * math.pi))


def kde(points, x, bandwidth=0.35):
    if not points:
        return 1e-12
    return sum(gaussian_kernel(x, point, bandwidth) for point in points) /
     ↪  len(points)


observations = []
for x in [-4.0, -2.5, -1.0, 0.0, 1.0, 1.8, 2.4, 3.2, 4.0]:
    observations.append((x, objective(x)))

gamma = 0.3
threshold = quantile([y for _, y in observations], gamma)
good = [x for x, y in observations if y < threshold]
bad = [x for x, y in observations if y >= threshold]

candidates = [round(-4.0 + 0.25 * i, 2) for i in range(33)]
scored = []

for x in candidates:
    l_x = kde(good, x)
    g_x = kde(bad, x)
    ratio = l_x / max(g_x, 1e-12)
    scored.append((ratio, x, l_x, g_x))

best_ratio, best_x, _, _ = max(scored)

print("Good trials:", good)
print("Bad trials:", bad)
print("Suggested next x:", best_x)
print("Best density ratio:", round(best_ratio, 4))
```

```
Good trials: [1.8, 2.4]
Bad trials: [-4.0, -2.5, -1.0, 0.0, 1.0, 3.2, 4.0]
Suggested next x: 2.0
Best density ratio: 243.5948
```

The important point is not the exact numerical value of the ratio. The point is that TPE selects candidates that look statistically similar to good trials while remaining dissimilar to bad ones.

3.3. Why TPE Flips the Bayesian Optimization View

Classical Bayesian optimization typically models:

$$p(y \mid x)$$

That asks: given a parameter setting x, what objective value y should we expect?

TPE flips the problem and models:

$$p(x \mid y)$$

That asks: which parameter values tend to be associated with good or bad objective values?

This is a major conceptual shift. Instead of learning a full regression model for the objective surface, TPE turns optimization into a density-estimation problem.

3.4. Splitting Observations into Good and Bad Trials

Assume we have already evaluated n trials:

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$$

TPE defines a threshold y^* such that the best fraction of observations is treated as the good region. If we denote that fraction by γ, then the good set is usually the best 10% to 25% of trials.

Typical values are:

$$\gamma \approx 0.1 \text{ to } 0.25$$

For a minimization problem, we then split the trials into:

- Good observations, where $y < y^*$.
- Bad observations, where $y \geq y^*$.

That split is what allows TPE to build two separate models of the search space.

3.5. The Two Density Models

TPE estimates two densities:

$$\ell(x) = p(x \mid y < y^*)$$

$$g(x) = p(x \mid y \geq y^*)$$

Interpretation:

Density	Meaning
$\ell(x)$	Parameter values associated with good objective values
$g(x)$	Parameter values associated with worse objective values

The optimizer then searches for candidates that score well under the good density and poorly under the bad density.

3.6. The Likelihood-Ratio Rule

The next candidate is chosen by maximizing a density ratio of the form:

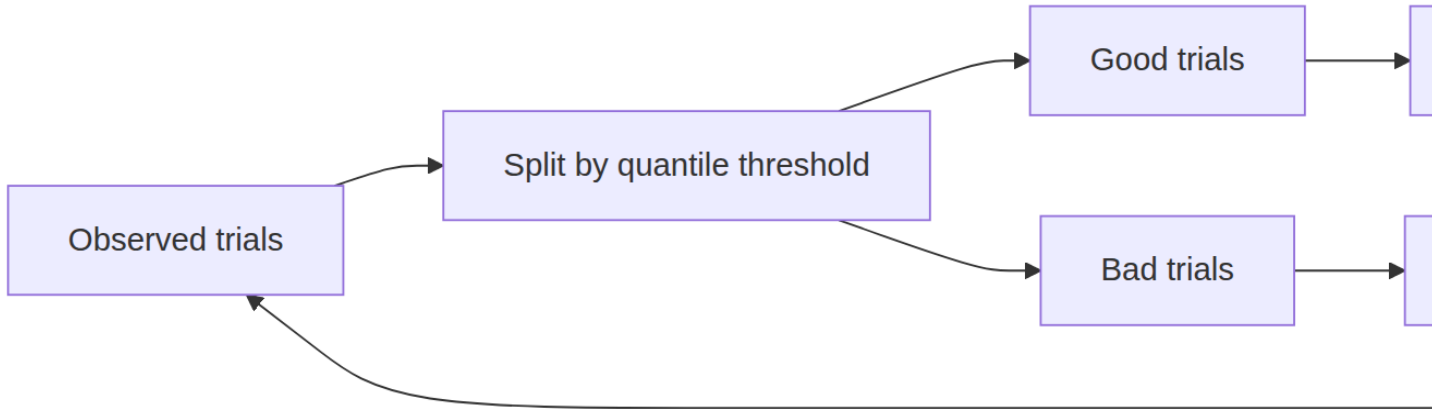
$$\frac{\ell(x)}{g(x)}$$

This plays a role similar to an acquisition function in classical Bayesian optimization.

Candidates with a high ratio are appealing because they:

- Resemble parameter settings that produced strong results.
- Avoid regions already associated with weak results.

This is the core decision rule behind TPE.



3.7. How Parzen Estimators Build the Densities

The density estimates in TPE are typically constructed using Parzen estimators, which are a form of kernel density estimation.

Given samples $x_1, x_2, \dots, x_n$, a kernel density estimate can be written as:

$$p(x) = \frac{1}{n} \sum_{i=1}^{n} K(x - x_i)$$

where K is a kernel function, often Gaussian in simple explanations.

The intuition is straightforward: each observed value contributes a smooth local bump, and the sum of those bumps forms a continuous approximation of the density.

In practice, TPE applies this idea parameter by parameter and uses a design that works well with continuous, discrete, and categorical search spaces.

3.8. Where Exploration Comes From in TPE

TPE still balances exploration and exploitation, but it does so differently from GP-based acquisition functions.

3.8.1. Density Ratio

The ratio

$$\frac{\ell(x)}{g(x)}$$

pushes the search toward promising regions.

3.8.2. Candidate Sampling

TPE does not optimize the ratio analytically over the full search space. Instead, it samples many candidates and ranks them. That sampling process preserves some exploration even when the model has strong preferences.

3.8.3. Quantile Threshold

The value of γ controls how selective the good set is. A lower γ makes the good set smaller and more exploitative. A higher γ broadens the good set and tends to encourage more exploration.

A typical value is:

$$\gamma = 0.15$$

3.9. The Full TPE Procedure

A simplified TPE workflow looks like this:

1. Sample an initial set of random trials.
2. Evaluate the objective.
3. Split the observed trials into good and bad sets.
4. Estimate $\ell(x)$ and $g(x)$.
5. Sample candidate points from the promising region.
6. Score candidates using $\ell(x)/g(x)$.
7. Evaluate the best-scoring candidate.
8. Update the observed trial set.
9. Repeat.

The crucial detail is that TPE samples many candidates and chooses among them, rather than solving a closed-form acquisition optimization problem.

> ⚠ Common Mistake
>
> Do not interpret TPE as directly estimating the optimum or directly predicting the objective value. TPE is ranking candidates through density comparison. That is a different kind of model from GP regression, and misunderstanding that difference often leads to confused explanations of how Optuna's sampler works.

3.10. Why TPE Works Well in Practice

TPE has several practical advantages over Gaussian-process-based Bayesian optimization.

3.10.1. High-Dimensional Spaces

Gaussian processes tend to become harder to manage as the search space grows in dimension and interaction complexity. TPE is often much more robust in problems with dozens of hyperparameters.

3.10.2. Categorical Variables

TPE handles categorical parameters naturally, which is a major advantage in real hyperparameter tuning.

For example, an Optuna search space may contain a categorical branch such as:

```
optimizer = trial.suggest_categorical(
    "optimizer",
    ["adam", "sgd", "rmsprop"],
)
```

3.10.3. Tree-Structured Search Spaces

TPE is especially well suited for conditional search spaces, which is where the "tree-structured" part of its name becomes practical.

```
model = trial.suggest_categorical("model", ["xgboost", "svm"])

if model == "xgboost":
    depth = trial.suggest_int("depth", 3, 10)
```

This kind of conditional structure is common in Optuna studies and awkward for classical GP formulations.

3.10.4. Better Scalability

In broad terms, TPE tends to scale much more gracefully than Gaussian-process approaches, whose training cost can become prohibitive as the number of observations grows.

3.11. Why Optuna Uses TPE by Default

Optuna is designed around three practical goals:

- Strong performance in hyperparameter optimization.
- Flexible search spaces.
- Scalability to realistic workloads.

TPE aligns well with all three, which is why Optuna uses it as the default sampler.

That does not mean TPE is the only valid strategy. Optuna also supports alternatives such as random search, CMA-ES, and NSGA-II, which we will revisit later in the book.

3.12. Practical Exercise: Implement a Simplified TPE Loop

Extend the fast example at the top of the chapter in four steps:

1. Add a loop so the newly suggested point is appended to the observed trials.
2. Recompute the good and bad split after each new evaluation.
3. Change the value of `gamma` and observe how the search behavior shifts.
4. Replace the candidate grid with random candidate sampling and compare the selected points.

The goal is not to reproduce Optuna's production sampler exactly. The goal is to internalize the density-ratio idea that drives the algorithm.

3.13. Recap

TPE reformulates Bayesian optimization as density estimation.

Instead of modeling:

$$p(y \mid x)$$

it models:

$$p(x \mid y)$$

This leads to two densities, $\ell(x)$ and $g(x)$, and a candidate-selection rule based on their ratio. That design is one reason Optuna can handle categorical variables, conditional spaces, and larger search problems more naturally than many GP-based approaches.

3.14. Decision Guide

Use the following mental model:

- Think of TPE as Bayesian optimization through density comparison rather than surrogate regression.
- Use it when the search space includes categorical or conditional choices.
- Expect it to be a strong default for practical Optuna studies.
- Do not expect it to provide the same style of uncertainty model as a Gaussian process.

Next, Optuna Architecture connects this optimization logic to the concrete building blocks of an Optuna study: studies, trials, samplers, pruners, and storage.

4. Optuna Architecture

This chapter explains how Optuna is structured internally and how its main components cooperate during optimization. Once you understand the architecture, the library becomes easier to reason about: a study manages trials, a sampler proposes parameters, a pruner decides whether a trial should continue, and storage records the whole process.

By the end of the chapter, you should be able to describe the role of the study, trial, objective function, search space, sampler, pruner, and storage backend, and explain how they interact inside Optuna's optimization loop.

This chapter assumes you already understand the optimization ideas behind The TPE Algorithm. Here the focus shifts from theory to the actual objects Optuna exposes.

4.1. Why This Matters

Many Optuna users start by copying a short example and calling `study.optimize(...)`. That is enough to get results, but not enough to debug failures, interpret trial states, reason about pruning, or choose the right storage backend once experiments become more serious.

Understanding the architecture matters because Optuna is not just a function minimizer. It is a coordination layer for optimization logic, experiment state, and execution flow.

4.2. A Fast First Win

The quickest way to see the architecture is to run a tiny study and inspect the central objects it produces.

```
import optuna


def objective(trial):
    x = trial.suggest_float("x", -10, 10)
    return (x - 2) ** 2


study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=8)

print("Best params:", study.best_params)
```

```
print("Best value:", round(study.best_value, 4))
print("Number of trials:", len(study.trials))
print("First trial state:", study.trials[0].state)
```

```
[I 2026-03-15 10:35:18,326] A new study created in memory with name: no-name-d6700ae5-9ab4-4b7
[I 2026-03-15 10:35:18,335] Trial 0 finished with value: 25.89167288328001 and parameters: {'x
[I 2026-03-15 10:35:18,337] Trial 1 finished with value: 48.749069327526215 and parameters: {'
[I 2026-03-15 10:35:18,339] Trial 2 finished with value: 5.093268726504466 and parameters: {'x
[I 2026-03-15 10:35:18,340] Trial 3 finished with value: 91.00744739048214 and parameters: {'x
[I 2026-03-15 10:35:18,341] Trial 4 finished with value: 49.51696101403246 and parameters: {'x
[I 2026-03-15 10:35:18,341] Trial 5 finished with value: 48.346398662645115 and parameters: {'
[I 2026-03-15 10:35:18,342] Trial 6 finished with value: 84.12215050212238 and parameters: {'x
[I 2026-03-15 10:35:18,343] Trial 7 finished with value: 58.284490104741735 and parameters: {'
```

```
Best params: {'x': 4.256827137045384}
Best value: 5.0933
Number of trials: 8
First trial state: TrialState.COMPLETE
```

This small example already exercises the core architecture:

- A `Study` manages the experiment.
- Each `Trial` evaluates one parameter configuration.
- The objective function defines the problem.
- The sampler proposes candidate parameter values.
- Results are stored and exposed through the study object.

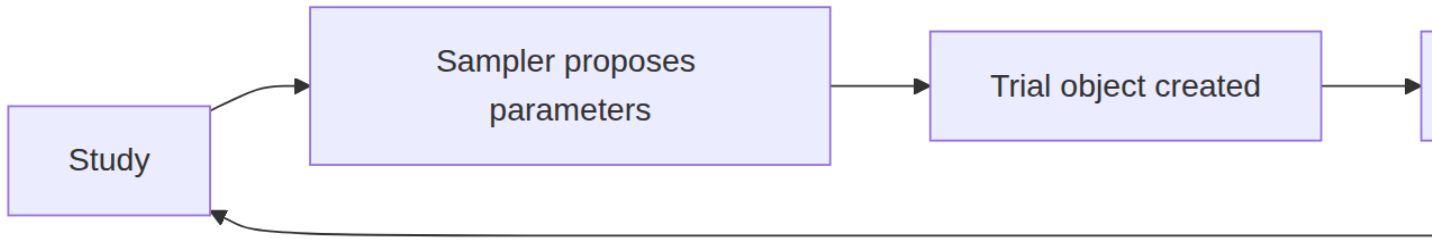
4.3. Architecture at a Glance

An Optuna optimization run consists of several interacting components:

```
Study
    Trials
          Parameters
          Objective value
          Intermediate results

    Sampler
    Pruner
    Storage
```

The `Study` is the central coordination object. Most of Optuna's public API is organized around it.



4.4. The Study Object

A `Study` represents one complete optimization experiment.

It manages:

- All trials in the experiment.
- The sampler.
- The pruner.
- The storage backend.
- The optimization direction.

Creating a study is usually the first Optuna step:

```
import optuna

study = optuna.create_study(direction="minimize")
print(type(study).__name__)
```

```
[I 2026-03-15 10:35:18,359] A new study created in memory with name: no-name-2125a483-2733-4ef
```

```
Study
```

You can think of the study as the experiment manager. It schedules trials, collects results, tracks the best outcome, and exposes the overall state of the optimization.

Important study operations include:

```
study.optimize(objective, n_trials=100)
study.best_params
study.best_value
study.best_trial
study.trials
```

4.5. The Trial Object

A `Trial` represents one evaluation of the objective function.

Each trial stores:

- Parameter values.
- The final objective value.
- Intermediate values reported during training or iteration.
- Metadata such as state and timestamps.

Conceptually, the flow looks like this:

```
Trial 1 -> parameters -> objective -> result
Trial 2 -> parameters -> objective -> result
Trial 3 -> parameters -> objective -> result
```

Trials are created and managed by the study, but the objective function interacts with them directly.

4.5.1. Trial lifecycle

```
CREATED
  ↓
RUNNING
  ↓
COMPLETE / PRUNED / FAILED
```

Common states include:

State	Meaning
`COMPLETE`	Trial finished successfully
`PRUNED`	Trial was stopped early
`FAILED`	Trial ended with an error

4.6. The Objective Function

The objective function defines the optimization problem. It receives a `Trial` object and returns the scalar value to minimize or maximize.

```python
def objective(trial):
    x = trial.suggest_float("x", -10, 10)
    return (x - 2) ** 2
```

Inside the objective function, you usually do three things:

1. Define parameters through `trial.suggest_*` methods.
2. Evaluate the model, simulation, or function.
3. Return the result.

That returned result becomes the stored trial value.

4.7. The Search Space and Define-by-Run

The search space is the set of parameters Optuna is allowed to explore.

Optuna uses a define-by-run API, which means the search space is created dynamically during execution rather than declared in a separate static schema.

For example:

```
def objective(trial):
    x = trial.suggest_float("x", -5, 5)
    depth = trial.suggest_int("depth", 3, 10)
    optimizer = trial.suggest_categorical("optimizer", ["adam", "sgd"])
    lr = trial.suggest_float("lr", 1e-5, 1e-1, log=True)
    return x**2
```

This approach is powerful because it supports dynamic and conditional spaces naturally.

```
def objective(trial):
    model = trial.suggest_categorical("model", ["svm", "rf"])

    if model == "rf":
        depth = trial.suggest_int("depth", 3, 10)

    return 0.0
```

That is exactly the kind of tree-structured space where TPE is especially useful, which is why this chapter connects directly back to The TPE Algorithm.

4.8. Samplers

The sampler decides how Optuna chooses new parameter values.

In other words, the sampler implements the optimization strategy.

Examples include:

Sampler	Method
`TPESampler`	Bayesian-style density estimation
`RandomSampler`	Random search
`CmaEsSampler`	Evolutionary strategy
`GridSampler`	Exhaustive grid search

You can pass a sampler explicitly when creating a study:

```
study = optuna.create_study(
    direction="minimize",
    sampler=optuna.samplers.TPESampler(),
)

print(type(study.sampler).__name__)
```

```
[I 2026-03-15 10:35:18,407] A new study created in memory with name: no-name-960d99b2-cc52-4e3
```

```
TPESampler
```

If you do not specify one, Optuna uses TPE by default for many standard single-objective optimization tasks.

4.9. Pruners

Pruners implement early stopping. They decide whether a trial is unlikely to become competitive and should be terminated before finishing.

This is especially useful when objective evaluations are expensive and produce intermediate progress signals, such as training loss by epoch.

Typical use cases include:

- Deep learning training.
- Boosting methods.
- Iterative simulations.

Here is the basic pattern:

```
def objective(trial):
    x = trial.suggest_float("x", -5, 5)

    for step in range(5):
        value = (x - 2) ** 2 + 1 / (step + 1)
        trial.report(value, step)
```

```
        if trial.should_prune():
            raise optuna.TrialPruned()

    return value
```

Popular pruners include:

Pruner	Strategy
`MedianPruner`	Stops trials worse than the historical median
`SuccessiveHalvingPruner`	Resource-based elimination
`HyperbandPruner`	Multi-bracket early stopping

We will study pruning in depth later, but architecturally the main point is simple: the pruner depends on intermediate reports from the trial.

4.10. Storage Backends

Optuna supports both in-memory and persistent storage.

Storage matters because it enables:

- Experiment tracking.
- Fault tolerance.
- Parallel or distributed optimization.

4.10.1. In-memory storage

```
study = optuna.create_study(direction="minimize")
```

```
[I 2026-03-15 10:35:18,436] A new study created in memory with name: no-name-0bd33160-d8dd-443
```

This is convenient for quick experiments, but the results disappear when the process ends.

4.10.2. SQLite storage

```
study = optuna.create_study(
    direction="minimize",
    storage="sqlite:///example.db",
)
```

This persists the study in a local database file.

4.10.3. Server databases

For larger or distributed workloads, Optuna can use databases such as PostgreSQL or MySQL.

```
storage = "postgresql://user:pass@host/db"
```

This allows multiple worker processes or machines to coordinate through shared study storage.

4.11. The Internal Optimization Loop

At a high level, Optuna repeatedly runs a loop like this:

```
while trials_remaining:
    sampler -> propose parameters
    trial -> run objective
    record result
    pruner -> check early stopping
    update sampler
```

A more detailed view is:

```
Study starts
   ↓
Sampler suggests parameters
   ↓
Trial object created
   ↓
Objective function executed
   ↓
Intermediate results reported
   ↓
Pruner may stop trial
   ↓
Result stored
   ↓
Sampler updates model
```

This repeats until a stopping condition is reached, such as a target number of trials, a timeout, or an external stop.

> ⚠ Common Mistake
>
> Do not think of the sampler, trial, and study as separate convenience features. They are parts of one coordinated system. Confusion usually appears when users try to reason about trial behavior without understanding which decisions belong to the study, which belong to

> the sampler, and which must happen inside the objective function.

4.12. A Minimal End-to-End Example

The following example exercises the whole architecture in a compact form:

```
import optuna


def objective(trial):
    x = trial.suggest_float("x", -5, 5)
    return (x - 2) ** 2


study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=20)

print("Best params:", study.best_params)
print("Best value:", round(study.best_value, 4))
```

```
[I 2026-03-15 10:35:18,450] A new study created in memory with name: no-name-462debc6-4bb0-479
[I 2026-03-15 10:35:18,453] Trial 0 finished with value: 4.92506520682731 and parameters: {'x'
[I 2026-03-15 10:35:18,455] Trial 1 finished with value: 9.615411320450013 and parameters: {'x
[I 2026-03-15 10:35:18,456] Trial 2 finished with value: 22.946631433084782 and parameters: {'
[I 2026-03-15 10:35:18,457] Trial 3 finished with value: 2.237712306004863 and parameters: {'x
[I 2026-03-15 10:35:18,458] Trial 4 finished with value: 4.829937427437122 and parameters: {'x
[I 2026-03-15 10:35:18,459] Trial 5 finished with value: 1.0899306592443323 and parameters: {'
[I 2026-03-15 10:35:18,461] Trial 6 finished with value: 0.6569417146067965 and parameters: {'
[I 2026-03-15 10:35:18,462] Trial 7 finished with value: 0.04484779905380006 and parameters: {
[I 2026-03-15 10:35:18,464] Trial 8 finished with value: 5.404241939117209 and parameters: {'x
[I 2026-03-15 10:35:18,466] Trial 9 finished with value: 3.573167338386294 and parameters: {'x
[I 2026-03-15 10:35:18,482] Trial 10 finished with value: 41.70698488697969 and parameters: {'
[I 2026-03-15 10:35:18,486] Trial 11 finished with value: 0.07648831160016967 and parameters:
[I 2026-03-15 10:35:18,491] Trial 12 finished with value: 0.0017839525770153631 and parameters
[I 2026-03-15 10:35:18,496] Trial 13 finished with value: 0.0021460583817575402 and parameters
[I 2026-03-15 10:35:18,501] Trial 14 finished with value: 11.726312034177141 and parameters: {
[I 2026-03-15 10:35:18,505] Trial 15 finished with value: 0.3090550375394498 and parameters: {
[I 2026-03-15 10:35:18,509] Trial 16 finished with value: 0.8571013003771267 and parameters: {
[I 2026-03-15 10:35:18,512] Trial 17 finished with value: 5.341683251914149 and parameters: {'
[I 2026-03-15 10:35:18,521] Trial 18 finished with value: 8.560155185879465 and parameters: {'
[I 2026-03-15 10:35:18,525] Trial 19 finished with value: 0.28892105728358997 and parameters:


Best params: {'x': 1.9577631372256963}
Best value: 0.0018
```

That code already includes the essential architecture: study creation, trial generation, search-space definition, sampler-driven suggestions, and result collection.

4.13. Practical Exercise: Inspect Trial Objects

After running a small study, inspect the recorded trial objects:

```
for trial in study.trials[:3]:
    print("Trial:", trial.number)
    print("Params:", trial.params)
    print("Value:", trial.value)
    print("State:", trial.state)
    print()
```

```
Trial: 0
Params: {'x': -0.21924879335943537}
Value: 4.92506520682731
State: TrialState.COMPLETE

Trial: 1
Params: {'x': -1.10087267078963}
Value: 9.615411320450013
State: TrialState.COMPLETE

Trial: 2
Params: {'x': -2.790264234161283}
Value: 22.946631433084782
State: TrialState.COMPLETE
```

You can also inspect one specific trial in more detail:

```
trial = study.trials[0]

trial.params
trial.value
trial.distributions
trial.datetime_start
trial.datetime_complete
```

This information is useful for debugging studies, analyzing optimization behavior, and later building dashboards or experiment summaries.

4.14. Recap

Optuna's architecture consists of a small number of key components with distinct responsibilities:

Component	Role
`Study`	Manages the optimization experiment
`Trial`	Represents one evaluation of the objective
Objective function	Defines the optimization problem
Search space	Defines the tunable parameters
`Sampler`	Proposes parameter values
`Pruner`	Stops weak trials early
Storage	Persists experiment state

The main architectural idea is not complexity, but coordination. Optuna works because these components form an iterative feedback loop between proposing parameters, running evaluations, deciding whether to stop early, and storing results.

4.15. Decision Guide

Use the following mental model:

- Reach for the `Study` when you want experiment-level control.
- Reach for the `Trial` when defining parameters or reporting intermediate progress.
- Reach for samplers when changing optimization strategy.
- Reach for pruners when trial evaluations are iterative and expensive.
- Reach for persistent storage when experiments need to survive process boundaries or run in parallel.

Next, Basic Optuna Usage turns this architecture into practical day-to-day Optuna workflows.

Part II.

Part II: Core Usage

5. Basic Optuna Usage

This chapter moves from theory and internal architecture to day-to-day Optuna usage. The goal is straightforward: create a study, define an objective function, declare parameters through the `trial.suggest_*` API, run optimization, and inspect the results.

By the end of the chapter, you should be able to build and run a small Optuna study from scratch and understand how to retrieve the best trial, best parameters, and full trial history.

This chapter assumes you already understand the high-level architecture from Optuna Architecture.

5.1. Why This Matters

Many optimization libraries look conceptually elegant but remain awkward in practice. Optuna's main strength is that its core workflow is compact and readable: you write a normal Python function, define parameters where they are used, and let the study coordinate the optimization loop.

That makes it a strong practical tool for hyperparameter tuning, simulation calibration, and general black-box optimization.

5.2. A Fast First Win

The quickest useful Optuna program fits on one screen. It creates a study, defines one parameter, runs several trials, and prints the best result.

```
import optuna

optuna.logging.set_verbosity(optuna.logging.WARNING)


def objective(trial):
    x = trial.suggest_float("x", -10, 10)
    return (x - 2) ** 2


study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=20)
```

```
print("Best value:", round(study.best_value, 4))
print("Best parameters:", study.best_params)
```

```
Best value: 0.0682
Best parameters: {'x': 2.2611793458286575}
```

That is already a complete optimization workflow. Everything else in this chapter is a refinement of that pattern.

5.3. Installation and Setup

Optuna is installed with `pip`:

```
pip install optuna
```

To verify the installation:

```
import optuna
print(optuna.__version__)
```

```
4.7.0
```

Optional packages become useful later:

```
pip install optuna-dashboard
pip install plotly
```

Those packages support dashboards and visualization, but they are not required for the core optimization loop.

5.4. Creating a Study

The `Study` object manages the whole optimization process.

The most basic form is:

```
import optuna

study = optuna.create_study(direction="minimize")
print(type(study).__name__)
```

The most important initial choice is the optimization direction.

Parameter	Meaning
`direction`	Either `"minimize"` or `"maximize"`

Examples:

```
study = optuna.create_study(direction="minimize")
study = optuna.create_study(direction="maximize")
```

Use minimization for losses or error metrics. Use maximization for accuracy, reward, or utility-type objectives.

5.5. Defining the Objective Function

The objective function defines the problem to be optimized. It receives a `Trial` object and returns a numeric value.

```
def objective(trial):
    x = trial.suggest_float("x", -10, 10)
    return (x - 2) ** 2
```

Inside the function, three things usually happen:

1. Parameters are suggested.
2. A model or mathematical function is evaluated.
3. A scalar value is returned.

Optuna then tries to minimize or maximize that returned value depending on the study configuration.

5.6. Running the Optimization

Once the study and objective function exist, optimization starts with:

```
study.optimize(objective, n_trials=100)
```

The key arguments are:

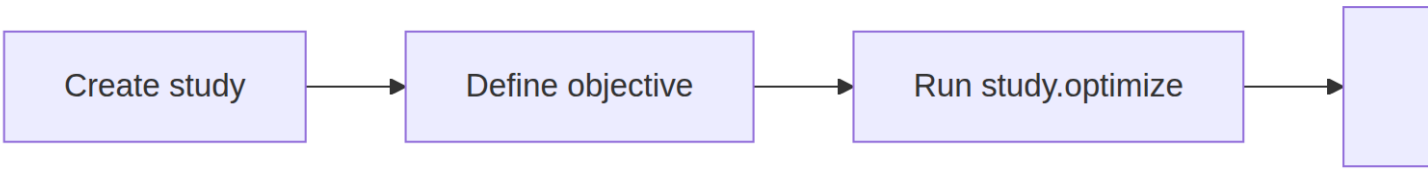
Parameter	Meaning
`objective`	The function to optimize

Parameter	Meaning
`n_trials`	The number of objective evaluations

Each trial corresponds to one evaluation of the objective.

```
Trial 1 -> parameters -> objective value
Trial 2 -> parameters -> objective value
Trial 3 -> parameters -> objective value
...
```

As trials accumulate, the sampler updates its search strategy using past results.



5.7. The Parameter Suggestion API

Optuna defines the search space through `trial.suggest_*` methods. This is the define-by-run paradigm: parameters are declared inside the objective exactly where they are needed.

5.7.1. `suggest_float`

Use this for continuous parameters.

```
x = trial.suggest_float("x", -10, 10)
```

This samples a floating-point value between the given bounds.

5.7.2. Log-scaled floats

For parameters spanning several orders of magnitude, use logarithmic sampling.

```
lr = trial.suggest_float("lr", 1e-5, 1e-1, log=True)
```

This is especially appropriate for quantities such as learning rates, regularization strengths, and tolerances.

5.7.3. suggest_int

Use this for integer-valued parameters.

```
depth = trial.suggest_int("depth", 2, 10)
```

Typical examples include tree depth, number of layers, and numbers of neighbors.

5.7.4. suggest_categorical

Use this for discrete choices.

```
optimizer = trial.suggest_categorical(
    "optimizer",
    ["adam", "sgd", "rmsprop"],
)
```

This is common for model families, optimizers, activations, and preprocessing variants.

5.8. A Study with Multiple Parameters

The objective function can define several parameters at once.

```
def objective(trial):
    x = trial.suggest_float("x", -10, 10)
    y = trial.suggest_float("y", -10, 10)
    return (x - 2) ** 2 + (y + 3) ** 2
```

This turns the search into a two-dimensional optimization problem. In practical machine-learning use, the same pattern extends naturally to many more parameters.

5.9. Retrieving Optimization Results

After optimization, the study stores everything needed to inspect the outcome.

5.9.1. Best value

```
print(study.best_value)
```

This returns the best objective value found.

5.9.2. Best parameters

```
print(study.best_params)
```

Example output might look like:

```
{'x': 1.98, 'y': -2.97}
```

5.9.3. Best trial

```
best = study.best_trial

print(best.params)
print(best.value)
```

The best trial contains both the winning parameters and the associated metadata.

5.10. Inspecting All Trials

The full trial history is available through `study.trials`.

```
for trial in study.trials[:5]:
    print(trial.number, round(trial.value, 4), trial.params)
```

This is useful for:

- Debugging.
- Visualizing search behavior.
- Building experiment summaries.
- Understanding whether the sampler is converging toward a promising region.

5.11. A Minimal End-to-End Example

The following example puts the full basic workflow in one place:

```python
import optuna

optuna.logging.set_verbosity(optuna.logging.WARNING)


def objective(trial):
    x = trial.suggest_float("x", -10, 10)
    return (x - 2) ** 2


study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=30)

print("Best value:", round(study.best_value, 4))
print("Best parameters:", study.best_params)
```

```
Best value: 0.0138
Best parameters: {'x': 2.1175837199966643}
```

This is the smallest workflow that still feels like real Optuna usage.

> **Common Mistake**
>
> Do not define parameters outside the objective function and then expect Optuna to optimize them. If a value does not come from `trial.suggest_*`, it is not part of the search space and the sampler cannot reason about it.

5.12. Practical Exercise: Optimize a Mathematical Function

As a first real exercise, optimize the function

$$f(x) = (x - 3)^2 + \sin(5x)$$

using Optuna.

```python
import math
import optuna

optuna.logging.set_verbosity(optuna.logging.WARNING)


def objective(trial):
    x = trial.suggest_float("x", -5, 5)
    return (x - 3) ** 2 + math.sin(5 * x)
```

```
study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=100)

print("Best x:", study.best_params)
print("Best value:", round(study.best_value, 4))
```

```
Best x: {'x': 3.414533227534076}
Best value: -0.807
```

What to observe:

- Early trials explore broadly.
- Later trials focus more often near strong regions.
- The sampler adapts based on what previous trials discovered.

Try extending the exercise in three ways:

1. Increase the search space from `[-5, 5]` to `[-20, 20]` and compare the result.
2. Replace the one-dimensional objective with a two-parameter function.
3. Add an integer or categorical parameter to see how naturally Optuna handles mixed spaces.

5.13. Recap

Basic Optuna usage follows a simple five-step pattern:

1. Install Optuna.
2. Create a study.
3. Define the objective function.
4. Use `trial.suggest_*` methods to declare parameters.
5. Run optimization with `study.optimize(...)`.

The most important beginner-facing APIs in this chapter are:

Function	Purpose
`suggest_float`	Continuous parameters
`suggest_int`	Integer parameters
`suggest_categorical`	Discrete choices

The most important result accessors are:

- `study.best_value`
- `study.best_params`
- `study.best_trial`

5.14. Decision Guide

Use the following rule of thumb:

- Reach for a minimal study first when testing a new optimization idea.
- Use `suggest_float`, `suggest_int`, and `suggest_categorical` to express the true structure of the search space.
- Inspect `study.trials` whenever the optimizer behaves unexpectedly.
- Keep the first objective function simple before scaling up to real pipelines.

Next, Designing Search Spaces focuses on the part of optimization that often matters most in practice: choosing parameter ranges and structures that the optimizer can search effectively.

6. Designing Search Spaces

This chapter focuses on one of the most important practical skills in optimization: designing a search space that the optimizer can explore effectively. In real Optuna workflows, a well-designed space often matters more than the choice between two strong samplers.

By the end of the chapter, you should be able to distinguish continuous, discrete, categorical, conditional, dynamic, and hierarchical search spaces, and choose parameter ranges that reflect both domain knowledge and optimization efficiency.

This chapter builds directly on Basic Optuna Usage and connects back to why The TPE Algorithm works well with tree-structured spaces.

6.1. Why This Matters

Suppose you are tuning a model and choose a learning-rate range from `1e-8` to `10`, a tree depth from `1` to `1000`, and several conditional parameters that are never active together. Even a strong optimizer will waste trials in irrelevant or pathological regions.

Search-space design matters because the optimizer can only search the space you give it. If the space is poorly structured, the optimization loop becomes inefficient no matter how sophisticated the sampler is.

6.2. A Fast First Win

The easiest way to see search-space design in action is to compare a few parameter types inside one small Optuna objective.

```
import optuna

optuna.logging.set_verbosity(optuna.logging.WARNING)


def objective(trial):
    x = trial.suggest_float("x", -10, 10)
    depth = trial.suggest_int("depth", 2, 8)
    optimizer = trial.suggest_categorical("optimizer", ["adam", "sgd"])
    lr = trial.suggest_float("lr", 1e-5, 1e-1, log=True)

    penalty = 0.0 if optimizer == "adam" else 0.2
```

```
    return (x - 2) ** 2 + 0.01 * depth + penalty + abs(lr - 1e-2)


study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=20)

print("Best params:", study.best_params)
print("Best value:", round(study.best_value, 4))
```

```
Best params: {'x': 1.7635276566102611, 'depth': 3, 'optimizer': 'sgd', 'lr': 0.007832540874741
Best value: 0.2881
```

This example already shows the main idea of the chapter: the search space is not just a range of numbers. It is a structured description of what the optimizer is allowed to try.

6.3. Continuous, Discrete, and Categorical Parameters

Search spaces usually combine different parameter types.

6.3.1. Continuous parameters

Continuous parameters represent real-valued ranges.

$$x \in [-10, 10]$$

In Optuna:

```
x = trial.suggest_float("x", -10, 10)
```

Typical continuous parameters include:

Parameter	Example range
Learning rate	`0.0001` to `0.1`
Regularization	`1e-6` to `1`
Dropout	`0` to `0.5`

6.3.2. Discrete parameters

Discrete parameters often take integer values.

$$depth \in \{1, 2, 3, \dots, 10\}$$

In Optuna:

```
depth = trial.suggest_int("depth", 1, 10)
```

Examples include tree depth, numbers of layers, and neighborhood sizes.

6.3.3. Categorical parameters

Categorical parameters represent choices among named options.

```
model = trial.suggest_categorical(
    "model",
    ["xgboost", "random_forest", "svm"],
)
```

These are common for model families, optimizers, activations, kernels, and preprocessing decisions.

6.4. Why Log-Scaled Parameters Matter

Some parameters span multiple orders of magnitude. Learning rates are the canonical example.

Instead of sampling uniformly over

$$[10^{-5}, 10^{-1}]$$

you usually want to sample in log-space:

```
lr = trial.suggest_float("lr", 1e-5, 1e-1, log=True)
```

This avoids oversampling the large end of the range and better reflects how such parameters behave in practice.

Typical log-scaled parameters include:

Parameter	Example range
Learning rate	`1e-5` to `1e-1`
Regularization	`1e-6` to `10`
Kernel width	`1e-3` to `10`

6.5. Conditional Parameters

Some parameters only make sense when another choice is active.

For example, if the selected model is a random forest, you may want to optimize `n_trees` and `depth`. If the selected model is an SVM, those parameters are irrelevant, and instead you care about `C` and `kernel`.

Optuna handles this naturally with define-by-run logic:

```
def objective(trial):
    model = trial.suggest_categorical("model", ["rf", "svm"])

    if model == "rf":
        n_trees = trial.suggest_int("n_trees", 50, 200)
        depth = trial.suggest_int("depth", 3, 10)
        return 0.001 * n_trees + 0.1 * depth

    c_value = trial.suggest_float("C", 1e-3, 10, log=True)
    kernel = trial.suggest_categorical("kernel", ["linear", "rbf"])
    kernel_penalty = 0.0 if kernel == "rbf" else 0.2
    return abs(c_value - 1.0) + kernel_penalty
```

This creates a conditional branch inside the search space. Structurally, that is exactly the kind of tree-shaped configuration TPE is designed to handle well.

6.6. Dynamic Search Spaces

Optuna also allows the number of active parameters to depend on earlier choices.

For example:

```
def objective(trial):
    n_layers = trial.suggest_int("n_layers", 1, 4)

    total_units = 0
    for i in range(n_layers):
        units = trial.suggest_int(f"units_layer_{i}", 32, 256)
        total_units += units
```

```
    return abs(total_units - 320)
```

Here, the dimensionality of the space changes from trial to trial because the number of active layer parameters depends on `n_layers`.

This flexibility is useful for neural architecture search, pipeline configuration, and other structured optimization problems.

6.7. Hierarchical Search Spaces

Hierarchical spaces are multi-level search spaces where one decision activates a whole family of sub-decisions.

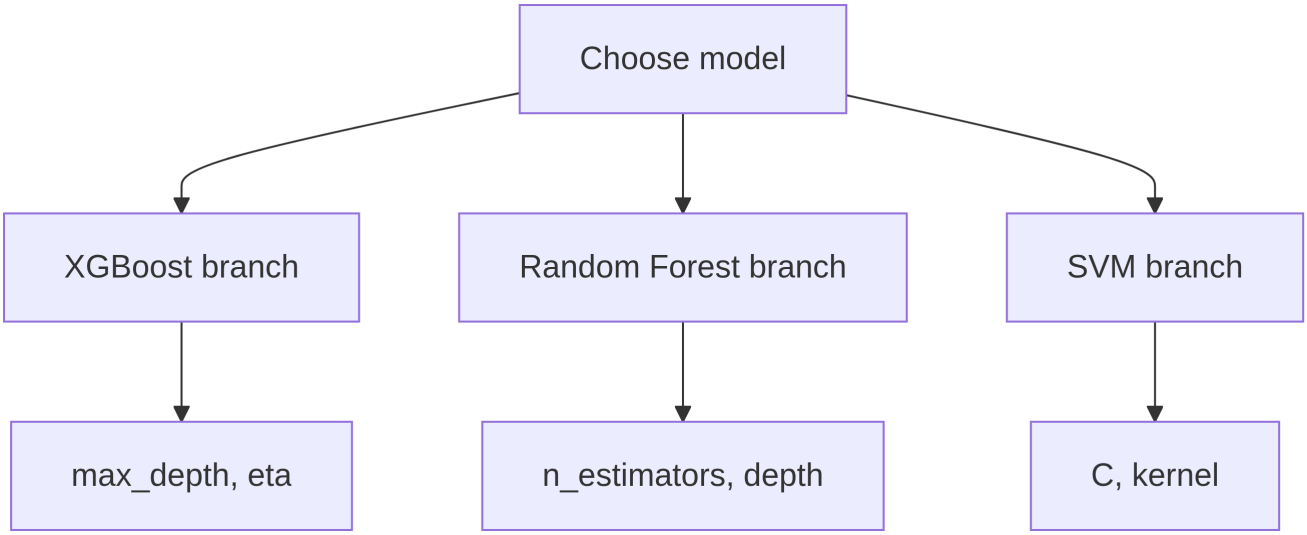
```
def objective(trial):
    model = trial.suggest_categorical("model", ["xgb", "rf", "svm"])

    if model == "xgb":
        max_depth = trial.suggest_int("max_depth", 3, 10)
        eta = trial.suggest_float("eta", 1e-3, 0.3, log=True)
        return 0.1 * max_depth + abs(eta - 0.05)

    if model == "rf":
        n_estimators = trial.suggest_int("n_estimators", 50, 300)
        return abs(n_estimators - 180)

    c_value = trial.suggest_float("C", 1e-3, 10, log=True)
    return abs(c_value - 1.5)
```

This kind of hierarchy is common in AutoML workflows and whole-pipeline optimization.



6.8. Example: Neural Architecture Search

Optuna is often used for neural architecture search precisely because define-by-run spaces can express variable structure compactly.

```
def objective(trial):
    n_layers = trial.suggest_int("n_layers", 1, 4)

    layers = []
    for i in range(n_layers):
        units = trial.suggest_int(f"units_{i}", 32, 256)
        dropout = trial.suggest_float(f"dropout_{i}", 0.0, 0.5)
        layers.append((units, dropout))

    return len(layers)
```

Each trial can represent a different architecture, which would be cumbersome to describe in a rigid static search-space definition.

6.9. Example: Algorithm Configuration

Search spaces do not have to be about model hyperparameters only. They can also represent algorithmic design choices.

```
solver = trial.suggest_categorical("solver", ["gradient", "newton"])

if solver == "gradient":
    lr = trial.suggest_float("lr", 1e-4, 1e-1, log=True)
else:
    damping = trial.suggest_float("damping", 1e-3, 1)
```

This lets Optuna search not only over numeric hyperparameters, but over whole families of algorithm behavior.

6.10. Best Practices for Search Space Design

6.10.1. Use log scales for multiplicative parameters

Bad:

```
lr = trial.suggest_float("lr", 0.00001, 0.1)
```

Better:

```
lr = trial.suggest_float("lr", 1e-5, 1e-1, log=True)
```

6.10.2. Avoid absurdly large ranges

Bad:

```
depth = trial.suggest_int("depth", 1, 1000)
```

Better:

```
depth = trial.suggest_int("depth", 3, 15)
```

Large unrealistic ranges waste trials and slow convergence.

6.10.3. Use domain knowledge aggressively

Good search spaces reflect:

- Theoretical constraints.
- Empirical experience.
- Practical operating limits.

> ⚠ Common Mistake
>
> Do not treat the search space as a neutral container. It encodes assumptions about what values are plausible. A badly chosen space can make a strong sampler look weak, hide the true optimum behind useless volume, or spend most trials on configurations you would never deploy.

6.11. Practical Exercise: Conditional Hyperparameter Optimization

As a hands-on exercise, optimize a tiny conditional problem where the study chooses between two model families.

```
import math
import optuna

optuna.logging.set_verbosity(optuna.logging.WARNING)


def objective(trial):
    model = trial.suggest_categorical("model", ["quadratic", "sin"])
```

```
    x = trial.suggest_float("x", -5, 5)

    if model == "quadratic":
        return (x - 2) ** 2

    return math.sin(x) + 1


study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=60)

print("Best parameters:", study.best_params)
print("Best value:", round(study.best_value, 4))
```

```
Best parameters: {'model': 'sin', 'x': 4.709168870705228}
Best value: 0.0
```

This example demonstrates:

- Categorical model selection.
- Conditional objective logic.
- Flexible search-space definition inside the objective.

Try extending it by adding model-specific parameters that only apply to one branch.

6.12. Recap

Effective search spaces often include a mix of:

Feature	Why it matters
Continuous parameters	Capture real-valued behavior
Discrete parameters	Respect integer constraints
Log distributions	Handle multi-scale parameters
Conditional parameters	Support structured model choices
Dynamic spaces	Allow variable architecture or pipeline depth
Hierarchical spaces	Enable multi-level AutoML workflows

The practical lesson is simple: a good search space can improve optimization more than switching between two strong optimizers.

6.13. Decision Guide

Use the following mental model:

- Use continuous ranges for genuinely real-valued quantities.
- Use integer or categorical parameters when the problem structure demands them.
- Use `log=True` whenever scale matters more than absolute distance.
- Use conditional and hierarchical branches when different model families require different parameter sets.
- Shrink overly broad ranges whenever domain knowledge allows it.

Next, Samplers focuses on the optimization engines that search these spaces and how their behavior differs in practice.

7. Samplers

This chapter explains the optimization engines that actually choose parameter values in Optuna. Once the objective function and search space are defined, the sampler determines how the study explores the space, what it learns from previous trials, and how quickly it tends to converge toward strong regions.

By the end of the chapter, you should be able to describe the main Optuna sampler families, understand when to use each one, and explain how their exploration behavior changes in high-dimensional or structured search spaces.

This chapter builds directly on Designing Search Spaces, because sampler performance only makes sense in the context of the space being searched.

7.1. Why This Matters

Once your search space is well designed, the next practical question is which optimization engine should drive the search. The answer depends on the structure of the problem: mixed or categorical spaces, continuous-only optimization, exhaustive benchmarking, or multiple objectives all push toward different samplers.

Optuna exposes those choices through a unified interface, which makes it easy to compare strategies without rewriting the rest of the optimization workflow.

7.2. A Fast First Win

The quickest way to see what a sampler changes is to run the same objective with two different search strategies.

```
import math
import optuna

optuna.logging.set_verbosity(optuna.logging.WARNING)


def objective(trial):
    x = trial.suggest_float("x", -5, 5)
    return (x - 3) ** 2 + math.sin(5 * x)
```

```
study_tpe = optuna.create_study(
    sampler=optuna.samplers.TPESampler(),
    direction="minimize",
)
study_tpe.optimize(objective, n_trials=40)

study_random = optuna.create_study(
    sampler=optuna.samplers.RandomSampler(),
    direction="minimize",
)
study_random.optimize(objective, n_trials=40)

print("TPE best:", round(study_tpe.best_value, 4))
print("Random best:", round(study_random.best_value, 4))
```

```
TPE best: -0.8063
Random best: -0.643
```

This does not prove that one sampler is always better. It shows the central point of the chapter: the sampler is the optimization policy that drives trial selection.

7.3. What a Sampler Does

In Optuna, the sampler is the component responsible for proposing new parameter values.

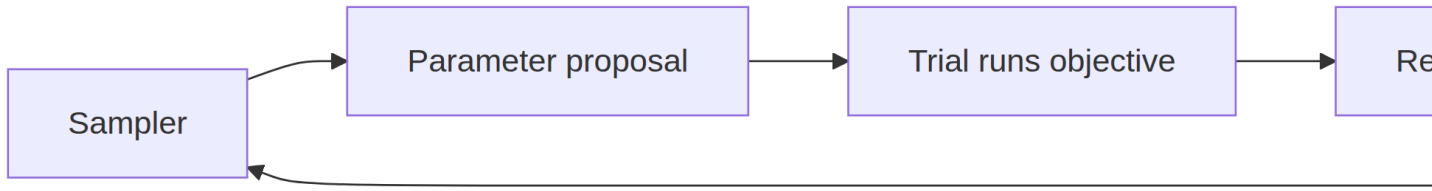
Conceptually, the loop looks like this:

```
Sampler -> proposes parameters
Trial -> evaluates objective
Sampler -> updates internal strategy
```

At a higher level, each trial follows the same pattern:

```
1. sampler proposes parameters
2. objective function evaluates them
3. result is stored
4. sampler learns from results
```

The crucial difference between samplers is how they balance exploration and exploitation.



7.4. TPE Sampler

Optuna's default optimizer is the Tree-structured Parzen Estimator, or TPE.

TPE is a Bayesian-style optimization method based on density estimation. Instead of modeling

$$p(y \mid x)$$

it models

$$p(x \mid y)$$

and uses the separation between good and bad trials to rank new candidates.

The core idea is:

```
good trials -> build density l(x)
bad trials -> build density g(x)
maximize l(x) / g(x)
```

Characteristics:

Property	Behavior
Type	Bayesian-style density estimation
Exploration	Adaptive
Scalability	Good
Categorical support	Excellent
High-dimensional robustness	Good

Use TPE for:

- General machine-learning hyperparameter tuning.
- Mixed parameter spaces.
- Categorical variables.
- Conditional or tree-structured spaces.

In practice, TPE is the default because it is the most versatile general-purpose sampler in Optuna.

7.5. Random Sampler

`RandomSampler` performs pure random search.

```
sampler = optuna.samplers.RandomSampler()

study = optuna.create_study(
    sampler=sampler,
    direction="minimize",
)
```

It does not learn from previous trials. Each proposal is independent.

Characteristics:

Property	Behavior
Type	Random search
Exploration	Maximum
Exploitation	None
Complexity	Minimal

Use random search for:

- Baseline comparisons.
- Early pipeline debugging.
- Very small or poorly understood spaces.
- Cases where you want a neutral reference strategy.

Random search is often stronger than people expect, especially as a baseline.

7.6. CMA-ES Sampler

CMA-ES stands for Covariance Matrix Adaptation Evolution Strategy.

It is an evolutionary method that maintains a multivariate Gaussian search distribution and updates that distribution based on successful samples.

Conceptually:

```
sample population
evaluate objective
select strong candidates
update mean and covariance
repeat
```

Characteristics:

Property	Behavior
Type	Evolutionary strategy

Property	Behavior
Exploration	Adaptive
Handles correlations	Very well
Continuous spaces	Excellent

Use CMA-ES for:

- Continuous optimization.
- Medium-dimensional spaces.
- Expensive smooth objectives.
- Problems where parameter correlations matter.

Limitations:

- It is not a natural fit for categorical variables.
- It often has a slower warm-up phase than simpler approaches.

Illustrative setup:

```
sampler = optuna.samplers.CmaEsSampler()
study = optuna.create_study(sampler=sampler, direction="minimize")
```

7.7. Grid Sampler

`GridSampler` performs exhaustive search over a predefined grid.

```
search_space = {
    "x": [-2, 0, 2],
    "y": [-1, 1],
}

sampler = optuna.samplers.GridSampler(search_space)
```

It does not learn from past results. Its purpose is deterministic coverage of a fixed set of combinations.

Characteristics:

Property	Behavior
Type	Exhaustive search
Exploration	Deterministic
Learning	None

Use grid search when:

- The space is genuinely small.
- Reproducible coverage matters more than efficiency.
- You want a controlled benchmark.

Its weakness is obvious: it becomes impractical very quickly as the number of parameters grows.

7.8. NSGA-II for Multi-Objective Optimization

`NSGAIISampler` is used for multi-objective optimization.

Instead of optimizing a single objective

$$f(x)$$

you optimize multiple objectives simultaneously, such as:

$$f_1(x), f_2(x)$$

For example, you may want to maximize accuracy while minimizing model size.

The goal is not a single best point, but a Pareto frontier: a set of trade-off solutions where improving one objective would worsen another.

```
study = optuna.create_study(
    directions=["minimize", "maximize"],
    sampler=optuna.samplers.NSGAIISampler(),
)
```

Characteristics:

Property	Behavior
Type	Evolutionary multi-objective optimization
Output	Pareto set
Exploration	Population-based

7.9. Exploration Behavior Across Samplers

Different samplers explore the space differently.

Sampler	Exploration strategy
Random	Uniform sampling
TPE	Probabilistic density modeling
CMA-ES	Adaptive Gaussian search

Sampler	Exploration strategy
Grid	Deterministic enumeration
NSGA-II	Evolutionary population search

These choices influence:

- Convergence speed.
- Sample efficiency.
- Robustness to local structure.
- Suitability for mixed or high-dimensional spaces.

7.10. Samplers in High-Dimensional Spaces

High-dimensional optimization is difficult because search volume grows rapidly with the number of parameters.

If each parameter has k relevant configurations and the dimension is d, then the effective search burden grows roughly like:

$$k^d$$

That is why high-dimensional behavior matters so much when choosing a sampler.

In broad practical terms:

Sampler	High-dimensional behavior
Random	Moderate baseline
TPE	Often strong in mixed spaces
CMA-ES	Useful in continuous spaces, but not universal
Grid	Usually poor

TPE often performs well in mixed, structured problems with dozens of parameters, which is one reason it became Optuna's default choice.

7.11. Choosing the Right Sampler

As a practical rule of thumb:

Situation	Recommended sampler
General hyperparameter tuning	TPE
Baseline comparison	Random
Continuous optimization	CMA-ES

Situation	Recommended sampler
Small discrete space	Grid
Multi-objective problem	NSGA-II

The default recommendation remains simple:

> Start with TPE unless you have a concrete reason to choose something else.

Common Mistake

Do not compare samplers on a poorly designed or unfairly scaled search space and then conclude that one algorithm is intrinsically better. Sampler performance depends heavily on search-space structure, budget, parameter types, and objective noise.

7.12. Practical Exercise: Compare Samplers

Compare TPE and random search on the same one-dimensional objective:

$$f(x) = (x - 3)^2 + \sin(5x)$$

```
import math
import optuna

optuna.logging.set_verbosity(optuna.logging.WARNING)


def objective(trial):
    x = trial.suggest_float("x", -5, 5)
    return (x - 3) ** 2 + math.sin(5 * x)


study_tpe = optuna.create_study(
    sampler=optuna.samplers.TPESampler(),
    direction="minimize",
)
study_tpe.optimize(objective, n_trials=100)

study_random = optuna.create_study(
    sampler=optuna.samplers.RandomSampler(),
    direction="minimize",
)
study_random.optimize(objective, n_trials=100)

print("TPE best:", round(study_tpe.best_value, 4))
print("Random best:", round(study_random.best_value, 4))
```

```
TPE best: -0.8056
Random best: -0.8032
```

What to observe:

- Early trials often look similar.
- TPE usually begins exploiting strong regions sooner.
- Random search continues broad exploration throughout the run.

If you want to deepen the exercise, repeat the comparison with a larger trial budget or a two-parameter function.

7.13. Recap

Samplers determine how Optuna chooses new parameter values.

Sampler	Best fit
TPE	General hyperparameter tuning
Random	Baselines and debugging
CMA-ES	Continuous optimization
Grid	Small discrete spaces
NSGA-II	Multi-objective optimization

TPE is the default because it balances exploration and exploitation, handles categorical and conditional spaces well, and remains practical in many high-dimensional problems.

7.14. Decision Guide

Use the following mental model:

- Use TPE as the general default.
- Use random search to establish a baseline or sanity check a pipeline.
- Use CMA-ES when the space is mostly continuous and parameter correlations matter.
- Use grid only when the space is intentionally tiny.
- Use NSGA-II when there is no single objective to optimize.

Next, Pruning turns to one of Optuna's most important efficiency features: stopping weak trials early to save compute.

8. Pruning

This chapter explains how Optuna avoids wasting computation on trials that are clearly underperforming. In many optimization problems, especially machine-learning workflows, each trial unfolds over a sequence of steps such as epochs, boosting rounds, or simulation iterations. Pruning allows Optuna to stop weak trials early and redirect resources to stronger candidates.

By the end of the chapter, you should be able to explain how intermediate results are reported, how pruning decisions are made, and when to use median pruning, successive halving, Hyperband, and related strategies.

This chapter builds directly on Samplers. Samplers decide what to try next; pruners decide whether the current trial is still worth continuing.

8.1. Why This Matters

Suppose two neural-network configurations are both scheduled to train for 100 epochs. One shows promising validation loss from the first few epochs. The other is clearly lagging. If you still force both to train to completion, most of your optimization budget gets spent confirming something you already suspected.

Pruning matters because early stopping at the trial level is often the biggest practical speedup in hyperparameter optimization.

8.2. A Fast First Win

The simplest pruning pattern is small enough to fit in one function: report an intermediate metric, ask the pruner whether the trial should continue, and stop early if not.

```
import optuna
import random

optuna.logging.set_verbosity(optuna.logging.WARNING)


def objective(trial):
    lr = trial.suggest_float("lr", 1e-4, 1e-1, log=True)
    loss = 1.2 + 0.2 * lr

    for epoch in range(20):
```

```
        loss *= random.uniform(0.82, 1.02)
        trial.report(loss, epoch)

        if trial.should_prune():
            raise optuna.TrialPruned()

    return loss


study = optuna.create_study(
    direction="minimize",
    pruner=optuna.pruners.MedianPruner(),
)
study.optimize(objective, n_trials=30)

completed = sum(t.state == optuna.trial.TrialState.COMPLETE for t in
 ↪  study.trials)
pruned = sum(t.state == optuna.trial.TrialState.PRUNED for t in study.trials)

print("Completed trials:", completed)
print("Pruned trials:", pruned)
```

```
Completed trials: 5
Pruned trials: 25
```

This captures the full pruning pattern used in real training loops. The only difference in larger systems is that the intermediate metric usually comes from a real model evaluation.

8.3. Why Pruning Helps

Consider two training runs after only a few epochs:

Epoch	Trial A loss	Trial B loss
1	0.92	1.40
2	0.91	1.35
3	0.90	1.31

Trial B is already much weaker. If both trials still train to epoch 100, the optimizer wastes time, compute, and often expensive accelerator resources.

Pruning makes the optimization budget more selective:

- Weak trials terminate early.
- Promising trials receive more compute.
- The study covers more configurations with the same budget.

8.4. Intermediate Results

Pruning only works when the objective provides progress updates during execution. Optuna uses:

```
trial.report(value, step)
```

where:

- `value` is the current metric, such as validation loss or accuracy.
- `step` is the iteration number, such as epoch or boosting round.

Example:

```
trial.report(validation_loss, epoch)
```

Without intermediate reports, the pruner has nothing to evaluate.

8.5. The Pruning Decision

After reporting an intermediate value, the objective asks Optuna whether the trial should continue:

```
trial.should_prune()
```

If the answer is yes, the standard Optuna pattern is:

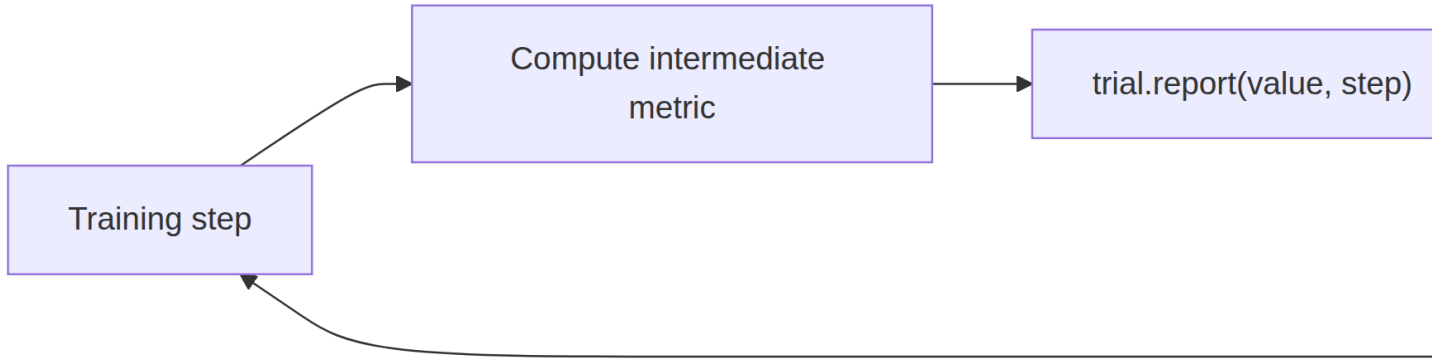
```
raise optuna.TrialPruned()
```

The full loop looks like this:

```
for epoch in range(100):
    loss = train_one_epoch()
    trial.report(loss, epoch)

    if trial.should_prune():
        raise optuna.TrialPruned()
```

That exception is not an error condition in the usual sense. It is the normal control-flow signal for a trial that has been intentionally stopped.



8.6. Median Pruning

`MedianPruner` is the simplest widely useful pruning strategy.

It compares the current trial against the median performance of earlier completed trials at the same step. If the current trial is worse than that reference, it is pruned.

Example at epoch 5:

Trial	Loss
1	0.70
2	0.65
3	0.75

The median is roughly:

$$0.70$$

If a new trial reports a loss like:

$$0.90$$

it is a natural candidate for pruning.

Advantages of median pruning:

- Simple.
- Robust.
- Often effective with little tuning.

8.7. Successive Halving

Successive halving is a resource-allocation strategy rather than a purely statistical threshold rule.

The idea is:

1. Start many trials with small budgets.
2. Evaluate them quickly.
3. Keep only the best fraction.
4. Allocate larger budgets to the survivors.

Example schedule:

Stage	Trials	Epochs
1	100	5
2	25	20
3	6	100

Most trials die early. Only promising trials receive full resources.

8.8. Hyperband

Hyperband improves on successive halving by running multiple resource-allocation schedules with different trade-offs.

Some brackets favor many short trials. Others favor fewer but longer trials. This makes Hyperband more robust when you do not know the right allocation schedule in advance.

Conceptually:

```
multiple successive-halving brackets
each bracket explores a different resource schedule
best trials survive longer
```

Hyperband is one of the most widely used pruning strategies because it balances aggressiveness and robustness across a broad range of workloads.

8.9. Asynchronous Pruning

In distributed optimization, trials may run in parallel on different workers.

If pruning required global synchronization, idle time would accumulate quickly. Optuna therefore supports asynchronous pruning behavior, where trials are judged independently as their intermediate results arrive.

This helps with:

- Better resource utilization.
- Reduced waiting time.
- More efficient parallel studies.

8.10. Pruners Available in Optuna

Optuna includes several pruner classes.

Pruner	Strategy
`MedianPruner`	Median-based pruning
`SuccessiveHalvingPruner`	Multi-stage elimination
`HyperbandPruner`	Multiple halving schedules
`ThresholdPruner`	Absolute thresholds
`NopPruner`	Disables pruning

A basic setup looks like this:

```
pruner = optuna.pruners.MedianPruner()

study = optuna.create_study(
    direction="minimize",
    pruner=pruner,
)
```

8.11. Integration with Training Loops

Pruning is especially valuable for iterative algorithms such as neural-network training, boosting, and simulation-based learning loops.

The integration pattern is always the same:

```
for epoch in range(50):
    train(model)
    val_loss = evaluate(model)

    trial.report(val_loss, epoch)

    if trial.should_prune():
        raise optuna.TrialPruned()
```

Three lines matter architecturally:

- `trial.report(...)` sends the progress signal.
- `trial.should_prune()` asks for a pruning decision.
- `TrialPruned` stops the current trial early.

8.12. Example: Deep-Learning-Style Pruning Loop

Here is a simplified training-style objective with pruning:

```
def objective(trial):
    lr = trial.suggest_float("lr", 1e-5, 1e-2, log=True)

    model = build_model()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    for epoch in range(20):
        train_epoch(model)
        val_loss = validate(model)

        trial.report(val_loss, epoch)

        if trial.should_prune():
            raise optuna.TrialPruned()

    return val_loss
```

This is intentionally shown as an illustrative code fence rather than an executable cell. The important point is the structural integration, not the specific framework.

8.13. Practical Exercise: Simulate Pruning

Use the following runnable example to see pruning behavior in a small study:

```
import optuna
import random

optuna.logging.set_verbosity(optuna.logging.WARNING)


def objective(trial):
    lr = trial.suggest_float("lr", 1e-5, 1e-1, log=True)
    loss = 1.0 + 0.05 * lr

    for epoch in range(20):
        loss *= random.uniform(0.8, 1.1)
        trial.report(loss, epoch)

        if trial.should_prune():
            raise optuna.TrialPruned()
```

```
    return loss


study = optuna.create_study(
    direction="minimize",
    pruner=optuna.pruners.MedianPruner(),
)
study.optimize(objective, n_trials=50)

completed = sum(t.state == optuna.trial.TrialState.COMPLETE for t in
 ↪  study.trials)
pruned = sum(t.state == optuna.trial.TrialState.PRUNED for t in study.trials)

print("Completed:", completed)
print("Pruned:", pruned)
```

```
Completed: 8
Pruned: 42
```

What to observe:

- Many trials stop early.
- Only a subset finishes.
- The study spends more effort on promising regions.

8.14. Recap

Pruning is one of Optuna's most important efficiency features. It reduces wasted computation by stopping weak trials before they consume their full budget.

The key pieces are:

Function or class	Purpose
`trial.report()`	Send intermediate results
`trial.should_prune()`	Ask whether to stop early
`optuna.TrialPruned`	Terminate the trial cleanly

Common pruning strategies include:

Pruner	Best fit
Median	Simple and robust
Successive Halving	Budget allocation
Hyperband	Multi-schedule early stopping

Pruning is especially useful in deep learning, boosting, and any iterative workload where weak trials reveal themselves long before completion.

8.15. Decision Guide

Use the following mental model:

- Use a pruner whenever the objective produces meaningful intermediate metrics.
- Start with `MedianPruner` for a simple default.
- Consider successive halving or Hyperband when budget allocation is central to the problem.
- Skip pruning when the objective only returns a final value and has no useful intermediate signal.

Next, Advanced Optimization Techniques extends the framework to more complex workflows such as multi-objective search, constrained optimization, and custom optimization components.

Part III.

Part III: Advanced Techniques

9. Advanced Optimization Techniques

This chapter shows how Optuna moves beyond single-objective hyperparameter tuning into richer optimization workflows. By the end, you should be able to model dynamic search spaces, reason about Pareto-optimal trade-offs, and decide when to use constrained optimization or custom optimization components.

9.1. Why This Matters

In production systems, the best configuration is rarely the one with the highest validation score alone. You may need to balance accuracy against latency, memory footprint, energy usage, fairness constraints, or engineering limits imposed by deployment.

That is where Optuna's more advanced features matter. They let you optimize over search spaces whose structure depends on runtime decisions, retain multiple equally valid trade-off solutions, and inject domain-specific search or pruning logic when the built-in algorithms are not enough.

9.2. Fast First Win: A Multi-Objective Study

The quickest way to see the difference between ordinary and advanced optimization is to optimize two conflicting objectives at once.

Here we minimize:

$$f_1(x) = (x - 2)^2$$

and

$$f_2(x) = (x + 2)^2$$

No single value of x minimizes both objectives simultaneously, so the result is a Pareto set rather than a single best trial.

```
import optuna


def objective(trial):
    x = trial.suggest_float("x", -5.0, 5.0)
    f1 = (x - 2.0) ** 2
```

```
    f2 = (x + 2.0) ** 2
    return f1, f2


study = optuna.create_study(
    directions=["minimize", "minimize"],
    sampler=optuna.samplers.NSGAIISampler(seed=42),
)
study.optimize(objective, n_trials=80)

pareto_trials = sorted(study.best_trials, key=lambda trial: trial.params["x"])

for trial in pareto_trials[:8]:
    print(
        f"x={trial.params['x']:.3f} | "
        f"f1={trial.values[0]:.3f} | "
        f"f2={trial.values[1]:.3f}"
    )

print(f"Pareto-optimal trials: {len(study.best_trials)}")
```

```
[I 2026-03-15 10:35:51,060] A new study created in memory with name: no-name-6a176bf3-026b-446
[I 2026-03-15 10:35:51,062] Trial 0 finished with values: [10.592413423988893, 0.5556229317778
[I 2026-03-15 10:35:51,064] Trial 1 finished with values: [6.285766343860528, 42.3429108566538
[I 2026-03-15 10:35:51,065] Trial 2 finished with values: [0.10236123126315734, 18.66187657617
[I 2026-03-15 10:35:51,067] Trial 3 finished with values: [1.027010282524228, 8.91968901828715
[I 2026-03-15 10:35:51,069] Trial 4 finished with values: [29.59157195460952, 2.07306319000443
[I 2026-03-15 10:35:51,071] Trial 5 finished with values: [29.594196190423823, 2.0737578173200
[I 2026-03-15 10:35:51,073] Trial 6 finished with values: [41.20566489670265, 5.85235387015861
[I 2026-03-15 10:35:51,075] Trial 7 finished with values: [2.76145114246125, 32.05554280445606
[I 2026-03-15 10:35:51,077] Trial 8 finished with values: [0.9778240902545737, 9.0670250297112
[I 2026-03-15 10:35:51,078] Trial 9 finished with values: [0.006516651227320591, 16.6523228749
[I 2026-03-15 10:35:51,079] Trial 10 finished with values: [46.16054293912905, 7.8073024827932
[I 2026-03-15 10:35:51,081] Trial 11 finished with values: [7.285132829410962, 44.877921002370
[I 2026-03-15 10:35:51,082] Trial 12 finished with values: [1.754105310218955, 28.349516574252
[I 2026-03-15 10:35:51,083] Trial 13 finished with values: [23.781314297405455, 0.768443151667
[I 2026-03-15 10:35:51,085] Trial 14 finished with values: [26.85053646099223, 1.3965338375602
[I 2026-03-15 10:35:51,086] Trial 15 finished with values: [26.6870900439771, 1.35945083225180
[I 2026-03-15 10:35:51,088] Trial 16 finished with values: [15.662420225769758, 0.001799662532
[I 2026-03-15 10:35:51,090] Trial 17 finished with values: [3.0710308254266523, 5.051545356005
[I 2026-03-15 10:35:51,091] Trial 18 finished with values: [7.185347303077568, 1.7409487944468
[I 2026-03-15 10:35:51,093] Trial 19 finished with values: [16.709361582323204, 0.007692798166
[I 2026-03-15 10:35:51,095] Trial 20 finished with values: [0.7769912168823914, 9.725222794672
[I 2026-03-15 10:35:51,096] Trial 21 finished with values: [31.416713224675267, 2.576222076838
[I 2026-03-15 10:35:51,098] Trial 22 finished with values: [16.634598771846075, 0.006170654663
[I 2026-03-15 10:35:51,099] Trial 23 finished with values: [11.131441961038313, 0.440389424533
[I 2026-03-15 10:35:51,101] Trial 24 finished with values: [5.950185259987708, 2.435783997350
```

```
[I 2026-03-15 10:35:51,103] Trial 25 finished with values: [0.725494439922415, 23.539571351363
[I 2026-03-15 10:35:51,104] Trial 26 finished with values: [25.03263242597205, 1.0065349986408
[I 2026-03-15 10:35:51,106] Trial 27 finished with values: [3.450884387150625, 4.5896394602395
[I 2026-03-15 10:35:51,107] Trial 28 finished with values: [1.1574624993140197, 8.550628008277
[I 2026-03-15 10:35:51,108] Trial 29 finished with values: [42.71270630338613, 6.4287393209859
[I 2026-03-15 10:35:51,110] Trial 30 finished with values: [0.8547954409926973, 9.458383593107
[I 2026-03-15 10:35:51,111] Trial 31 finished with values: [28.034470359711058, 1.676400254694
[I 2026-03-15 10:35:51,112] Trial 32 finished with values: [40.31594795705311, 5.5200753958754
[I 2026-03-15 10:35:51,114] Trial 33 finished with values: [6.194401065388036, 42.105244045654
[I 2026-03-15 10:35:51,115] Trial 34 finished with values: [7.0560376995323795, 44.30660034549
[I 2026-03-15 10:35:51,116] Trial 35 finished with values: [1.1749985078681247, 25.846786357185
[I 2026-03-15 10:35:51,118] Trial 36 finished with values: [15.633027152728861, 0.002128686598
[I 2026-03-15 10:35:51,119] Trial 37 finished with values: [36.27988822455386, 4.0936573450645
[I 2026-03-15 10:35:51,120] Trial 38 finished with values: [0.02485974529663481, 14.763501866
[I 2026-03-15 10:35:51,122] Trial 39 finished with values: [6.7520726509747915, 1.964272150142
[I 2026-03-15 10:35:51,124] Trial 40 finished with values: [33.4039801981339, 3.167038985716
[I 2026-03-15 10:35:51,126] Trial 41 finished with values: [4.19524981515667, 3.809402624058
[I 2026-03-15 10:35:51,127] Trial 42 finished with values: [44.303864082318604, 7.054945771536
[I 2026-03-15 10:35:51,128] Trial 43 finished with values: [4.381503072642301, 37.127135238944
[I 2026-03-15 10:35:51,130] Trial 44 finished with values: [19.46751046368814, 0.169908991689
[I 2026-03-15 10:35:51,131] Trial 45 finished with values: [0.14045791700437812, 13.142240665
[I 2026-03-15 10:35:51,132] Trial 46 finished with values: [15.07682884316432, 0.013714930317
[I 2026-03-15 10:35:51,133] Trial 47 finished with values: [3.2375517002868737, 4.842993394511
[I 2026-03-15 10:35:51,135] Trial 48 finished with values: [2.349773845901536, 6.086596193639
[I 2026-03-15 10:35:51,136] Trial 49 finished with values: [26.537493199190116, 1.325849641232
[I 2026-03-15 10:35:51,141] Trial 50 finished with values: [7.267587152695562, 44.834357373860
[I 2026-03-15 10:35:51,143] Trial 51 finished with values: [0.5644941146212441, 22.57511998351
[I 2026-03-15 10:35:51,145] Trial 52 finished with values: [5.735974301036689, 40.895889626171
[I 2026-03-15 10:35:51,147] Trial 53 finished with values: [3.795769647465785, 35.381957681677
[I 2026-03-15 10:35:51,149] Trial 54 finished with values: [1.0424414326776867, 8.874439737564
[I 2026-03-15 10:35:51,152] Trial 55 finished with values: [4.922817616709324, 38.672756418558
[I 2026-03-15 10:35:51,155] Trial 56 finished with values: [37.39414200467217, 4.4735421688257
[I 2026-03-15 10:35:51,157] Trial 57 finished with values: [25.40332749751983, 1.0819564910514
[I 2026-03-15 10:35:51,159] Trial 58 finished with values: [42.8727303187444, 6.49091343158744
[I 2026-03-15 10:35:51,160] Trial 59 finished with values: [14.037736104596489, 0.064162565657
[I 2026-03-15 10:35:51,162] Trial 60 finished with values: [9.69218299550867, 0.786366170667231
[I 2026-03-15 10:35:51,163] Trial 61 finished with values: [18.374165256117653, 0.082087798029
[I 2026-03-15 10:35:51,165] Trial 62 finished with values: [1.6573346262643114, 27.956335358418
[I 2026-03-15 10:35:51,167] Trial 63 finished with values: [11.781827873591784, 0.322094009078
[I 2026-03-15 10:35:51,169] Trial 64 finished with values: [17.5615885170956, 0.03634929208605
[I 2026-03-15 10:35:51,172] Trial 65 finished with values: [2.4744522253756673, 5.890138878035
[I 2026-03-15 10:35:51,174] Trial 66 finished with values: [31.256572222006987, 2.530510219987
[I 2026-03-15 10:35:51,176] Trial 67 finished with values: [1.0444222875241573, 25.22018074784
[I 2026-03-15 10:35:51,178] Trial 68 finished with values: [39.118689732138904, 5.082741226520
[I 2026-03-15 10:35:51,180] Trial 69 finished with values: [8.230411439202923, 47.181366367244
[I 2026-03-15 10:35:51,182] Trial 70 finished with values: [0.5219306690727248, 22.30151221280
[I 2026-03-15 10:35:51,183] Trial 71 finished with values: [25.128596793974932, 1.025851316708
[I 2026-03-15 10:35:51,187] Trial 72 finished with values: [48.229952980448346, 8.671722350336
```

```
[I 2026-03-15 10:35:51,189] Trial 73 finished with values: [1.3331341460830772, 26.57004842246
[I 2026-03-15 10:35:51,190] Trial 74 finished with values: [0.00470231646444529, 16.5532898242
[I 2026-03-15 10:35:51,192] Trial 75 finished with values: [0.0841415797758078, 18.40471502305
[I 2026-03-15 10:35:51,194] Trial 76 finished with values: [0.5079462316734894, 22.20957396654
[I 2026-03-15 10:35:51,196] Trial 77 finished with values: [39.18200980226962, 5.1055819409968
[I 2026-03-15 10:35:51,197] Trial 78 finished with values: [11.664565857879447, 0.3418241414212
[I 2026-03-15 10:35:51,199] Trial 79 finished with values: [34.12089556200564, 3.3904203240160
```

```
x=-1.958 | f1=15.662 | f2=0.002
x=-1.954 | f1=15.633 | f2=0.002
x=-1.883 | f1=15.077 | f2=0.014
x=-1.747 | f1=14.038 | f2=0.064
x=-1.432 | f1=11.782 | f2=0.322
x=-1.415 | f1=11.665 | f2=0.342
x=-1.336 | f1=11.131 | f2=0.440
x=-1.255 | f1=10.592 | f2=0.556
Pareto-optimal trials: 27
```

The important shift is conceptual: Optuna is no longer returning one optimum. It is returning a frontier of non-dominated solutions that reflect different engineering trade-offs.

9.3. When Does the Search Space Become Part of the Algorithm?

Earlier chapters treated the search space as something you define once and then optimize over. In practice, many workflows are conditional.

If a trial selects a linear model, you want regularization strength. If it selects a neural network, you want layers, hidden units, and dropout. The structure of the search space changes with the sampled path.

This is where Optuna's define-by-run design becomes decisive. You can express branching logic directly in the objective function.

```python
def objective(trial):
    model = trial.suggest_categorical("model", ["linear", "mlp"])

    if model == "linear":
        alpha = trial.suggest_float("alpha", 1e-4, 1.0, log=True)
        return train_linear_model(alpha)

    n_layers = trial.suggest_int("n_layers", 1, 3)
    widths = []
    for layer_index in range(n_layers):
        widths.append(trial.suggest_int(f"units_{layer_index}", 32, 256))
    dropout = trial.suggest_float("dropout", 0.0, 0.5)
    return train_mlp(widths, dropout)
```

This is the natural continuation of the search-space design choices from Designing Search Spaces. The difference here is emphasis: the search space is not just descriptive; it becomes an adaptive program that encodes model families and structural decisions.

9.4. What Changes in Multi-Objective Optimization?

In single-objective optimization, we solve:

$$x^* = \arg\min_{x \in \mathcal{X}} f(x)$$

In multi-objective optimization, the target becomes a vector:

$$\mathbf{f}(x) = (f_1(x), f_2(x), \dots, f_k(x))$$

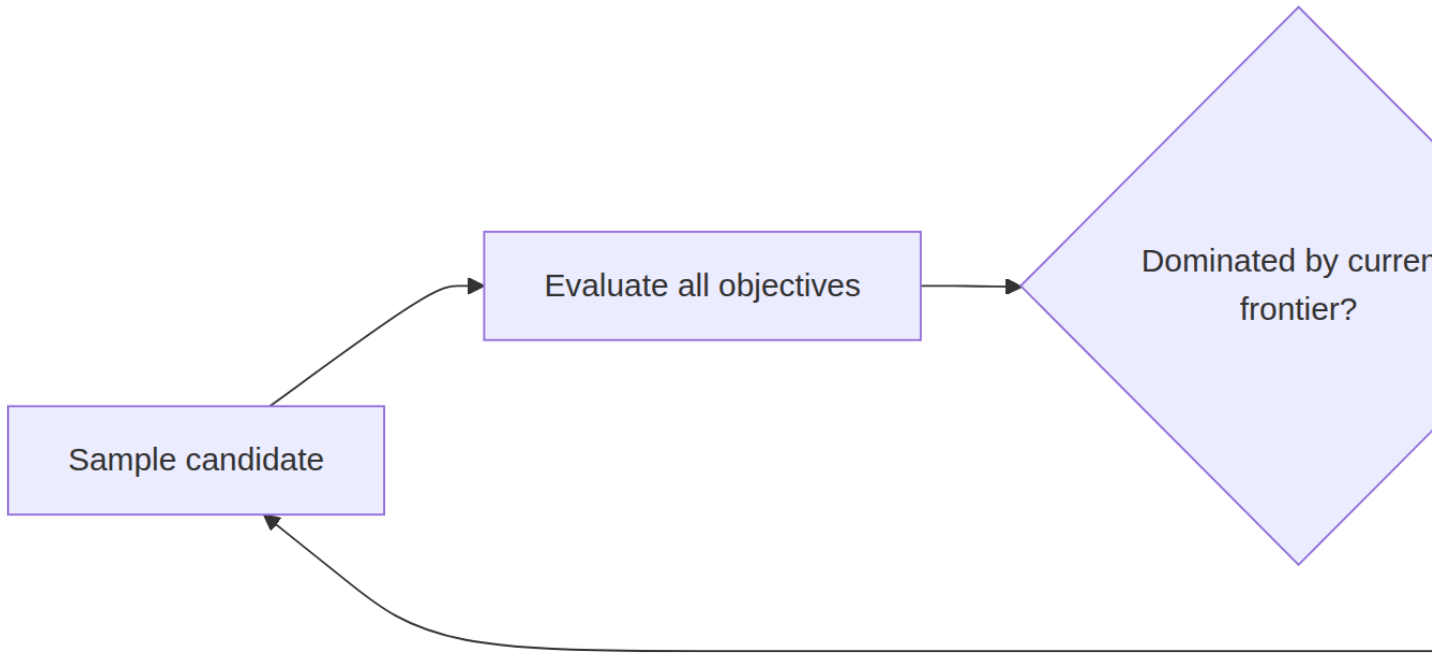
There is usually no total ordering over all candidates, because improving one objective often worsens another. Instead, we use dominance.

A candidate x_a dominates x_b if it is no worse on every objective and strictly better on at least one.

The Pareto front is the set of non-dominated solutions. In Optuna, those solutions are exposed through `study.best_trials` for multi-objective studies.

Configuration	Accuracy	Latency	Dominated?
A	0.95	45 ms	No
B	0.94	18 ms	No
C	0.92	16 ms	No
D	0.93	40 ms	Yes

Configuration D is dominated because another configuration can be both more accurate and faster.



For advanced practitioners, this matters because model selection stops being a single scalar decision. The optimizer produces a set of candidates, and downstream decision-making becomes a policy question: which trade-off is operationally acceptable?

9.5. How Should You Read a Pareto Front?

A Pareto front is useful only if you treat it as a decision surface rather than as a ranking.

Three practical interpretations are common:

- Knee-point selection: choose the point where small objective gains start costing a lot elsewhere.
- Budget-based selection: choose the best solution under a latency, memory, or cost cap.
- Portfolio selection: keep several frontier points for A/B tests or environment-specific deployment.

That distinction matters because the optimizer cannot decide your business trade-off for you. It can only expose the feasible frontier cleanly.

9.6. How Do Constraints Enter the Search?

Some requirements are not preferences but hard limits. You may need:

- latency under 20 ms
- model size under 200 MB
- memory usage below device capacity
- parameter combinations that satisfy physical or safety rules

There are two broad strategies.

9.6.1. Penalty-Based Constraints

The simplest approach is to encode constraint violations directly in the objective value.

```
def objective(trial):
    x = trial.suggest_float("x", -5.0, 5.0)
    y = trial.suggest_float("y", -5.0, 5.0)

    value = x**2 + y**2
    violation = max(0.0, x + y - 2.0)

    return value + 1000.0 * violation
```

This is easy to implement, but it mixes feasibility and quality into one scalar. A large penalty may make the landscape unstable, while a small penalty may let invalid solutions survive too long.

9.6.2. Constraint-Aware Multi-Objective Search

For algorithms that support explicit constraints, it is cleaner to separate objective values from feasibility information.

```
def objective(trial):
    x = trial.suggest_float("x", -5.0, 5.0)
    y = trial.suggest_float("y", -5.0, 5.0)
    trial.set_user_attr("constraint", x + y - 2.0)
    return x**2, y**2


def constraints_func(frozen_trial):
    return (frozen_trial.user_attrs["constraint"],)


sampler = optuna.samplers.NSGAIISampler(constraints_func=constraints_func)
```

```
/tmp/ipykernel_153209/2264972085.py:12: ExperimentalWarning: Argument ``constraints_func`` is a
  sampler = optuna.samplers.NSGAIISampler(constraints_func=constraints_func)
```

The rule is simple: negative or zero values indicate feasible solutions, while positive values indicate violations. This keeps the optimization objective readable and the constraint handling explicit.

9.7. When Should You Write a Custom Sampler?

Most Optuna workloads should start with the built-in samplers from Samplers. A custom sampler is justified only when you need search behavior that Optuna does not already implement.

Typical reasons include:

- integrating a research algorithm
- reproducing a proprietary search strategy
- injecting domain heuristics
- combining Optuna with an external optimizer that proposes candidates

At the API level, a custom sampler subclasses `BaseSampler` and decides how new parameter values are proposed.

```
from optuna.samplers import BaseSampler


class MySampler(BaseSampler):
    def infer_relative_search_space(self, study, trial):
        return {}

    def sample_relative(self, study, trial, search_space):
        return {}

    def sample_independent(self, study, trial, param_name, param_distribution):
        return param_distribution.to_external_repr(0.5)
```

In practice, writing a strong sampler is significantly harder than wiring up the class. You are taking responsibility for the exploration-exploitation policy, the statistical assumptions, and the failure modes that Optuna's built-in samplers already handle for you.

9.8. When Should You Write a Custom Pruner?

The same caution applies to pruners. Optuna already ships with median pruning, successive halving, Hyperband, and related policies discussed in Pruning. A custom pruner is worthwhile only when your stopping logic depends on domain knowledge unavailable to the built-in rules.

Examples include:

- simulation pipelines with custom convergence criteria
- training systems with resource-aware stop rules
- domain-specific safety checks that should terminate invalid trials early

```
from optuna.pruners import BasePruner


class ThresholdPruner(BasePruner):
    def prune(self, study, trial):
        if trial.last_step is None:
            return False

        latest_value = trial.intermediate_values[trial.last_step]
        return latest_value > 0.5
```

The engineering question is not whether you can subclass `BasePruner`, but whether the custom rule produces better resource allocation than the standard pruners with less maintenance risk.

> ⚠ Common Mistake
>
> Do not treat multi-objective optimization as if it should produce one universally best trial. If you immediately collapse several objectives into a single score without a clear reason, you throw away the main benefit of the Pareto formulation and hide the trade-offs you were supposed to inspect.

9.9. Practical Exercise: Constrained Multi-Objective Optimization with NSGA-II

Extend the fast first win into a more realistic workflow.

Build a study with two objectives and one explicit feasibility rule:

$$g(x, y) = x + y - 1 \leq 0$$

Use NSGA-II to minimize:

$$f_1(x, y) = (x - 1)^2 + y^2$$

and

$$f_2(x, y) = x^2 + (y - 1)^2$$

Work through the exercise in four passes:

1. Reproduce the study with `directions=["minimize", "minimize"]`.
2. Attach the constraint value to each trial with `trial.set_user_attr(...)` and pass `constraints_func` into `NSGAIISampler`.

3. Print the feasible Pareto-optimal trials and verify that infeasible points are filtered out of the frontier.
4. Change the constraint to $x + y - 0.25 \leq 0$ and compare how the frontier shrinks.

Questions to answer after the run:

- Which points survive after the feasible region tightens?
- Does the new frontier move toward one axis or remain balanced?
- If you were deploying under a strict latency or memory budget, which frontier point would you pick and why?

9.10. Recap

This chapter introduced the Optuna features that matter once optimization becomes structurally complex.

- Dynamic parameter spaces let the search space depend on earlier trial decisions.
- Multi-objective studies return Pareto-optimal sets rather than one scalar optimum.
- Constraint handling separates feasibility from pure objective quality.
- Custom samplers and pruners are extension points, but they shift algorithmic responsibility onto you.

9.11. Decision Guide

Use the simplest mechanism that matches the problem.

- Use dynamic spaces when the model family or pipeline structure depends on trial choices.
- Use multi-objective optimization when competing metrics should remain visible instead of being collapsed prematurely.
- Use penalty methods for quick prototypes, but prefer explicit constraint handling when feasibility is a first-class concern.
- Use custom samplers or pruners only when built-in Optuna components cannot express the domain logic you need.

The next chapter turns these advanced runs into something inspectable. In Optuna Visualization and Diagnostics, we will analyze optimization histories, parameter importance, and failure patterns so that search behavior becomes easier to interpret and improve.

10. Visualization and Diagnostics

This chapter is about reading an optimization process rather than merely running one. By the end, you should be able to use Optuna's visualization tools to diagnose convergence, identify influential parameters, detect bad search-space design, and decide when a study needs redesign rather than more trials.

10.1. Why This Matters

An optimizer can fail quietly. A study may appear to run correctly while wasting budget on irrelevant parameters, spending trials in the wrong scale, or bouncing around a noisy objective with little real progress.

That is why diagnostics matter. Once you can inspect optimization history, parameter importance, and interactions, you stop treating hyperparameter tuning as a black box and start treating it as an iterative engineering process.

10.2. Fast First Win: Build a Study Worth Diagnosing

To make the diagnostic tools meaningful, start with a study that is deliberately imperfect: a noisy objective with a search range that is wider than necessary.

```
import random

import optuna


def objective(trial):
    x = trial.suggest_float("x", -100.0, 100.0)
    y = trial.suggest_float("y", -100.0, 100.0)

    signal = (x - 3.0) ** 2 + (y + 2.0) ** 2
    noise = random.gauss(0.0, 1.0)
    return signal + noise


sampler = optuna.samplers.TPESampler(seed=42)
study = optuna.create_study(direction="minimize", sampler=sampler)
study.optimize(objective, n_trials=120)
```

```
best_so_far = []
incumbent = float("inf")
for trial in study.trials:
    if trial.value is not None:
        incumbent = min(incumbent, trial.value)
        best_so_far.append(round(incumbent, 3))

print("Best value:", round(study.best_value, 3))
print("Best params:", study.best_params)
print("First 10 incumbent values:", best_so_far[:10])
print("Last 10 incumbent values:", best_so_far[-10:])
```

```
[I 2026-03-15 10:35:57,627] A new study created in memory with name: no-name-cab85ef8-f0ad-4d3
[I 2026-03-15 10:35:57,629] Trial 0 finished with value: 9279.111815792805 and parameters: {'x
[I 2026-03-15 10:35:57,631] Trial 1 finished with value: 2355.969401098084 and parameters: {'x
[I 2026-03-15 10:35:57,632] Trial 2 finished with value: 9616.290593065167 and parameters: {'x
[I 2026-03-15 10:35:57,633] Trial 3 finished with value: 14010.38328627961 and parameters: {'x
[I 2026-03-15 10:35:57,634] Trial 4 finished with value: 2198.6233852113733 and parameters: {'
[I 2026-03-15 10:35:57,636] Trial 5 finished with value: 18991.44010053415 and parameters: {'x
[I 2026-03-15 10:35:57,639] Trial 6 finished with value: 7114.472672739325 and parameters: {'x
[I 2026-03-15 10:35:57,640] Trial 7 finished with value: 8200.725338215443 and parameters: {'x
[I 2026-03-15 10:35:57,641] Trial 8 finished with value: 1826.1936832783372 and parameters: {'
[I 2026-03-15 10:35:57,643] Trial 9 finished with value: 1856.0570780925648 and parameters: {'
[I 2026-03-15 10:35:57,648] Trial 10 finished with value: 2051.542207583607 and parameters: {'
[I 2026-03-15 10:35:57,653] Trial 11 finished with value: 197.22897836473365 and parameters: {
[I 2026-03-15 10:35:57,657] Trial 12 finished with value: 224.44457763860564 and parameters: {
[I 2026-03-15 10:35:57,662] Trial 13 finished with value: 729.4817593528223 and parameters: {'
[I 2026-03-15 10:35:57,666] Trial 14 finished with value: 16558.155102662815 and parameters: {
[I 2026-03-15 10:35:57,671] Trial 15 finished with value: 1790.435409729045 and parameters: {'
[I 2026-03-15 10:35:57,676] Trial 16 finished with value: 1970.488275106413 and parameters: {'
[I 2026-03-15 10:35:57,682] Trial 17 finished with value: 714.3628909062634 and parameters: {'
[I 2026-03-15 10:35:57,688] Trial 18 finished with value: 1225.5942315785283 and parameters: {
[I 2026-03-15 10:35:57,692] Trial 19 finished with value: 6044.243758781552 and parameters: {'
[I 2026-03-15 10:35:57,696] Trial 20 finished with value: 3621.215891549256 and parameters: {'
[I 2026-03-15 10:35:57,702] Trial 21 finished with value: 805.4328570386513 and parameters: {'
[I 2026-03-15 10:35:57,707] Trial 22 finished with value: 858.9889208337835 and parameters: {'
[I 2026-03-15 10:35:57,713] Trial 23 finished with value: 1396.9723032883876 and parameters: {
[I 2026-03-15 10:35:57,718] Trial 24 finished with value: 1067.86735242103 and parameters: {'x
[I 2026-03-15 10:35:57,724] Trial 25 finished with value: 3955.0078524951323 and parameters: {
[I 2026-03-15 10:35:57,728] Trial 26 finished with value: 910.0449899272091 and parameters: {'
[I 2026-03-15 10:35:57,734] Trial 27 finished with value: 3153.8264279209193 and parameters: {
[I 2026-03-15 10:35:57,739] Trial 28 finished with value: 4361.992132527291 and parameters: {'
[I 2026-03-15 10:35:57,743] Trial 29 finished with value: 6737.693777331981 and parameters: {'
[I 2026-03-15 10:35:57,748] Trial 30 finished with value: 258.0371348169175 and parameters: {'
[I 2026-03-15 10:35:57,754] Trial 31 finished with value: 315.10073187805625 and parameters: {
[I 2026-03-15 10:35:57,759] Trial 32 finished with value: 1079.4894050666257 and parameters: {
[I 2026-03-15 10:35:57,764] Trial 33 finished with value: 197.37766476803662 and parameters: {
```

```
[I 2026-03-15 10:35:57,769] Trial 34 finished with value: 1792.09506164506 and parameters: {'x
[I 2026-03-15 10:35:57,773] Trial 35 finished with value: 2771.367118595126 and parameters: {'
[I 2026-03-15 10:35:57,778] Trial 36 finished with value: 1217.0808230261443 and parameters: {
[I 2026-03-15 10:35:57,784] Trial 37 finished with value: 2181.4276401373945 and parameters: {
[I 2026-03-15 10:35:57,789] Trial 38 finished with value: 989.9408631361832 and parameters: {'
[I 2026-03-15 10:35:57,793] Trial 39 finished with value: 1030.562025465599 and parameters: {'
[I 2026-03-15 10:35:57,798] Trial 40 finished with value: 5082.31517478037 and parameters: {'x
[I 2026-03-15 10:35:57,803] Trial 41 finished with value: 257.1757217645765 and parameters: {'
[I 2026-03-15 10:35:57,807] Trial 42 finished with value: 21.263711029340747 and parameters: {
[I 2026-03-15 10:35:57,812] Trial 43 finished with value: 40.75247813235404 and parameters: {'
[I 2026-03-15 10:35:57,816] Trial 44 finished with value: 39.22702375709464 and parameters: {'
[I 2026-03-15 10:35:57,821] Trial 45 finished with value: 1171.1431413605508 and parameters: {
[I 2026-03-15 10:35:57,826] Trial 46 finished with value: 1903.8260043688663 and parameters: {
[I 2026-03-15 10:35:57,833] Trial 47 finished with value: 2694.644568592813 and parameters: {'
[I 2026-03-15 10:35:57,838] Trial 48 finished with value: 3268.2374327355387 and parameters: {
[I 2026-03-15 10:35:57,844] Trial 49 finished with value: 70.32775443587728 and parameters: {'
[I 2026-03-15 10:35:57,851] Trial 50 finished with value: 1100.5745606402108 and parameters: {
[I 2026-03-15 10:35:57,857] Trial 51 finished with value: 142.65840032114107 and parameters: {
[I 2026-03-15 10:35:57,864] Trial 52 finished with value: 754.2073116736667 and parameters: {'
[I 2026-03-15 10:35:57,870] Trial 53 finished with value: 128.49357904026877 and parameters: {
[I 2026-03-15 10:35:57,876] Trial 54 finished with value: 1241.5401299673345 and parameters: {
[I 2026-03-15 10:35:57,884] Trial 55 finished with value: 651.4250569114166 and parameters: {'
[I 2026-03-15 10:35:57,890] Trial 56 finished with value: 666.5254140171617 and parameters: {'
[I 2026-03-15 10:35:57,896] Trial 57 finished with value: 902.4684616667058 and parameters: {'
[I 2026-03-15 10:35:57,903] Trial 58 finished with value: 5680.3695866395265 and parameters: {
[I 2026-03-15 10:35:57,909] Trial 59 finished with value: 2592.0742891327664 and parameters: {
[I 2026-03-15 10:35:57,916] Trial 60 finished with value: 543.4168602065148 and parameters: {'
[I 2026-03-15 10:35:57,921] Trial 61 finished with value: 148.5909310008524 and parameters: {'
[I 2026-03-15 10:35:57,927] Trial 62 finished with value: 136.46962754598735 and parameters: {
[I 2026-03-15 10:35:57,936] Trial 63 finished with value: 604.3293152497179 and parameters: {'
[I 2026-03-15 10:35:57,941] Trial 64 finished with value: 100.9480339316388 and parameters: {'
[I 2026-03-15 10:35:57,947] Trial 65 finished with value: 95.95455203572207 and parameters: {'
[I 2026-03-15 10:35:57,952] Trial 66 finished with value: 663.6112775487492 and parameters: {'
[I 2026-03-15 10:35:57,957] Trial 67 finished with value: 1127.4629820111027 and parameters: {
[I 2026-03-15 10:35:57,962] Trial 68 finished with value: 2653.9952261749077 and parameters: {
[I 2026-03-15 10:35:57,967] Trial 69 finished with value: 2982.0459513131977 and parameters: {
[I 2026-03-15 10:35:57,972] Trial 70 finished with value: 81.85034416413845 and parameters: {'
[I 2026-03-15 10:35:57,978] Trial 71 finished with value: 79.31563569151172 and parameters: {'
[I 2026-03-15 10:35:57,985] Trial 72 finished with value: 178.13048370672144 and parameters: {
[I 2026-03-15 10:35:57,990] Trial 73 finished with value: 10.390507804830103 and parameters: {
[I 2026-03-15 10:35:57,996] Trial 74 finished with value: 370.15538119786015 and parameters: {
[I 2026-03-15 10:35:58,001] Trial 75 finished with value: 4.699092981988805 and parameters: {'
[I 2026-03-15 10:35:58,006] Trial 76 finished with value: 398.2889246875177 and parameters: {'
[I 2026-03-15 10:35:58,011] Trial 77 finished with value: 771.7111315962012 and parameters: {'
[I 2026-03-15 10:35:58,016] Trial 78 finished with value: 1460.345249513596 and parameters: {'
[I 2026-03-15 10:35:58,021] Trial 79 finished with value: 457.24185724418237 and parameters: {
[I 2026-03-15 10:35:58,026] Trial 80 finished with value: 15.603119041908688 and parameters: {
[I 2026-03-15 10:35:58,033] Trial 81 finished with value: 364.81939293485027 and parameters: {
```

```
[I 2026-03-15 10:35:58,038] Trial 82 finished with value: 757.368507764165 and parameters: {'x
[I 2026-03-15 10:35:58,044] Trial 83 finished with value: 776.7781453727608 and parameters: {'
[I 2026-03-15 10:35:58,049] Trial 84 finished with value: 10046.268544441342 and parameters: {
[I 2026-03-15 10:35:58,054] Trial 85 finished with value: 7574.9510923215785 and parameters: {
[I 2026-03-15 10:35:58,060] Trial 86 finished with value: 266.7827023919912 and parameters: {'
[I 2026-03-15 10:35:58,065] Trial 87 finished with value: 65.3099876402943 and parameters: {'x
[I 2026-03-15 10:35:58,071] Trial 88 finished with value: 1118.3483005369735 and parameters: {
[I 2026-03-15 10:35:58,079] Trial 89 finished with value: 1159.705607893284 and parameters: {'
[I 2026-03-15 10:35:58,084] Trial 90 finished with value: 317.67161261348014 and parameters: {
[I 2026-03-15 10:35:58,089] Trial 91 finished with value: 393.71954955663284 and parameters: {
[I 2026-03-15 10:35:58,095] Trial 92 finished with value: 312.26230224222957 and parameters: {
[I 2026-03-15 10:35:58,100] Trial 93 finished with value: 69.04865775360346 and parameters: {'
[I 2026-03-15 10:35:58,105] Trial 94 finished with value: 573.3930349004495 and parameters: {'
[I 2026-03-15 10:35:58,113] Trial 95 finished with value: 80.66915624497848 and parameters: {'
[I 2026-03-15 10:35:58,118] Trial 96 finished with value: 232.0709853734094 and parameters: {'
[I 2026-03-15 10:35:58,123] Trial 97 finished with value: 323.0228346938232 and parameters: {'
[I 2026-03-15 10:35:58,130] Trial 98 finished with value: 227.8864820472947 and parameters: {'
[I 2026-03-15 10:35:58,134] Trial 99 finished with value: 206.4661983694811 and parameters: {'
[I 2026-03-15 10:35:58,140] Trial 100 finished with value: 597.5616074266061 and parameters: {
[I 2026-03-15 10:35:58,146] Trial 101 finished with value: 103.41737242533898 and parameters:
[I 2026-03-15 10:35:58,151] Trial 102 finished with value: 95.67430176424516 and parameters: {
[I 2026-03-15 10:35:58,156] Trial 103 finished with value: 184.58694372984476 and parameters:
[I 2026-03-15 10:35:58,162] Trial 104 finished with value: 78.17331648465307 and parameters: {
[I 2026-03-15 10:35:58,167] Trial 105 finished with value: 283.25079127417064 and parameters:
[I 2026-03-15 10:35:58,174] Trial 106 finished with value: 88.110538042248 and parameters: {'x
[I 2026-03-15 10:35:58,181] Trial 107 finished with value: 505.8368159445195 and parameters: {
[I 2026-03-15 10:35:58,186] Trial 108 finished with value: 535.3650981934296 and parameters: {
[I 2026-03-15 10:35:58,194] Trial 109 finished with value: 86.49535774684637 and parameters: {
[I 2026-03-15 10:35:58,199] Trial 110 finished with value: 176.91007512828358 and parameters:
[I 2026-03-15 10:35:58,204] Trial 111 finished with value: 93.59071595607074 and parameters: {
[I 2026-03-15 10:35:58,210] Trial 112 finished with value: 9442.780396080518 and parameters: {
[I 2026-03-15 10:35:58,216] Trial 113 finished with value: 69.65769993411881 and parameters: {
[I 2026-03-15 10:35:58,221] Trial 114 finished with value: 243.3194549522551 and parameters: {
[I 2026-03-15 10:35:58,228] Trial 115 finished with value: 204.0113827607768 and parameters: {
[I 2026-03-15 10:35:58,234] Trial 116 finished with value: 15.001937190819936 and parameters:
[I 2026-03-15 10:35:58,240] Trial 117 finished with value: 466.8780856391032 and parameters: {
[I 2026-03-15 10:35:58,246] Trial 118 finished with value: 427.50895027955914 and parameters:
[I 2026-03-15 10:35:58,251] Trial 119 finished with value: 5744.243466013759 and parameters: {


Best value: 4.699
Best params: {'x': 0.871109160042193, 'y': -2.649595257916454}
First 10 incumbent values: [9279.112, 2355.969, 2355.969, 2355.969, 2198.623, 2198.623, 2198.62
Last 10 incumbent values: [4.699, 4.699, 4.699, 4.699, 4.699, 4.699, 4.699, 4.699, 4.699, 4.69
```

This example gives you something realistic to inspect: convergence under noise, a search space that is broader than needed, and two parameters that should both matter.

10.3. Which Tools Does Optuna Provide?

Optuna exposes its plotting tools through `optuna.visualization`, which generates interactive Plotly figures.

```
import optuna.visualization as vis

vis.plot_optimization_history(study)
vis.plot_param_importances(study)
vis.plot_slice(study)
vis.plot_parallel_coordinate(study)
vis.plot_contour(study)
```

If Plotly is not already installed in your environment, add it explicitly:

```
pip install plotly
```

For advanced studies, these plots are not presentation artifacts. They are debugging tools for your search strategy.

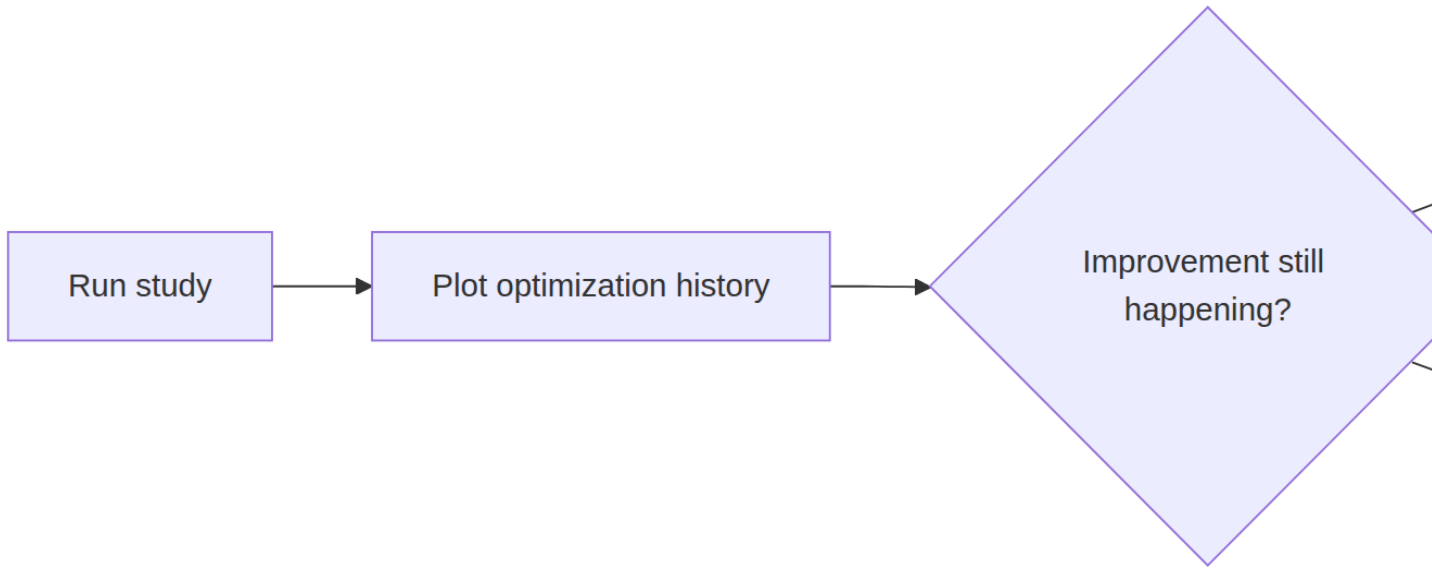
10.4. How Should You Read Optimization History?

The optimization history plot shows objective value as a function of trial number.

```
optuna.visualization.plot_optimization_history(study)
```

Two different curves matter conceptually:

- Trial values: the raw objective values observed at each evaluation.
- Best-so-far values: the incumbent trajectory, which shows whether the optimizer is still making meaningful progress.



Typical interpretations:

- Healthy convergence: the best-so-far curve keeps improving, even if raw trial values remain noisy.
- Stagnation: many trials pass without a better incumbent, suggesting a poor search space, insufficient exploration, or an almost-exhausted budget.
- High noise: raw values swing widely, but the incumbent barely changes, indicating evaluation instability or stochastic objectives.

Convergence plots are especially useful when read alongside Pruning, because apparent stagnation may mean the search space is bad, not merely that pruning is aggressive.

10.5. What Does Parameter Importance Actually Mean?

Parameter importance estimates how much each parameter contributes to variation in the objective.

```
optuna.visualization.plot_param_importances(study)
```

Optuna commonly uses fANOVA-style reasoning. At a high level, if total objective variation is $\mathrm{Var}(f(x))$, the importance of parameter x_i is approximated by the fraction of variance explained by changes in that parameter:

$$\mathrm{Importance}(x_i) = \frac{\mathrm{Var}_{x_i}\left(\mathbb{E}[f(x) \mid x_i]\right)}{\mathrm{Var}(f(x))}$$

Interpretation is practical rather than philosophical:

- High importance suggests the parameter meaningfully changes outcomes in the explored region.

- Low importance suggests the parameter is either genuinely weak or not yet explored in the right regime.

This distinction matters. Low importance does not automatically mean the parameter is useless. It may only mean that the current search space or trial budget has not revealed its effect yet.

10.6. How Do Slice Plots Expose Useful Structure?

Slice plots visualize one parameter against the objective.

```
optuna.visualization.plot_slice(study)
```

Use them to answer questions such as:

- Is there a narrow region where the objective improves sharply?
- Is the relationship monotonic, U-shaped, or mostly flat?
- Are strong results concentrated near one boundary of the search range?

This is one of the quickest ways to catch poorly designed spaces from Designing Search Spaces. If all strong trials cluster near one edge, your range is probably too wide or mis-scaled.

10.7. When Do Parallel Coordinates Help More Than Slices?

Slice plots show one parameter at a time. Parallel coordinate plots help when performance depends on combinations of parameters.

```
optuna.visualization.plot_parallel_coordinate(study)
```

Each line represents a single trial crossing multiple parameter axes and the objective value. This is useful for discovering patterns such as:

- strong trials combining high learning rates with low regularization
- a parameter becoming useful only when another parameter is within a narrow interval
- certain regions consistently producing weak results regardless of other choices

For advanced practitioners, this is the bridge between raw visualization and search-space redesign: you use the observed structure to introduce conditional or hierarchical spaces rather than continuing with a flat parameter definition.

10.8. What Do Contour Plots Add?

Contour plots focus on pairwise interactions between parameters.

```
optuna.visualization.plot_contour(study)
```

They are especially informative when good performance is not attached to one parameter alone, but to a combination. A contour plot can reveal a valley, ridge, or island of strong performance that slice plots would hide.

Common diagnostic outcomes:

- one parameter matters only when another is small
- two parameters compensate for one another
- a supposedly important parameter is important only because it interacts with another one

That is often the point where a search problem stops being "pick the right values" and becomes "represent the structure correctly."

10.9. How Do Visuals Reveal a Bad Search Space?

Most failed studies are not failures of the sampler alone. They are failures of problem formulation.

Typical warning signs include:

Symptom	Likely issue	Typical fix
Best trials sit on parameter boundaries	Range too wide or too narrow	Re-center or rescale the space
Slice plots are flat	Parameter is weak or irrelevant	Remove it or reduce its emphasis
Good trials cluster logarithmically	Linear scale is inappropriate	Use log sampling
Convergence stalls early	Space is too broad or too noisy	Narrow the space or stabilize the objective

This is why visualization is not an afterthought. It is how you decide whether the next 200 trials will be useful or wasteful.

10.10. How Do You Diagnose Parameter Interactions?

Interactions occur when the effect of one parameter depends on the value of another.

For example, a high learning rate may work only with a large batch size, while the same learning rate fails at smaller batches. If you ignore that interaction, you may incorrectly conclude that the learning rate itself is unstable.

The best tools for this are:

- contour plots for pairwise structure

- parallel coordinate plots for higher-dimensional patterns
- parameter importance plots to see whether interaction-heavy parameters are globally influential

These signals often justify a redesign into conditional search spaces or alternative samplers, which connects directly back to Advanced Optimization Techniques and Samplers.

10.11. When Is the Dashboard Worth Using?

Static analysis is useful, but long-running or collaborative studies benefit from a live interface.

```
optuna-dashboard sqlite:///example.db
```

The dashboard is especially useful when you want:

- live trial monitoring
- filtering and comparing subsets of runs
- interactive Pareto-front inspection for multi-objective studies
- lightweight review of a shared experiment store

If you are moving from one-off studies to an experiment workflow, the dashboard is often the fastest route to operational visibility.

> ⚠ Common Mistake
>
> Do not interpret diagnostic plots as ground truth when you have too few trials or a highly noisy objective. A parameter-importance ranking built from weak coverage can look precise while still being misleading.

10.12. Practical Exercise: Diagnose a Weak Study and Repair It

Use the study from the opening example as your baseline. Then generate the core diagnostics.

```
import optuna.visualization as vis

vis.plot_optimization_history(study)
vis.plot_param_importances(study)
vis.plot_slice(study)
vis.plot_parallel_coordinate(study)
vis.plot_contour(study)
```

Work through the exercise in four passes:

1. Inspect the optimization history and decide whether the study is still improving or mostly plateaued.

2. Use slice and importance plots to determine whether both `x` and `y` deserve the current range of `[-100, 100]`.
3. Redefine the objective with a narrower range such as `[-10, 10]` and rerun the study with the same trial budget.
4. Compare the two studies and explain whether the redesign improved convergence, interpretability, or both.

Questions to answer after the run:

- Did the incumbent improve earlier after narrowing the range?
- Did parameter importance become easier to interpret?
- Did the plots suggest a log scale, a narrower linear scale, or a conditional redesign?
- If the objective remained noisy, would you spend more trials or reduce evaluation noise first?

10.13. Recap

Diagnostics turn Optuna from a trial generator into an optimization workflow you can reason about.

- Optimization history tells you whether progress is real, stalled, or hidden by noise.
- Parameter-importance analysis helps prioritize which parameters deserve budget and attention.
- Slice, contour, and parallel coordinate plots reveal boundaries, scaling mistakes, and interactions.
- The dashboard is useful once studies become large, persistent, or collaborative.

10.14. Decision Guide

Use diagnostic tools as a redesign loop, not just as a report at the end.

- Start with optimization history when you want to know whether the budget is still producing value.
- Use slice plots when you suspect a parameter range or scale is poorly chosen.
- Use contour and parallel coordinate plots when performance depends on interactions.
- Use parameter importance to simplify the space, but only after checking that the study has enough coverage.
- Use the dashboard when repeated or long-running studies need live visibility.

The next chapter extends this diagnostic mindset into infrastructure. In Distributed and Scalable Optimization, we will look at storage backends, parallel execution, and the operational concerns that appear once optimization leaves the single-process notebook setting.

11. Distributed and Scalable Optimization

This chapter explains how to scale Optuna beyond the single-process workflow used in earlier modules. By the end, you should be able to choose between local parallelism and shared-storage distributed execution, understand the operational role of storage backends, and recognize the failure modes that appear once many workers collaborate on the same study.

11.1. Why This Matters

Many real optimization problems are limited less by the quality of the sampler than by the cost of objective evaluation. If a single trial takes minutes or hours, sequential optimization quickly becomes the real bottleneck.

Scaling Optuna is therefore not an infrastructure afterthought. It is part of the optimization design itself. The storage backend, worker model, and failure handling strategy determine whether a large study is reproducible, efficient, and resumable.

11.2. Fast First Win: Local Parallel Optimization

The simplest form of scaling is to run several trials concurrently on one machine.

```
import math
import time

import optuna


def objective(trial):
    x = trial.suggest_float("x", -10.0, 10.0)
    time.sleep(0.02)
    return (x - 2.0) ** 2 + math.sin(5.0 * x)


study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=40, n_jobs=4)

print("Best value:", round(study.best_value, 4))
print("Best params:", study.best_params)
print("Completed trials:", len(study.trials))
```

```
[I 2026-03-15 10:36:04,549] A new study created in memory with name: no-name-fee3457d-96f7-48d
[I 2026-03-15 10:36:04,586] Trial 0 finished with value: 37.19198374047841 and parameters: {'x
[I 2026-03-15 10:36:04,588] Trial 1 finished with value: 5.756559412310951 and parameters: {'x
[I 2026-03-15 10:36:04,589] Trial 2 finished with value: 5.497299314391726 and parameters: {'x
[I 2026-03-15 10:36:04,591] Trial 3 finished with value: 53.6244287316969 and parameters: {'x'
[I 2026-03-15 10:36:04,611] Trial 4 finished with value: 115.39988481716175 and parameters: {'
[I 2026-03-15 10:36:04,617] Trial 5 finished with value: -0.9622381659318704 and parameters: {
[I 2026-03-15 10:36:04,618] Trial 6 finished with value: 0.9053829833790574 and parameters: {'
[I 2026-03-15 10:36:04,621] Trial 7 finished with value: 6.649901110362538 and parameters: {'x
[I 2026-03-15 10:36:04,634] Trial 8 finished with value: 2.427610424057315 and parameters: {'x
[I 2026-03-15 10:36:04,640] Trial 9 finished with value: 31.120445081493944 and parameters: {'
[I 2026-03-15 10:36:04,646] Trial 10 finished with value: 48.33739338276106 and parameters: {'
[I 2026-03-15 10:36:04,647] Trial 11 finished with value: 66.75520652267139 and parameters: {'
[I 2026-03-15 10:36:04,661] Trial 12 finished with value: 53.139052958132936 and parameters: {
[I 2026-03-15 10:36:04,666] Trial 13 finished with value: 37.76397921136135 and parameters: {'
[I 2026-03-15 10:36:04,674] Trial 14 finished with value: 3.1007775750486912 and parameters: {
[I 2026-03-15 10:36:04,681] Trial 15 finished with value: 0.8902396170700966 and parameters: {
[I 2026-03-15 10:36:04,685] Trial 16 finished with value: 3.9679834109777246 and parameters: {
[I 2026-03-15 10:36:04,691] Trial 17 finished with value: 2.206347189654319 and parameters: {'
[I 2026-03-15 10:36:04,698] Trial 18 finished with value: 0.6679516265298118 and parameters: {
[I 2026-03-15 10:36:04,705] Trial 19 finished with value: 13.81701530984656 and parameters: {'
[I 2026-03-15 10:36:04,710] Trial 20 finished with value: 18.926603857386695 and parameters: {
[I 2026-03-15 10:36:04,715] Trial 21 finished with value: 15.438455690495362 and parameters: {
[I 2026-03-15 10:36:04,723] Trial 22 finished with value: 20.367772400476596 and parameters: {
[I 2026-03-15 10:36:04,729] Trial 23 finished with value: 16.76890023381202 and parameters: {'
[I 2026-03-15 10:36:04,734] Trial 24 finished with value: 0.643224014560684 and parameters: {'
[I 2026-03-15 10:36:04,740] Trial 25 finished with value: 1.5852122699856772 and parameters: {
[I 2026-03-15 10:36:04,748] Trial 26 finished with value: 1.3353462579733888 and parameters: {
[I 2026-03-15 10:36:04,754] Trial 27 finished with value: 1.0936986713289683 and parameters: {
[I 2026-03-15 10:36:04,759] Trial 28 finished with value: 1.6036969223684976 and parameters: {
[I 2026-03-15 10:36:04,765] Trial 29 finished with value: 15.374987505337423 and parameters: {
[I 2026-03-15 10:36:04,773] Trial 30 finished with value: 17.929463748218495 and parameters: {
[I 2026-03-15 10:36:04,778] Trial 31 finished with value: 13.363637714776436 and parameters: {
[I 2026-03-15 10:36:04,785] Trial 32 finished with value: 25.161654565180758 and parameters: {
[I 2026-03-15 10:36:04,790] Trial 33 finished with value: 26.455694130211263 and parameters: {
[I 2026-03-15 10:36:04,797] Trial 34 finished with value: 1.1856563804661788 and parameters: {
[I 2026-03-15 10:36:04,804] Trial 35 finished with value: 1.0407886336811696 and parameters: {
[I 2026-03-15 10:36:04,810] Trial 36 finished with value: 1.0355773248120115 and parameters: {
[I 2026-03-15 10:36:04,816] Trial 37 finished with value: 0.882784136127005 and parameters: {'
[I 2026-03-15 10:36:04,828] Trial 38 finished with value: 9.561011315192978 and parameters: {'
[I 2026-03-15 10:36:04,833] Trial 39 finished with value: 5.665415821954967 and parameters: {'

Best value: -0.9622
Best params: {'x': 2.1931928988462364}
Completed trials: 40
```

This does not make the optimization distributed, but it does make the concurrency model visible.

Multiple workers are evaluating different trials concurrently while the study coordinates results in one process.

11.3. What Does n_jobs Actually Buy You?

For many users, n_jobs is the first scaling lever:

```
study.optimize(objective, n_trials=100, n_jobs=4)
```

Conceptually, the scheduler is simple:

```
Trial 1 -> worker slot 1
Trial 2 -> worker slot 2
Trial 3 -> worker slot 3
Trial 4 -> worker slot 4
```

This is attractive because it requires no extra infrastructure, but it has limits.

- It is easiest when all workers live on one machine.
- It does not solve cross-machine coordination.
- CPU-bound Python workloads may not scale the same way as I/O-bound or external-process workloads.

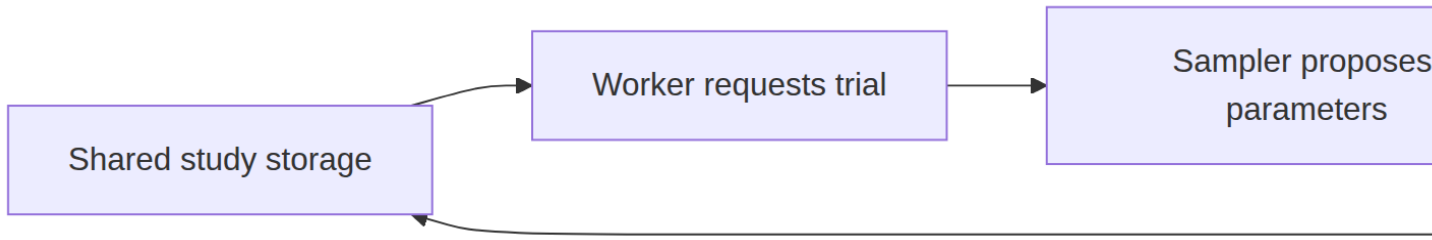
So n_jobs is the right place to start, but not the right answer for every large experiment.

11.4. When Do You Need Shared Storage?

Once multiple processes or machines need to collaborate on the same study, Optuna needs persistent shared state. That state lives in the storage backend.

At that point, each worker follows the same loop:

1. Ask storage for the next trial.
2. Evaluate the objective.
3. Write the result back.
4. Request another trial.



The critical design shift is that storage is no longer just for persistence. It becomes the coordination mechanism for the whole experiment.

11.5. Which Storage Backend Should You Use?

Optuna can work with several relational databases, but they are not interchangeable in practice.

Backend	Best fit	Main limitation
In-memory	quick local experiments	no persistence, no cross-process coordination
SQLite	small local shared studies	limited concurrency under heavier parallel load
PostgreSQL	serious distributed experiments	requires external service management
MySQL	distributed environments with existing MySQL ops	operational tuning still matters

11.5.1. In-Memory Storage

```
study = optuna.create_study(direction="minimize")
```

```
[I 2026-03-15 10:36:04,851] A new study created in memory with name: no-name-8165340a-722f-4c6
```

This is ideal for exploratory work, but once the process exits, the study is gone.

11.5.2. SQLite Storage

```
study = optuna.create_study(
    direction="minimize",
    study_name="demo",
    storage="sqlite:///example.db",
    load_if_exists=True,
)
```

SQLite is a useful stepping stone for local experiments and demonstrations. It is not the backend you should automatically choose for many concurrent workers across serious production workloads.

11.5.3. PostgreSQL or MySQL

```
study = optuna.create_study(
    direction="minimize",
    study_name="experiment",
    storage="postgresql://user:pass@host/db",
    load_if_exists=True,
)
```

For real distributed optimization, PostgreSQL is often the safest default because it handles concurrency, transactions, and failure recovery more robustly than SQLite.

11.6. How Does Distributed Execution Work?

Distributed Optuna usually follows a producer-consumer pattern, but the important detail is that workers are asynchronous.

They do not advance in lockstep. Fast workers complete more trials. Slow workers may still be evaluating while others have already requested new configurations.

That means the system can keep hardware busy even when objective runtimes vary widely.

```
Create study
   -> worker requests trial
   -> sampler proposes parameters
   -> worker evaluates objective
   -> result written to storage
   -> next available worker requests another trial
```

Asynchronous behavior is usually an advantage, because synchronization barriers would waste compute. But it also means your study history reflects real system timing, not a clean round-based schedule.

11.7. What Happens When Workers Fail?

Distributed experiments are only useful if they survive crashes, interruptions, and restarts.

If a worker dies mid-trial, Optuna does not lose the whole study. Completed trials remain in storage, and other workers can continue. This is the operational reason storage matters so much.

Typical failure scenarios include:

- a worker process crashes
- a machine is preempted
- a container restarts
- a cluster job times out

With persistent storage, the study can be resumed instead of recomputed from scratch.

This is especially important in long-running workloads where trial budgets are expensive and experiments span multiple sessions.

11.8. How Should You Think About Scalability?

There are different scaling regimes, and they imply different engineering choices.

Scale	Typical budget	Natural strategy
Small	50-200 trials	single process or `n_jobs`
Medium	hundreds to low thousands	multiple local or remote workers with persistent storage
Large	10,000+ trials	cluster execution with robust shared database and monitoring

The question is not simply "how many workers can I launch?" The real question is whether the objective function, storage backend, and monitoring stack can support that many workers without creating coordination or observability problems.

11.9. What Does a Multi-Worker Setup Look Like?

At the code level, all workers typically run the same script and point to the same study.

11.9.1. Step 1: Create or Load the Study

```
import optuna


study = optuna.create_study(
    study_name="distributed-demo",
    storage="sqlite:///example.db",
    load_if_exists=True,
    direction="minimize",
)
```

11.9.2. Step 2: Define the Worker Objective

```
def objective(trial):
    x = trial.suggest_float("x", -10.0, 10.0)
    return (x - 2.0) ** 2
```

11.9.3. Step 3: Run the Worker

```
study.optimize(objective, n_trials=100)
```

Launch the same worker script multiple times:

```
python worker.py
python worker.py
python worker.py
```

All workers collaborate through the same storage backend. That is the core distributed pattern.

11.10. How Do You Monitor a Large Study?

Once experiments become large, you need visibility while the optimization is still running.

Programmatic monitoring is the minimum level:

```
print("Best value:", study.best_value)
print("Completed trials:", len(study.trials))
```

For interactive monitoring, the dashboard is more useful:

```
optuna-dashboard sqlite:///example.db
```

This is especially helpful when you want to inspect trial progress, compare parameter regions, or review multi-objective frontiers without writing custom reporting code.

11.11. How Does Optuna Fit Into Larger Distributed Systems?

Optuna is often not the orchestrator itself. Instead, it is the optimization layer inside a broader execution environment.

Common combinations include:

System	Typical role
Ray	distributed ML workloads and task execution
Dask	parallel Python workflows
Kubernetes	container orchestration for scalable workers
SLURM	batch scheduling on HPC clusters

The pattern is usually the same:

- the external platform launches workers
- Optuna storage coordinates the study
- workers report results back into the shared study state

That separation of concerns is useful. Optuna handles optimization logic, while the surrounding platform handles resource scheduling.

> **Common Mistake**
>
> Do not assume SQLite is a production-grade answer for every distributed setup just because it works for a small demo. Once concurrency, fault tolerance, and worker count increase, backend limitations become part of the optimization bottleneck.

11.12. Practical Exercise: Turn a Local Study Into a Shared Study

Start from a simple single-process optimization script and upgrade it into a shared-storage workflow.

Use this objective:

```
import math

import optuna


def objective(trial):
    x = trial.suggest_float("x", -10.0, 10.0)
    return (x - 2.0) ** 2 + math.sin(5.0 * x)
```

Then work through the exercise in four passes:

1. Run the study in a single process and record the best value after 50 trials.
2. Run it again with `n_jobs=4` and compare wall-clock behavior.
3. Move the study to shared SQLite storage using `load_if_exists=True` and launch the worker script from multiple terminals.
4. Replace SQLite with PostgreSQL conceptually and list what operational risks improve when you move to a stronger backend.

Questions to answer after the run:

- Which part of the speedup came from concurrency and which part was still limited by the objective?
- Did local parallelism solve the problem, or did you still need shared storage for resumability?
- At what point would SQLite stop being a comfortable choice?
- If your workers lived on Kubernetes, Ray, or SLURM, what would Optuna handle and what would the platform handle?

11.13. Recap

Distributed Optuna is fundamentally about coordination, not just raw worker count.

- `n_jobs` provides the simplest local concurrency.
- Shared storage is required once multiple processes or machines collaborate on the same study.
- SQLite is useful for small shared experiments, while PostgreSQL or MySQL are better suited to serious distributed workloads.
- Persistent storage enables fault tolerance, resumability, and monitoring.

11.14. Decision Guide

Choose the simplest scaling model that matches the workload.

- Use a single process for cheap exploratory studies.
- Use `n_jobs` when one machine is enough and concurrency is the main need.
- Use shared storage when workers must coordinate across processes or hosts.
- Prefer PostgreSQL-class backends when the study is important enough that failures, locking, and concurrency behavior matter.
- Use external platforms such as Ray, Dask, Kubernetes, or SLURM for resource orchestration, while letting Optuna remain the optimization layer.

The next chapter moves from infrastructure back to applications. In Optuna for Machine Learning, we will apply the ideas from this book to realistic model-selection and pipeline-tuning workflows.

Part IV.

Part IV: Applications

12. Optuna for Machine Learning

This chapter applies Optuna to real machine-learning workflows rather than toy functions. By the end, you should be able to structure an Optuna study around robust model evaluation, tune scikit-learn pipelines safely, understand how Optuna integrates with major ML frameworks, and avoid the most common evaluation errors such as leakage and validation overfitting.

12.1. Why This Matters

In machine learning, a good optimizer is not enough. If the evaluation loop is flawed, Optuna will simply optimize the wrong target more efficiently.

That is why ML optimization is a distinct topic. The hard part is often not parameter suggestion itself, but building an objective function that reflects generalization performance, respects data boundaries, and remains reproducible across runs.

12.2. Fast First Win: Tune a Real Classifier Safely

Start with a compact but realistic workflow: a scikit-learn pipeline evaluated with cross-validation on a real dataset.

```
import optuna
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler


X, y = load_breast_cancer(return_X_y=True)


def objective(trial):
    n_estimators = trial.suggest_int("n_estimators", 50, 250)
    max_depth = trial.suggest_int("max_depth", 2, 16)
    min_samples_split = trial.suggest_int("min_samples_split", 2, 10)

    model = Pipeline(
        [
            ("scaler", StandardScaler()),
```

```
            (
                "clf",
                RandomForestClassifier(
                    n_estimators=n_estimators,
                    max_depth=max_depth,
                    min_samples_split=min_samples_split,
                    random_state=42,
                ),
            ),
        ]
    )

    scores = cross_val_score(model, X, y, cv=5, scoring="accuracy")
    return scores.mean()


sampler = optuna.samplers.TPESampler(seed=42)
study = optuna.create_study(direction="maximize", sampler=sampler)
study.optimize(objective, n_trials=30)

print("Best score:", round(study.best_value, 4))
print("Best params:", study.best_params)
```

```
[I 2026-03-15 10:36:16,150] A new study created in memory with name: no-name-6baf9ac8-68d5-4d2c
[I 2026-03-15 10:36:18,511] Trial 0 finished with value: 0.968390001552554 and parameters: {'n_
[I 2026-03-15 10:36:21,350] Trial 1 finished with value: 0.9578326346840553 and parameters: {'n
[I 2026-03-15 10:36:22,260] Trial 2 finished with value: 0.9595870206489675 and parameters: {'n
[I 2026-03-15 10:36:25,628] Trial 3 finished with value: 0.9473063188945815 and parameters: {'n
[I 2026-03-15 10:36:29,844] Trial 4 finished with value: 0.9596025461884802 and parameters: {'n
[I 2026-03-15 10:36:31,389] Trial 5 finished with value: 0.956078248719143 and parameters: {'n_
[I 2026-03-15 10:36:33,532] Trial 6 finished with value: 0.9596025461884802 and parameters: {'n
[I 2026-03-15 10:36:34,731] Trial 7 finished with value: 0.9578481602235678 and parameters: {'n
[I 2026-03-15 10:36:37,165] Trial 8 finished with value: 0.9578481602235678 and parameters: {'n
[I 2026-03-15 10:36:39,793] Trial 9 finished with value: 0.9543393882937432 and parameters: {'n
[I 2026-03-15 10:36:41,910] Trial 10 finished with value: 0.9578636857630803 and parameters: {
[I 2026-03-15 10:36:45,538] Trial 11 finished with value: 0.9613569321533924 and parameters: {
[I 2026-03-15 10:36:50,557] Trial 12 finished with value: 0.9613414066138798 and parameters: {
[I 2026-03-15 10:36:53,091] Trial 13 finished with value: 0.9578326346840551 and parameters: {
[I 2026-03-15 10:36:54,152] Trial 14 finished with value: 0.9631268436578171 and parameters: {
[I 2026-03-15 10:36:55,198] Trial 15 finished with value: 0.9613724576929048 and parameters: {
[I 2026-03-15 10:36:56,249] Trial 16 finished with value: 0.9631268436578171 and parameters: {
[I 2026-03-15 10:36:57,970] Trial 17 finished with value: 0.9631113181183046 and parameters: {
[I 2026-03-15 10:36:59,335] Trial 18 finished with value: 0.9631113181183046 and parameters: {
[I 2026-03-15 10:37:00,210] Trial 19 finished with value: 0.9631113181183046 and parameters: {
[I 2026-03-15 10:37:00,708] Trial 20 finished with value: 0.9613569321533924 and parameters: {
[I 2026-03-15 10:37:02,333] Trial 21 finished with value: 0.9631268436578171 and parameters: {
[I 2026-03-15 10:37:03,861] Trial 22 finished with value: 0.9613569321533924 and parameters: {
```

```
[I 2026-03-15 10:37:05,012] Trial 23 finished with value: 0.9631113181183046 and parameters: {
[I 2026-03-15 10:37:05,715] Trial 24 finished with value: 0.9613569321533924 and parameters: {
[I 2026-03-15 10:37:06,692] Trial 25 finished with value: 0.9578481602235678 and parameters: {
[I 2026-03-15 10:37:08,406] Trial 26 finished with value: 0.9613569321533924 and parameters: {
[I 2026-03-15 10:37:10,127] Trial 27 finished with value: 0.9578636857630803 and parameters: {
[I 2026-03-15 10:37:11,611] Trial 28 finished with value: 0.968390001552554 and parameters: {'
[I 2026-03-15 10:37:13,834] Trial 29 finished with value: 0.9613569321533924 and parameters: {


Best score: 0.9684
Best params: {'n_estimators': 125, 'max_depth': 16, 'min_samples_split': 8}
```

This example captures the essential structure of ML optimization with Optuna: define hyperparameters inside an objective, evaluate the model with a stable validation protocol, and optimize the metric that actually matters.

12.3. Why Are Pipelines So Important?

Real ML systems rarely consist of a model in isolation. They usually combine preprocessing, feature engineering, and a predictive model.

```
raw data
   -> preprocessing
   -> feature transformation
   -> model
   -> evaluation metric
```

In scikit-learn, pipelines are not just a convenience. They are a safety mechanism. They ensure every preprocessing step is fitted only on the training portion of each fold.

That is exactly what prevents leakage during hyperparameter optimization.

12.4. How Should You Structure a Scikit-Learn Objective?

The objective function should do three things:

1. Suggest hyperparameters.
2. Build a complete pipeline or estimator.
3. Return a robust validation score.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score


def objective(trial):
    n_estimators = trial.suggest_int("n_estimators", 50, 300)
    max_depth = trial.suggest_int("max_depth", 2, 20)

    model = Pipeline(
        [
            ("scaler", StandardScaler()),
            (
                "clf",
                RandomForestClassifier(
                    n_estimators=n_estimators,
                    max_depth=max_depth,
                    random_state=42,
                ),
            ),
        ]
    )

    return cross_val_score(model, X, y, cv=5).mean()
```

This pattern is intentionally simple, but it scales well to feature selectors, imputers, encoders, and different estimator families.

12.5. What Changes With Gradient Boosting Libraries?

Optuna is widely used with XGBoost and LightGBM because these models have large, interaction-heavy hyperparameter spaces and expensive training loops.

12.5.1. XGBoost Pattern

```
import xgboost as xgb


def objective(trial):
    params = {
        "max_depth": trial.suggest_int("max_depth", 3, 10),
        "learning_rate": trial.suggest_float("learning_rate", 1e-3, 0.3,
        ↪  log=True),
```

```
        "subsample": trial.suggest_float("subsample", 0.5, 1.0),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.5, 1.0),
    }

    model = xgb.XGBClassifier(**params)
    return cross_val_score(model, X, y, cv=5).mean()
```

12.5.2. LightGBM Pattern

```
import lightgbm as lgb


def objective(trial):
    params = {
        "num_leaves": trial.suggest_int("num_leaves", 16, 128),
        "learning_rate": trial.suggest_float("learning_rate", 1e-3, 0.1,
         ↪  log=True),
        "feature_fraction": trial.suggest_float("feature_fraction", 0.5, 1.0),
    }

    model = lgb.LGBMClassifier(**params)
    return cross_val_score(model, X, y, cv=5).mean()
```

These libraries are especially good candidates for Optuna because pruning can save a significant amount of training time when bad trials become obvious early.

12.6. How Does Optuna Fit Deep Learning Loops?

For PyTorch or TensorFlow, the objective function usually wraps a training loop rather than a ready-made estimator API.

12.6.1. PyTorch Pattern

```
def objective(trial):
    lr = trial.suggest_float("lr", 1e-5, 1e-2, log=True)
    hidden_size = trial.suggest_int("hidden_size", 32, 256)

    model = NeuralNet(hidden_size)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    for epoch in range(20):
```

```
        train(model)
        val_loss = validate(model)

        trial.report(val_loss, epoch)
        if trial.should_prune():
            raise optuna.TrialPruned()

    return val_loss
```

12.6.2. TensorFlow / Keras Pattern

```
def objective(trial):
    units = trial.suggest_int("units", 32, 256)
    lr = trial.suggest_float("lr", 1e-4, 1e-2, log=True)

    model = tf.keras.Sequential(
        [
            tf.keras.layers.Dense(units, activation="relu"),
            tf.keras.layers.Dense(1),
        ]
    )

    optimizer = tf.keras.optimizers.Adam(learning_rate=lr)
    model.compile(optimizer=optimizer, loss="mse")

    history = model.fit(
        X_train,
        y_train,
        validation_data=(X_val, y_val),
        epochs=20,
        verbose=0,
    )

    return min(history.history["val_loss"])
```

The main advantage here is flexibility. Optuna can tune architecture choices, optimizers, regularization, and early-stopping-aware training loops using the same study abstraction.

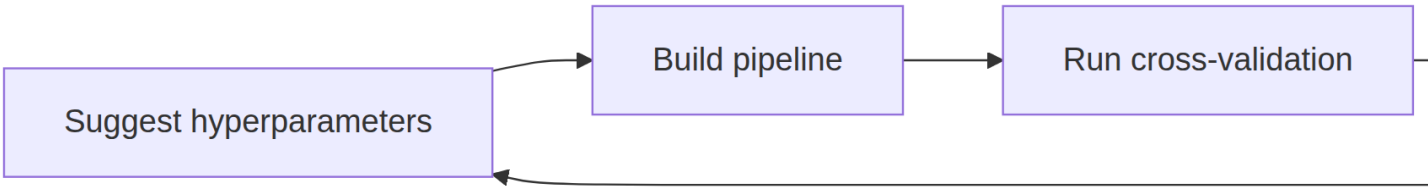
12.7. Why Is Cross-Validation Still Central?

Machine-learning objectives are vulnerable to noise, data splits, and accidental overfitting to one validation set.

Cross-validation reduces that instability by averaging performance over several train-validation partitions.

```
cross_val_score(model, X, y, cv=5)
```

For many tabular problems, this is the default baseline you should trust before reaching for more elaborate evaluation schemes.



This is also where ML optimization differs from simpler examples earlier in the book. The objective value is not just a formula; it is the output of an evaluation protocol.

12.8. When Do You Need Nested Cross-Validation?

If you want an unbiased estimate of the performance of the full optimization procedure, nested cross-validation is the correct design.

```
outer split
   -> training partition
      -> inner cross-validation
      -> Optuna hyperparameter search
   -> outer validation score
```

This matters because ordinary cross-validation inside Optuna estimates the best hyperparameters for a dataset, while nested cross-validation estimates the generalization performance of the whole tuning workflow.

It is more expensive, but it is the right tool when you care about honest model assessment rather than only parameter selection.

12.9. How Do You Avoid Data Leakage?

Leakage is one of the easiest ways to get optimistic results.

The classic mistake is to preprocess the full dataset before cross-validation.

```
Wrong: scale entire dataset -> then split into folds
Right: split into folds -> fit preprocessing inside each training fold
```

This is precisely why pipelines matter so much in ML optimization. They make the correct evaluation path the default path.

12.10. What Makes an ML Study Reproducible?

Reproducibility is not a cosmetic concern. It is the difference between tuning knowledge and tuning folklore.

Key practices include:

- fixing Optuna sampler seeds
- fixing framework random seeds where appropriate
- storing study results in persistent storage
- recording dataset versions and code versions
- making train/validation boundaries explicit

```
sampler = optuna.samplers.TPESampler(seed=42)
```

```
storage = "sqlite:///experiment.db"
```

Once optimization becomes part of a team workflow, these details matter as much as the sampler choice.

> ⚠ Common Mistake
>
> Do not optimize directly on a single validation split and treat the best score as trustworthy. Optuna will optimize whatever signal you provide, including leakage, unstable folds, and accidental validation-specific patterns.

12.11. Practical Exercise: End-to-End Pipeline Optimization

Extend the opening example into a more realistic ML workflow.

Use the breast-cancer dataset and optimize a scikit-learn pipeline with at least one preprocessing step and one model.

```
import optuna
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler


X, y = load_breast_cancer(return_X_y=True)


def objective(trial):
    n_estimators = trial.suggest_int("n_estimators", 50, 300)
```

```
    max_depth = trial.suggest_int("max_depth", 2, 20)
    min_samples_split = trial.suggest_int("min_samples_split", 2, 10)

    model = Pipeline(
        [
            ("scaler", StandardScaler()),
            (
                "clf",
                RandomForestClassifier(
                    n_estimators=n_estimators,
                    max_depth=max_depth,
                    min_samples_split=min_samples_split,
                    random_state=42,
                ),
            ),
        ]
    )

    return cross_val_score(model, X, y, cv=5, scoring="accuracy").mean()
```

Then work through the exercise in four passes:

1. Run a 30-trial study and record the best score and best parameter set.
2. Add one more hyperparameter, such as `max_features`, and check whether performance changes materially.
3. Replace the single estimator with a pipeline that includes feature selection or a different classifier.
4. Explain whether the search-space expansion improved the model or merely made optimization noisier.

Questions to answer after the run:

- Did cross-validation produce stable rankings between trials?
- Which hyperparameters actually mattered according to the study history or later visualization?
- Would nested cross-validation be worth the extra cost for this problem?
- Where would leakage appear if preprocessing were done outside the pipeline?

12.12. Recap

Optuna becomes genuinely useful in ML once the evaluation protocol is as carefully designed as the search itself.

- Pipelines prevent leakage and keep preprocessing inside the fold structure.
- Cross-validation usually provides a stronger objective than a single validation split.
- Gradient boosting and deep learning integrations benefit from Optuna because their spaces are large and training can be pruned.

- Reproducibility depends on seeds, storage, and explicit evaluation design.

12.13. Decision Guide

Choose the evaluation protocol before you choose the sampler.

- Use scikit-learn pipelines whenever preprocessing is part of the model workflow.
- Use cross-validation as the default for tabular ML unless cost is prohibitive.
- Use nested cross-validation when you need an unbiased estimate of the whole tuning procedure.
- Use pruning when the training loop exposes useful intermediate metrics.
- Treat reproducibility and leakage prevention as first-class parts of the optimization problem.

The next chapter broadens the scope again. In Optuna for General Optimization, we will leave machine learning behind and use Optuna as a general-purpose optimizer for mathematical, simulation-based, and algorithmic objectives.

13. Optuna for General Optimization

This chapter moves Optuna outside the usual machine-learning framing and treats it as a general black-box optimizer. By the end, you should be able to recognize which non-ML problems fit the Optuna workflow, design objective functions for simulations and algorithmic systems, and use benchmark functions to reason about optimizer behavior on difficult landscapes.

13.1. Why This Matters

It is easy to think of Optuna as a hyperparameter tuning library because that is where many users first encounter it. But the underlying abstraction is much broader: if you can write a Python function that evaluates a candidate configuration and returns a score, you can often optimize it with Optuna.

That makes Optuna relevant far beyond model tuning. Engineering calibration, scientific simulation, control design, numerical algorithms, reinforcement learning, and benchmark optimization all share the same structural problem: expensive black-box search over a space with limited analytic structure.

13.2. Fast First Win: Optimize a Non-Convex Function

The fastest way to see this generality is to leave datasets behind and optimize a purely mathematical objective.

Consider:

$$f(x) = (x - 3)^2 + \sin(5x)$$

This is not a trivial convex bowl. The sinusoidal term creates oscillations and several local basins, which is exactly the kind of landscape where naive search can waste evaluations.

```
import math

import optuna


def objective(trial):
    x = trial.suggest_float("x", -5.0, 5.0)
    return (x - 3.0) ** 2 + math.sin(5.0 * x)
```

```
study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=120)

print("Best value:", round(study.best_value, 5))
print("Best params:", study.best_params)
```

```
[I 2026-03-15 10:37:22,487] A new study created in memory with name: no-name-13eb6d3c-8130-48e
[I 2026-03-15 10:37:22,499] Trial 0 finished with value: 1.9147520393986757 and parameters: {'
[I 2026-03-15 10:37:22,501] Trial 1 finished with value: 3.219807368003844 and parameters: {'x
[I 2026-03-15 10:37:22,503] Trial 2 finished with value: 4.105451948355437 and parameters: {'x
[I 2026-03-15 10:37:22,504] Trial 3 finished with value: 31.604821420269136 and parameters: {'
[I 2026-03-15 10:37:22,506] Trial 4 finished with value: 3.0940824187121105 and parameters: {'
[I 2026-03-15 10:37:22,507] Trial 5 finished with value: 0.7876213507413785 and parameters: {'
[I 2026-03-15 10:37:22,509] Trial 6 finished with value: 45.52497021886874 and parameters: {'x
[I 2026-03-15 10:37:22,510] Trial 7 finished with value: 62.42563207704675 and parameters: {'x
[I 2026-03-15 10:37:22,511] Trial 8 finished with value: 46.51293517722147 and parameters: {'x
[I 2026-03-15 10:37:22,513] Trial 9 finished with value: 16.86448260069166 and parameters: {'x
[I 2026-03-15 10:37:22,523] Trial 10 finished with value: 2.209131025851065 and parameters: {'
[I 2026-03-15 10:37:22,526] Trial 11 finished with value: 3.8417696191977915 and parameters: {
[I 2026-03-15 10:37:22,534] Trial 12 finished with value: 0.14323358543575296 and parameters:
[I 2026-03-15 10:37:22,538] Trial 13 finished with value: 0.9535033978983539 and parameters: {
[I 2026-03-15 10:37:22,546] Trial 14 finished with value: 0.7827698842129106 and parameters: {
[I 2026-03-15 10:37:22,552] Trial 15 finished with value: 0.0075505359411412575 and parameters
[I 2026-03-15 10:37:22,556] Trial 16 finished with value: 13.679793755762025 and parameters: {
[I 2026-03-15 10:37:22,560] Trial 17 finished with value: 0.029142351219102136 and parameters:
[I 2026-03-15 10:37:22,565] Trial 18 finished with value: -0.07919952741187353 and parameters:
[I 2026-03-15 10:37:22,571] Trial 19 finished with value: 1.1106396753960948 and parameters: {
[I 2026-03-15 10:37:22,574] Trial 20 finished with value: 9.268537689584624 and parameters: {'
[I 2026-03-15 10:37:22,578] Trial 21 finished with value: 0.5368577956362043 and parameters: {
[I 2026-03-15 10:37:22,583] Trial 22 finished with value: 0.5746725953611865 and parameters: {
[I 2026-03-15 10:37:22,589] Trial 23 finished with value: -0.3894765882525952 and parameters:
[I 2026-03-15 10:37:22,593] Trial 24 finished with value: -0.34089470973255775 and parameters:
[I 2026-03-15 10:37:22,597] Trial 25 finished with value: -0.0750281492425584 and parameters:
[I 2026-03-15 10:37:22,602] Trial 26 finished with value: 8.467557734784457 and parameters: {'
[I 2026-03-15 10:37:22,606] Trial 27 finished with value: -0.39493246397016857 and parameters:
[I 2026-03-15 10:37:22,610] Trial 28 finished with value: 1.1930004275118742 and parameters: {
[I 2026-03-15 10:37:22,615] Trial 29 finished with value: 18.79629229624988 and parameters: {'
[I 2026-03-15 10:37:22,619] Trial 30 finished with value: 5.742510603364059 and parameters: {'
[I 2026-03-15 10:37:22,623] Trial 31 finished with value: 2.0273236558002115 and parameters: {
[I 2026-03-15 10:37:22,627] Trial 32 finished with value: 3.025478047636264 and parameters: {'
[I 2026-03-15 10:37:22,633] Trial 33 finished with value: 0.30195012461389226 and parameters:
[I 2026-03-15 10:37:22,637] Trial 34 finished with value: 0.627707576866425 and parameters: {'
[I 2026-03-15 10:37:22,641] Trial 35 finished with value: 3.947613846153508 and parameters: {'
[I 2026-03-15 10:37:22,647] Trial 36 finished with value: 3.2239639371532287 and parameters: {
[I 2026-03-15 10:37:22,651] Trial 37 finished with value: -0.7219661722033319 and parameters:
```

```
[I 2026-03-15 10:37:22,655] Trial 38 finished with value: 25.390362998297253 and parameters: {
[I 2026-03-15 10:37:22,659] Trial 39 finished with value: -0.4032263347886965 and parameters: -
[I 2026-03-15 10:37:22,665] Trial 40 finished with value: 2.253579777194866 and parameters: {'x
[I 2026-03-15 10:37:22,670] Trial 41 finished with value: -0.19067838723210778 and parameters:
[I 2026-03-15 10:37:22,673] Trial 42 finished with value: 3.111572792425561 and parameters: {'x
[I 2026-03-15 10:37:22,678] Trial 43 finished with value: -0.5208077653018799 and parameters: -
[I 2026-03-15 10:37:22,683] Trial 44 finished with value: -0.5324340275327437 and parameters: -
[I 2026-03-15 10:37:22,687] Trial 45 finished with value: -0.7503500617652714 and parameters: -
[I 2026-03-15 10:37:22,691] Trial 46 finished with value: -0.4219818938019863 and parameters: -
[I 2026-03-15 10:37:22,697] Trial 47 finished with value: -0.6944507559383358 and parameters: -
[I 2026-03-15 10:37:22,702] Trial 48 finished with value: 1.836763407651702 and parameters: {'x
[I 2026-03-15 10:37:22,706] Trial 49 finished with value: -0.7836586180709121 and parameters: -
[I 2026-03-15 10:37:22,711] Trial 50 finished with value: 2.634279048457641 and parameters: {'x
[I 2026-03-15 10:37:22,716] Trial 51 finished with value: -0.8065621681794006 and parameters: -
[I 2026-03-15 10:37:22,721] Trial 52 finished with value: -0.7672669254208946 and parameters: -
[I 2026-03-15 10:37:22,727] Trial 53 finished with value: 2.044195169111055 and parameters: {'x
[I 2026-03-15 10:37:22,732] Trial 54 finished with value: 0.9778499254023043 and parameters: {
[I 2026-03-15 10:37:22,737] Trial 55 finished with value: 0.3225197163511173 and parameters: {
[I 2026-03-15 10:37:22,741] Trial 56 finished with value: 1.7356636788428748 and parameters: {
[I 2026-03-15 10:37:22,745] Trial 57 finished with value: 1.946360161288155 and parameters: {'x
[I 2026-03-15 10:37:22,750] Trial 58 finished with value: -0.695502250509318 and parameters: {
[I 2026-03-15 10:37:22,754] Trial 59 finished with value: 1.0037616496964066 and parameters: {
[I 2026-03-15 10:37:22,758] Trial 60 finished with value: -0.5220831039564996 and parameters: -
[I 2026-03-15 10:37:22,764] Trial 61 finished with value: -0.7478655030958397 and parameters: -
[I 2026-03-15 10:37:22,768] Trial 62 finished with value: 2.2534194791694464 and parameters: {
[I 2026-03-15 10:37:22,771] Trial 63 finished with value: 0.6835597289750955 and parameters: {
[I 2026-03-15 10:37:22,776] Trial 64 finished with value: -0.6551661184272892 and parameters: -
[I 2026-03-15 10:37:22,781] Trial 65 finished with value: 63.94517772422507 and parameters: {'x
[I 2026-03-15 10:37:22,785] Trial 66 finished with value: 0.6203691609226595 and parameters: {
[I 2026-03-15 10:37:22,789] Trial 67 finished with value: 47.78119391680605 and parameters: {'x
[I 2026-03-15 10:37:22,794] Trial 68 finished with value: 1.7292602488957705 and parameters: {
[I 2026-03-15 10:37:22,798] Trial 69 finished with value: 1.5078740804758932 and parameters: {
[I 2026-03-15 10:37:22,802] Trial 70 finished with value: 3.1255898050954785 and parameters: {
[I 2026-03-15 10:37:22,806] Trial 71 finished with value: -0.7616062594897557 and parameters: -
[I 2026-03-15 10:37:22,812] Trial 72 finished with value: -0.7279974110455878 and parameters: -
[I 2026-03-15 10:37:22,817] Trial 73 finished with value: 0.3468773212646244 and parameters: {
[I 2026-03-15 10:37:22,821] Trial 74 finished with value: 2.8697897105936545 and parameters: {
[I 2026-03-15 10:37:22,826] Trial 75 finished with value: 0.5226533478500621 and parameters: {
[I 2026-03-15 10:37:22,830] Trial 76 finished with value: 2.138104387208891 and parameters: {'x
[I 2026-03-15 10:37:22,835] Trial 77 finished with value: -0.7073060634619477 and parameters: -
[I 2026-03-15 10:37:22,840] Trial 78 finished with value: 1.3101814733423474 and parameters: {
[I 2026-03-15 10:37:22,847] Trial 79 finished with value: 1.029932570121842 and parameters: {'x
[I 2026-03-15 10:37:22,852] Trial 80 finished with value: 1.926389438802969 and parameters: {'x
[I 2026-03-15 10:37:22,857] Trial 81 finished with value: -0.730426156026163 and parameters: {
[I 2026-03-15 10:37:22,863] Trial 82 finished with value: -0.10063656697402189 and parameters:
[I 2026-03-15 10:37:22,867] Trial 83 finished with value: -0.8001964598814042 and parameters: -
[I 2026-03-15 10:37:22,872] Trial 84 finished with value: 0.8096806231693198 and parameters: {
[I 2026-03-15 10:37:22,878] Trial 85 finished with value: -0.0664450172835398 and parameters: -
```

```
[I 2026-03-15 10:37:22,883] Trial 86 finished with value: 2.2795476455850636 and parameters: {
[I 2026-03-15 10:37:22,887] Trial 87 finished with value: -0.7579147506550393 and parameters: -
[I 2026-03-15 10:37:22,892] Trial 88 finished with value: 0.9117322221704324 and parameters: {
[I 2026-03-15 10:37:22,897] Trial 89 finished with value: -0.10889452697055765 and parameters:
[I 2026-03-15 10:37:22,901] Trial 90 finished with value: 1.900937375447603 and parameters: {'x
[I 2026-03-15 10:37:22,905] Trial 91 finished with value: 0.5281698617019164 and parameters: {
[I 2026-03-15 10:37:22,909] Trial 92 finished with value: -0.8076720013789591 and parameters: -
[I 2026-03-15 10:37:22,913] Trial 93 finished with value: -0.7667161583698134 and parameters: -
[I 2026-03-15 10:37:22,917] Trial 94 finished with value: 0.6839585885080921 and parameters: {
[I 2026-03-15 10:37:22,921] Trial 95 finished with value: -0.6691488501864966 and parameters: -
[I 2026-03-15 10:37:22,926] Trial 96 finished with value: 2.071316224471209 and parameters: {'x
[I 2026-03-15 10:37:22,931] Trial 97 finished with value: 1.0200394066406089 and parameters: {
[I 2026-03-15 10:37:22,935] Trial 98 finished with value: 1.8599040900532775 and parameters: {
[I 2026-03-15 10:37:22,941] Trial 99 finished with value: 31.320866524192933 and parameters: {
[I 2026-03-15 10:37:22,945] Trial 100 finished with value: -0.6628459913327949 and parameters:
[I 2026-03-15 10:37:22,949] Trial 101 finished with value: -0.10538535272295163 and parameters
[I 2026-03-15 10:37:22,953] Trial 102 finished with value: -0.7954257628159461 and parameters:
[I 2026-03-15 10:37:22,959] Trial 103 finished with value: 0.4832318568776777 and parameters: -
[I 2026-03-15 10:37:22,962] Trial 104 finished with value: 11.499160161322571 and parameters: -
[I 2026-03-15 10:37:22,966] Trial 105 finished with value: 0.1585692024467916 and parameters: -
[I 2026-03-15 10:37:22,970] Trial 106 finished with value: 1.5177541908443135 and parameters: -
[I 2026-03-15 10:37:22,975] Trial 107 finished with value: -0.7276931110963268 and parameters:
[I 2026-03-15 10:37:22,978] Trial 108 finished with value: 2.27689640395353 and parameters: {'x
[I 2026-03-15 10:37:22,982] Trial 109 finished with value: -0.2751391530520074 and parameters:
[I 2026-03-15 10:37:22,986] Trial 110 finished with value: 0.9250113671999067 and parameters: -
[I 2026-03-15 10:37:22,991] Trial 111 finished with value: -0.18555460817910213 and parameters
[I 2026-03-15 10:37:22,995] Trial 112 finished with value: -0.8035056646075942 and parameters:
[I 2026-03-15 10:37:22,999] Trial 113 finished with value: -0.7690421901284703 and parameters:
[I 2026-03-15 10:37:23,004] Trial 114 finished with value: 0.9141370381506657 and parameters: -
[I 2026-03-15 10:37:23,009] Trial 115 finished with value: -0.10736462798702259 and parameters
[I 2026-03-15 10:37:23,013] Trial 116 finished with value: -0.7714108596427234 and parameters:
[I 2026-03-15 10:37:23,018] Trial 117 finished with value: 0.9247568334576983 and parameters: -
[I 2026-03-15 10:37:23,025] Trial 118 finished with value: 0.34302492981488925 and parameters:
[I 2026-03-15 10:37:23,030] Trial 119 finished with value: 0.0034986397677398973 and parameters

Best value: -0.80767
Best params: {'x': 3.4201486416402855}
```

Nothing about this workflow is machine-learning specific. The objective is just a callable black box, and Optuna treats it the same way it would treat a validation loss.

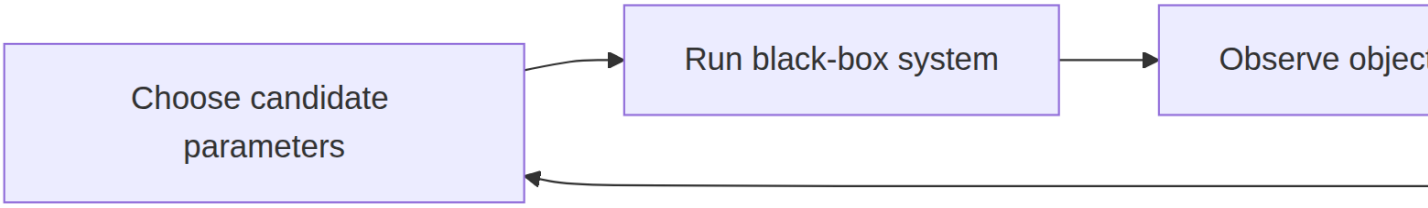
13.3. What Makes a Problem a Good Fit for Optuna?

The common pattern is not "has hyperparameters." The common pattern is:

- you can evaluate a candidate configuration programmatically

- evaluations are expensive enough that sample efficiency matters
- gradients are unavailable, unreliable, or irrelevant
- the search space may mix continuous, discrete, and categorical choices

In other words, Optuna is useful whenever you have to search intelligently rather than differentiate analytically.



This pattern appears in surprisingly different domains, which is why general optimization is such an important part of understanding Optuna well.

13.4. How Does Mathematical Optimization Differ From ML Tuning?

Mathematical benchmark functions strip away dataset complexity and let you inspect the optimizer itself.

For example, if the objective is deterministic and cheap, then poor performance is more likely to come from the search strategy or search-space design than from noisy evaluation. That makes benchmark functions useful for intuition, debugging, and comparing samplers before moving into more realistic workloads.

At the same time, benchmark functions can be misleading if you treat them as the whole story. Real objectives often include stochastic noise, failed runs, discontinuities, or simulation costs that simple benchmark surfaces do not capture.

13.5. How Should You Think About Simulation Calibration?

Many scientific and engineering workflows use a simulator to generate outputs that should match observed data or desired physical behavior.

The general pattern looks like this:

$$\min_{x \in \mathcal{X}} \|\text{simulation}(x) - \text{data}\|$$

where x is the parameter vector you want to calibrate.

Typical examples include:

- epidemiological models fitted to observed case counts
- climate or energy simulations calibrated to measurements
- financial Monte Carlo models tuned to market statistics

- robotics simulators matched to real system behavior

```
def objective(trial):
    alpha = trial.suggest_float("alpha", 0.0, 2.0)
    beta = trial.suggest_float("beta", 0.0, 2.0)

    prediction = simulate(alpha, beta)
    error = mean_squared_error(prediction, observed)
    return error
```

The important point is not the specific domain. It is that Optuna only needs the final scalar objective. The simulator can be arbitrarily complex internally.

13.6. Why Are Physics and Engineering Objectives Good Examples?

Physics and engineering problems often contain exactly the features that make black-box optimization hard:

- expensive evaluations
- non-differentiable or piecewise behavior
- noisy measurements
- constraints tied to safety or feasibility

That makes them a strong match for the ideas from Advanced Optimization Techniques. Constraints, multi-objective trade-offs, and conditional spaces often appear naturally here.

```
def objective(trial):
    friction = trial.suggest_float("friction", 0.0, 1.0)
    mass = trial.suggest_float("mass", 0.5, 10.0)

    result = run_physics_simulation(friction, mass)
    return result["energy_loss"]
```

If you can run the simulation and measure what matters, you can usually wrap it in an Optuna objective.

13.7. What About Algorithm Configuration?

Algorithms themselves are often optimization targets. A search heuristic, numerical solver, planner, or controller may expose parameters whose best setting depends heavily on the workload.

Examples include:

System	Tunable parameters
Genetic algorithm	mutation rate, crossover rate, population size
Monte Carlo tree search	exploration constant, rollout budget
Numerical solver	tolerance, step size, iteration cap
Heuristic planner	beam width, search depth, penalty weights

```
def objective(trial):
    step_size = trial.suggest_float("step_size", 1e-4, 1e-1, log=True)
    tolerance = trial.suggest_float("tol", 1e-6, 1e-2, log=True)
    return run_solver(step_size, tolerance)
```

This is often where Optuna becomes especially useful in research code, because algorithm configuration tends to combine expensive evaluation with poorly understood parameter interactions.

13.8. Why Is Reinforcement Learning a Special Case?

Reinforcement learning is a natural use case for Optuna, but it is also one of the messiest.

RL objectives are typically:

- noisy across runs
- expensive to evaluate
- sensitive to seeds and environment details
- sometimes non-stationary during training

That makes Optuna useful, but it also makes the objective design more fragile.

```
def objective(trial):
    lr = trial.suggest_float("lr", 1e-5, 1e-3, log=True)
    gamma = trial.suggest_float("gamma", 0.9, 0.999)

    reward = train_rl_agent(lr, gamma)
    return -reward
```

The optimization logic is still straightforward, but the engineering judgment is harder: how many episodes make a fair evaluation, how much variance is acceptable, and whether pruning will save time without biasing results too aggressively.

13.9. How Do You Handle Expensive or Noisy Objectives?

General optimization problems are often slow enough that each objective evaluation matters.

Practical strategies include:

- using pruning when intermediate metrics are meaningful
- parallelizing trials when evaluations are independent
- narrowing the search space with domain knowledge
- relying on adaptive samplers such as TPE to reduce wasted evaluations

This is where earlier modules connect directly. Samplers, Pruning, and Distributed and Scalable Optimization are not ML-only topics; they are core tools for general optimization too.

13.10. Why Do Benchmark Functions Still Matter?

Benchmark functions are not toys in the trivial sense. They are controlled stress tests for optimization algorithms.

Three classic examples are:

Function	Typical difficulty
Rastrigin	many local minima and strong periodic structure
Ackley	broad flat region with a narrow central basin
Rosenbrock	curved valley with deceptively hard geometry

These functions are useful because they make different optimizer weaknesses visible.

- A method may explore well but refine poorly.
- Another may perform well in smooth valleys but struggle with multimodality.
- A third may work in low dimensions and degrade quickly as dimensionality grows.

So benchmark functions are best treated as diagnostic surfaces, not as substitutes for real applications.

13.11. A Closer Look: The Rastrigin Function

Rastrigin is a good example because it is simple to write down and still difficult to optimize globally.

For an n-dimensional vector x, it is defined as:

$$f(x) = 10n + \sum_{i=1}^{n} \left(x_i^2 - 10\cos(2\pi x_i)\right)$$

Its global minimum is at the origin, where $f(x) = 0$, but the cosine term creates many local minima across the domain. This makes it a good test of whether the optimizer can keep exploring rather than collapsing too early into one local basin.

13.12. Practical Exercise: Optimize a Benchmark Function

Use Optuna to optimize a two-dimensional Rastrigin function.

```
import math

import optuna


def rastrigin(values):
    amplitude = 10.0
    dimension = len(values)
    return amplitude * dimension + sum(
        value**2 - amplitude * math.cos(2.0 * math.pi * value)
        for value in values
    )


def objective(trial):
    x1 = trial.suggest_float("x1", -5.12, 5.12)
    x2 = trial.suggest_float("x2", -5.12, 5.12)
    return rastrigin([x1, x2])


study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=300)

print("Best parameters:", study.best_params)
print("Best value:", round(study.best_value, 6))
```

```
[I 2026-03-15 10:37:23,049] A new study created in memory with name: no-name-7635b93f-98dc-4d4
[I 2026-03-15 10:37:23,053] Trial 0 finished with value: 12.478004052980708 and parameters: {'
[I 2026-03-15 10:37:23,055] Trial 1 finished with value: 29.05503586077483 and parameters: {'x
[I 2026-03-15 10:37:23,057] Trial 2 finished with value: 43.99702430920541 and parameters: {'x
[I 2026-03-15 10:37:23,059] Trial 3 finished with value: 46.01701101650309 and parameters: {'x
[I 2026-03-15 10:37:23,061] Trial 4 finished with value: 36.00183258274498 and parameters: {'x
[I 2026-03-15 10:37:23,063] Trial 5 finished with value: 26.52229674516526 and parameters: {'x
[I 2026-03-15 10:37:23,065] Trial 6 finished with value: 24.679851804856163 and parameters: {'
[I 2026-03-15 10:37:23,066] Trial 7 finished with value: 60.46971518319381 and parameters: {'x
[I 2026-03-15 10:37:23,068] Trial 8 finished with value: 52.86367585858577 and parameters: {'x
[I 2026-03-15 10:37:23,071] Trial 9 finished with value: 31.521798433289124 and parameters: {'
[I 2026-03-15 10:37:23,077] Trial 10 finished with value: 40.82860900263853 and parameters: {'
[I 2026-03-15 10:37:23,085] Trial 11 finished with value: 37.15658747952989 and parameters: {'
[I 2026-03-15 10:37:23,092] Trial 12 finished with value: 23.05261992595845 and parameters: {'
[I 2026-03-15 10:37:23,098] Trial 13 finished with value: 37.815927234835904 and parameters: {
[I 2026-03-15 10:37:23,105] Trial 14 finished with value: 35.42098481464526 and parameters: {'
[I 2026-03-15 10:37:23,111] Trial 15 finished with value: 13.565485684114915 and parameters: {
```

```
[I 2026-03-15 10:37:23,117] Trial 16 finished with value: 6.590417572832067 and parameters: {'x
[I 2026-03-15 10:37:23,123] Trial 17 finished with value: 21.69361256293088 and parameters: {'x
[I 2026-03-15 10:37:23,128] Trial 18 finished with value: 45.95055605528322 and parameters: {'x
[I 2026-03-15 10:37:23,133] Trial 19 finished with value: 25.910980579899388 and parameters: {
[I 2026-03-15 10:37:23,148] Trial 20 finished with value: 17.063154600220223 and parameters: {
[I 2026-03-15 10:37:23,154] Trial 21 finished with value: 18.50340592573014 and parameters: {'x
[I 2026-03-15 10:37:23,159] Trial 22 finished with value: 6.720631397517906 and parameters: {'x
[I 2026-03-15 10:37:23,164] Trial 23 finished with value: 29.521216092651663 and parameters: {
[I 2026-03-15 10:37:23,171] Trial 24 finished with value: 22.629665258296377 and parameters: {
[I 2026-03-15 10:37:23,176] Trial 25 finished with value: 28.110566825510695 and parameters: {
[I 2026-03-15 10:37:23,182] Trial 26 finished with value: 33.92640265071594 and parameters: {'x
[I 2026-03-15 10:37:23,189] Trial 27 finished with value: 27.537422215066673 and parameters: {
[I 2026-03-15 10:37:23,195] Trial 28 finished with value: 44.66013549145473 and parameters: {'x
[I 2026-03-15 10:37:23,205] Trial 29 finished with value: 39.16878135253777 and parameters: {'x
[I 2026-03-15 10:37:23,211] Trial 30 finished with value: 16.472824776592514 and parameters: {
[I 2026-03-15 10:37:23,219] Trial 31 finished with value: 16.858421719318457 and parameters: {
[I 2026-03-15 10:37:23,225] Trial 32 finished with value: 12.464178687469177 and parameters: {
[I 2026-03-15 10:37:23,231] Trial 33 finished with value: 44.16142110355045 and parameters: {'x
[I 2026-03-15 10:37:23,240] Trial 34 finished with value: 31.02701753015308 and parameters: {'x
[I 2026-03-15 10:37:23,250] Trial 35 finished with value: 52.347076503587886 and parameters: {
[I 2026-03-15 10:37:23,258] Trial 36 finished with value: 24.96486984448014 and parameters: {'x
[I 2026-03-15 10:37:23,267] Trial 37 finished with value: 28.390571217454394 and parameters: {
[I 2026-03-15 10:37:23,273] Trial 38 finished with value: 25.34078245432847 and parameters: {'x
[I 2026-03-15 10:37:23,280] Trial 39 finished with value: 22.92311464828814 and parameters: {'x
[I 2026-03-15 10:37:23,288] Trial 40 finished with value: 20.579255232731096 and parameters: {
[I 2026-03-15 10:37:23,295] Trial 41 finished with value: 15.344828970358222 and parameters: {
[I 2026-03-15 10:37:23,303] Trial 42 finished with value: 22.3131864601339 and parameters: {'x
[I 2026-03-15 10:37:23,308] Trial 43 finished with value: 40.96778696288736 and parameters: {'x
[I 2026-03-15 10:37:23,316] Trial 44 finished with value: 25.72529816919772 and parameters: {'x
[I 2026-03-15 10:37:23,322] Trial 45 finished with value: 3.3152440305075217 and parameters: {
[I 2026-03-15 10:37:23,330] Trial 46 finished with value: 16.37014983964285 and parameters: {'x
[I 2026-03-15 10:37:23,339] Trial 47 finished with value: 34.43055833175892 and parameters: {'x
[I 2026-03-15 10:37:23,348] Trial 48 finished with value: 20.229919845459275 and parameters: {
[I 2026-03-15 10:37:23,358] Trial 49 finished with value: 19.123236188331113 and parameters: {
[I 2026-03-15 10:37:23,366] Trial 50 finished with value: 39.396965439651645 and parameters: {
[I 2026-03-15 10:37:23,373] Trial 51 finished with value: 27.430041999814247 and parameters: {
[I 2026-03-15 10:37:23,382] Trial 52 finished with value: 27.224161889970635 and parameters: {
[I 2026-03-15 10:37:23,389] Trial 53 finished with value: 12.183497264565073 and parameters: {
[I 2026-03-15 10:37:23,398] Trial 54 finished with value: 10.11042011699029 and parameters: {'x
[I 2026-03-15 10:37:23,405] Trial 55 finished with value: 5.432523733451161 and parameters: {'x
[I 2026-03-15 10:37:23,414] Trial 56 finished with value: 21.940535101413804 and parameters: {
[I 2026-03-15 10:37:23,421] Trial 57 finished with value: 34.28799742572418 and parameters: {'x
[I 2026-03-15 10:37:23,428] Trial 58 finished with value: 25.39679519485867 and parameters: {'x
[I 2026-03-15 10:37:23,436] Trial 59 finished with value: 23.475765670224103 and parameters: {
[I 2026-03-15 10:37:23,444] Trial 60 finished with value: 13.725379746120005 and parameters: {
[I 2026-03-15 10:37:23,452] Trial 61 finished with value: 28.467200975835517 and parameters: {
[I 2026-03-15 10:37:23,458] Trial 62 finished with value: 3.0779730432882637 and parameters: {
[I 2026-03-15 10:37:23,467] Trial 63 finished with value: 20.371874360616246 and parameters: {
```

```
[I 2026-03-15 10:37:23,474] Trial 64 finished with value: 25.19145049704894 and parameters: {'x
[I 2026-03-15 10:37:23,483] Trial 65 finished with value: 24.985136858089334 and parameters: {
[I 2026-03-15 10:37:23,489] Trial 66 finished with value: 25.97961684984248 and parameters: {'x
[I 2026-03-15 10:37:23,497] Trial 67 finished with value: 32.82793807195615 and parameters: {'x
[I 2026-03-15 10:37:23,503] Trial 68 finished with value: 20.082599589942298 and parameters: {
[I 2026-03-15 10:37:23,511] Trial 69 finished with value: 30.55132066890542 and parameters: {'x
[I 2026-03-15 10:37:23,519] Trial 70 finished with value: 24.12592950940123 and parameters: {'x
[I 2026-03-15 10:37:23,526] Trial 71 finished with value: 1.3149112999434323 and parameters: {
[I 2026-03-15 10:37:23,533] Trial 72 finished with value: 17.988694345081683 and parameters: {
[I 2026-03-15 10:37:23,541] Trial 73 finished with value: 22.04778762147939 and parameters: {'x
[I 2026-03-15 10:37:23,549] Trial 74 finished with value: 15.744945286906217 and parameters: {
[I 2026-03-15 10:37:23,556] Trial 75 finished with value: 4.830646332417713 and parameters: {'x
[I 2026-03-15 10:37:23,564] Trial 76 finished with value: 15.740046152641092 and parameters: {
[I 2026-03-15 10:37:23,571] Trial 77 finished with value: 6.0000699662955554 and parameters: {
[I 2026-03-15 10:37:23,579] Trial 78 finished with value: 2.290273547177044 and parameters: {'x
[I 2026-03-15 10:37:23,586] Trial 79 finished with value: 13.053815935709121 and parameters: {
[I 2026-03-15 10:37:23,595] Trial 80 finished with value: 25.65124650998716 and parameters: {'x
[I 2026-03-15 10:37:23,602] Trial 81 finished with value: 21.932061161094314 and parameters: {
[I 2026-03-15 10:37:23,611] Trial 82 finished with value: 1.5118387318483855 and parameters: {
[I 2026-03-15 10:37:23,623] Trial 83 finished with value: 20.37855867630003 and parameters: {'x
[I 2026-03-15 10:37:23,633] Trial 84 finished with value: 2.8213068844324276 and parameters: {
[I 2026-03-15 10:37:23,641] Trial 85 finished with value: 13.108464243618958 and parameters: {
[I 2026-03-15 10:37:23,651] Trial 86 finished with value: 21.346839700517037 and parameters: {
[I 2026-03-15 10:37:23,665] Trial 87 finished with value: 37.82696071285291 and parameters: {'x
[I 2026-03-15 10:37:23,676] Trial 88 finished with value: 30.511525037043427 and parameters: {
[I 2026-03-15 10:37:23,683] Trial 89 finished with value: 22.042624472559826 and parameters: {
[I 2026-03-15 10:37:23,694] Trial 90 finished with value: 17.616787059361865 and parameters: {
[I 2026-03-15 10:37:23,706] Trial 91 finished with value: 15.220910553215866 and parameters: {
[I 2026-03-15 10:37:23,715] Trial 92 finished with value: 12.699221579948706 and parameters: {
[I 2026-03-15 10:37:23,725] Trial 93 finished with value: 24.486380072495493 and parameters: {
[I 2026-03-15 10:37:23,731] Trial 94 finished with value: 29.05562736712671 and parameters: {'x
[I 2026-03-15 10:37:23,742] Trial 95 finished with value: 36.7656473829086 and parameters: {'xi
[I 2026-03-15 10:37:23,749] Trial 96 finished with value: 21.339578443238242 and parameters: {
[I 2026-03-15 10:37:23,757] Trial 97 finished with value: 22.716089312288346 and parameters: {
[I 2026-03-15 10:37:23,766] Trial 98 finished with value: 15.636228249844002 and parameters: {
[I 2026-03-15 10:37:23,775] Trial 99 finished with value: 4.768672068896773 and parameters: {'x
[I 2026-03-15 10:37:23,782] Trial 100 finished with value: 25.14135613205894 and parameters: {
[I 2026-03-15 10:37:23,794] Trial 101 finished with value: 22.962210843824593 and parameters: -
[I 2026-03-15 10:37:23,802] Trial 102 finished with value: 16.420868913803385 and parameters: -
[I 2026-03-15 10:37:23,815] Trial 103 finished with value: 16.69717951997557 and parameters: {
[I 2026-03-15 10:37:23,826] Trial 104 finished with value: 23.405659062075642 and parameters: -
[I 2026-03-15 10:37:23,834] Trial 105 finished with value: 38.12840370368652 and parameters: {
[I 2026-03-15 10:37:23,842] Trial 106 finished with value: 31.20159769827849 and parameters: {
[I 2026-03-15 10:37:23,849] Trial 107 finished with value: 18.127880399473845 and parameters: -
[I 2026-03-15 10:37:23,861] Trial 108 finished with value: 2.5299651356472737 and parameters: -
[I 2026-03-15 10:37:23,868] Trial 109 finished with value: 18.55897202936749 and parameters: {
[I 2026-03-15 10:37:23,881] Trial 110 finished with value: 3.6555618939702015 and parameters: -
[I 2026-03-15 10:37:23,905] Trial 111 finished with value: 3.7081395145818057 and parameters: -
```

```
[I 2026-03-15 10:37:23,922] Trial 112 finished with value: 5.2781866707441765 and parameters: {
[I 2026-03-15 10:37:23,935] Trial 113 finished with value: 5.5557410805392085 and parameters: {
[I 2026-03-15 10:37:23,944] Trial 114 finished with value: 13.257370506031418 and parameters: {
[I 2026-03-15 10:37:23,954] Trial 115 finished with value: 21.64190623625781 and parameters: {
[I 2026-03-15 10:37:23,961] Trial 116 finished with value: 29.44069546359219 and parameters: {
[I 2026-03-15 10:37:23,972] Trial 117 finished with value: 16.0778766002855 and parameters: {'
[I 2026-03-15 10:37:23,979] Trial 118 finished with value: 20.919185207101165 and parameters: {
[I 2026-03-15 10:37:23,990] Trial 119 finished with value: 47.008860035065794 and parameters: {
[I 2026-03-15 10:37:24,002] Trial 120 finished with value: 41.885906379686915 and parameters: {
[I 2026-03-15 10:37:24,011] Trial 121 finished with value: 50.15817049105439 and parameters: {
[I 2026-03-15 10:37:24,026] Trial 122 finished with value: 13.966287429625124 and parameters: {
[I 2026-03-15 10:37:24,039] Trial 123 finished with value: 19.08939364367284 and parameters: {
[I 2026-03-15 10:37:24,053] Trial 124 finished with value: 38.026898257801996 and parameters: {
[I 2026-03-15 10:37:24,067] Trial 125 finished with value: 27.985462393011296 and parameters: {
[I 2026-03-15 10:37:24,080] Trial 126 finished with value: 30.62204297433802 and parameters: {
[I 2026-03-15 10:37:24,096] Trial 127 finished with value: 28.657294672154016 and parameters: {
[I 2026-03-15 10:37:24,112] Trial 128 finished with value: 21.113144825649602 and parameters: {
[I 2026-03-15 10:37:24,130] Trial 129 finished with value: 22.776423326521815 and parameters: {
[I 2026-03-15 10:37:24,155] Trial 130 finished with value: 29.8680935628865 and parameters: {'
[I 2026-03-15 10:37:24,176] Trial 131 finished with value: 21.924468914393607 and parameters: {
[I 2026-03-15 10:37:24,194] Trial 132 finished with value: 5.188104230382464 and parameters: {
[I 2026-03-15 10:37:24,209] Trial 133 finished with value: 29.935565751944488 and parameters: {
[I 2026-03-15 10:37:24,225] Trial 134 finished with value: 24.419733823802346 and parameters: {
[I 2026-03-15 10:37:24,242] Trial 135 finished with value: 44.47334057396845 and parameters: {
[I 2026-03-15 10:37:24,261] Trial 136 finished with value: 23.266953782969882 and parameters: {
[I 2026-03-15 10:37:24,276] Trial 137 finished with value: 5.332113080735693 and parameters: {
[I 2026-03-15 10:37:24,290] Trial 138 finished with value: 4.834522191252793 and parameters: {
[I 2026-03-15 10:37:24,309] Trial 139 finished with value: 9.959456270266848 and parameters: {
[I 2026-03-15 10:37:24,326] Trial 140 finished with value: 14.317987200024564 and parameters: {
[I 2026-03-15 10:37:24,344] Trial 141 finished with value: 26.26513984485677 and parameters: {
[I 2026-03-15 10:37:24,360] Trial 142 finished with value: 12.893061137750458 and parameters: {
[I 2026-03-15 10:37:24,376] Trial 143 finished with value: 5.771785716094801 and parameters: {
[I 2026-03-15 10:37:24,391] Trial 144 finished with value: 32.455366883186635 and parameters: {
[I 2026-03-15 10:37:24,408] Trial 145 finished with value: 23.115930596681988 and parameters: {
[I 2026-03-15 10:37:24,423] Trial 146 finished with value: 6.583077354275957 and parameters: {
[I 2026-03-15 10:37:24,432] Trial 147 finished with value: 24.48427207715894 and parameters: {
[I 2026-03-15 10:37:24,443] Trial 148 finished with value: 38.532571548502716 and parameters: {
[I 2026-03-15 10:37:24,453] Trial 149 finished with value: 24.737532357019774 and parameters: {
[I 2026-03-15 10:37:24,467] Trial 150 finished with value: 29.38660036882923 and parameters: {
[I 2026-03-15 10:37:24,482] Trial 151 finished with value: 28.86663533687389 and parameters: {
[I 2026-03-15 10:37:24,494] Trial 152 finished with value: 4.133864602384495 and parameters: {
[I 2026-03-15 10:37:24,507] Trial 153 finished with value: 5.506367162060959 and parameters: {
[I 2026-03-15 10:37:24,517] Trial 154 finished with value: 19.87017022233539 and parameters: {
[I 2026-03-15 10:37:24,530] Trial 155 finished with value: 24.978423167740818 and parameters: {
[I 2026-03-15 10:37:24,543] Trial 156 finished with value: 19.44646178034263 and parameters: {
[I 2026-03-15 10:37:24,559] Trial 157 finished with value: 24.453652587976883 and parameters: {
[I 2026-03-15 10:37:24,575] Trial 158 finished with value: 26.713347872666592 and parameters: {
[I 2026-03-15 10:37:24,587] Trial 159 finished with value: 31.118081452355696 and parameters: {
```

```
[I 2026-03-15 10:37:24,601] Trial 160 finished with value: 1.8654821362865768 and parameters: {
[I 2026-03-15 10:37:24,619] Trial 161 finished with value: 18.19320882219341 and parameters: {
[I 2026-03-15 10:37:24,637] Trial 162 finished with value: 46.10317508888775 and parameters: {
[I 2026-03-15 10:37:24,660] Trial 163 finished with value: 6.2974352858037825 and parameters: {
[I 2026-03-15 10:37:24,677] Trial 164 finished with value: 52.9846069226044 and parameters: {'
[I 2026-03-15 10:37:24,692] Trial 165 finished with value: 15.689019363060789 and parameters: {
[I 2026-03-15 10:37:24,706] Trial 166 finished with value: 35.402565635119096 and parameters: {
[I 2026-03-15 10:37:24,723] Trial 167 finished with value: 16.01946874572524 and parameters: {
[I 2026-03-15 10:37:24,740] Trial 168 finished with value: 37.90345383274487 and parameters: {
[I 2026-03-15 10:37:24,759] Trial 169 finished with value: 16.971376443811764 and parameters: {
[I 2026-03-15 10:37:24,772] Trial 170 finished with value: 10.780573050677848 and parameters: {
[I 2026-03-15 10:37:24,785] Trial 171 finished with value: 4.877855967383116 and parameters: {
[I 2026-03-15 10:37:24,799] Trial 172 finished with value: 20.199050707420476 and parameters: {
[I 2026-03-15 10:37:24,811] Trial 173 finished with value: 8.584495030431002 and parameters: {
[I 2026-03-15 10:37:24,823] Trial 174 finished with value: 34.839739904316446 and parameters: {
[I 2026-03-15 10:37:24,836] Trial 175 finished with value: 15.22745700725185 and parameters: {
[I 2026-03-15 10:37:24,856] Trial 176 finished with value: 23.029449650693735 and parameters: {
[I 2026-03-15 10:37:24,875] Trial 177 finished with value: 27.134504967533303 and parameters: {
[I 2026-03-15 10:37:24,893] Trial 178 finished with value: 9.34388486405187 and parameters: {'
[I 2026-03-15 10:37:24,908] Trial 179 finished with value: 38.82290766052415 and parameters: {
[I 2026-03-15 10:37:24,921] Trial 180 finished with value: 35.55836389556126 and parameters: {
[I 2026-03-15 10:37:24,932] Trial 181 finished with value: 22.336388400707406 and parameters: {
[I 2026-03-15 10:37:24,946] Trial 182 finished with value: 9.027410333411492 and parameters: {
[I 2026-03-15 10:37:24,963] Trial 183 finished with value: 23.88536529439128 and parameters: {
[I 2026-03-15 10:37:24,985] Trial 184 finished with value: 17.903402535553976 and parameters: {
[I 2026-03-15 10:37:25,009] Trial 185 finished with value: 7.435352995631893 and parameters: {
[I 2026-03-15 10:37:25,031] Trial 186 finished with value: 10.896566683990104 and parameters: {
[I 2026-03-15 10:37:25,046] Trial 187 finished with value: 20.10247109677296 and parameters: {
[I 2026-03-15 10:37:25,061] Trial 188 finished with value: 18.12520069615989 and parameters: {
[I 2026-03-15 10:37:25,077] Trial 189 finished with value: 24.141972923788565 and parameters: {
[I 2026-03-15 10:37:25,090] Trial 190 finished with value: 28.02009036270629 and parameters: {
[I 2026-03-15 10:37:25,109] Trial 191 finished with value: 17.043897885213244 and parameters: {
[I 2026-03-15 10:37:25,128] Trial 192 finished with value: 2.8799139354410173 and parameters: {
[I 2026-03-15 10:37:25,149] Trial 193 finished with value: 21.98888887141793 and parameters: {
[I 2026-03-15 10:37:25,164] Trial 194 finished with value: 19.38192608484146 and parameters: {
[I 2026-03-15 10:37:25,191] Trial 195 finished with value: 34.13610240084447 and parameters: {
[I 2026-03-15 10:37:25,214] Trial 196 finished with value: 13.112703560774644 and parameters: {
[I 2026-03-15 10:37:25,227] Trial 197 finished with value: 19.75488155567938 and parameters: {
[I 2026-03-15 10:37:25,243] Trial 198 finished with value: 23.313002302230803 and parameters: {
[I 2026-03-15 10:37:25,254] Trial 199 finished with value: 32.87566535191651 and parameters: {
[I 2026-03-15 10:37:25,264] Trial 200 finished with value: 29.210733825764684 and parameters: {
[I 2026-03-15 10:37:25,276] Trial 201 finished with value: 4.273568879274389 and parameters: {
[I 2026-03-15 10:37:25,288] Trial 202 finished with value: 1.831169570601915 and parameters: {
[I 2026-03-15 10:37:25,301] Trial 203 finished with value: 2.720631194803971 and parameters: {
[I 2026-03-15 10:37:25,316] Trial 204 finished with value: 6.308475832521381 and parameters: {
[I 2026-03-15 10:37:25,328] Trial 205 finished with value: 13.399106274044843 and parameters: {
[I 2026-03-15 10:37:25,341] Trial 206 finished with value: 37.86922576251404 and parameters: {
[I 2026-03-15 10:37:25,358] Trial 207 finished with value: 28.283041816886602 and parameters: {
```

```
[I 2026-03-15 10:37:25,369] Trial 208 finished with value: 4.125644138914355 and parameters: {
[I 2026-03-15 10:37:25,383] Trial 209 finished with value: 3.7448885189703383 and parameters: {
[I 2026-03-15 10:37:25,398] Trial 210 finished with value: 3.199862492833674 and parameters: {
[I 2026-03-15 10:37:25,411] Trial 211 finished with value: 4.905092264056776 and parameters: {
[I 2026-03-15 10:37:25,424] Trial 212 finished with value: 13.720859754112382 and parameters: {
[I 2026-03-15 10:37:25,439] Trial 213 finished with value: 13.40533254437176 and parameters: {
[I 2026-03-15 10:37:25,456] Trial 214 finished with value: 22.896093301714416 and parameters: {
[I 2026-03-15 10:37:25,469] Trial 215 finished with value: 27.714449929343502 and parameters: {
[I 2026-03-15 10:37:25,489] Trial 216 finished with value: 2.0718283234811707 and parameters: {
[I 2026-03-15 10:37:25,515] Trial 217 finished with value: 1.4868873629361943 and parameters: {
[I 2026-03-15 10:37:25,539] Trial 218 finished with value: 8.519509367210523 and parameters: {
[I 2026-03-15 10:37:25,551] Trial 219 finished with value: 17.573322812049952 and parameters: {
[I 2026-03-15 10:37:25,567] Trial 220 finished with value: 23.423901129209522 and parameters: {
[I 2026-03-15 10:37:25,579] Trial 221 finished with value: 1.891461955483127 and parameters: {
[I 2026-03-15 10:37:25,609] Trial 222 finished with value: 1.199222891690006 and parameters: {
[I 2026-03-15 10:37:25,642] Trial 223 finished with value: 16.96932285087017 and parameters: {
[I 2026-03-15 10:37:25,687] Trial 224 finished with value: 16.927133913909756 and parameters: {
[I 2026-03-15 10:37:25,722] Trial 225 finished with value: 13.049666587575924 and parameters: {
[I 2026-03-15 10:37:25,767] Trial 226 finished with value: 13.943344864227953 and parameters: {
[I 2026-03-15 10:37:25,802] Trial 227 finished with value: 34.61119308567612 and parameters: {
[I 2026-03-15 10:37:25,818] Trial 228 finished with value: 22.59757852237984 and parameters: {
[I 2026-03-15 10:37:25,835] Trial 229 finished with value: 19.070852613710606 and parameters: {
[I 2026-03-15 10:37:25,863] Trial 230 finished with value: 2.5119064533509032 and parameters: {
[I 2026-03-15 10:37:25,889] Trial 231 finished with value: 7.48939535541075 and parameters: {'x
[I 2026-03-15 10:37:25,907] Trial 232 finished with value: 16.674939027759038 and parameters: {
[I 2026-03-15 10:37:25,927] Trial 233 finished with value: 2.2947683082769785 and parameters: {
[I 2026-03-15 10:37:25,947] Trial 234 finished with value: 5.777785736689491 and parameters: {
[I 2026-03-15 10:37:25,968] Trial 235 finished with value: 19.620314382134985 and parameters: {
[I 2026-03-15 10:37:25,997] Trial 236 finished with value: 18.91847781690379 and parameters: {
[I 2026-03-15 10:37:26,020] Trial 237 finished with value: 15.112858009809877 and parameters: {
[I 2026-03-15 10:37:26,042] Trial 238 finished with value: 2.043540777446548 and parameters: {
[I 2026-03-15 10:37:26,068] Trial 239 finished with value: 15.483569700457927 and parameters: {
[I 2026-03-15 10:37:26,082] Trial 240 finished with value: 20.250473195090606 and parameters: {
[I 2026-03-15 10:37:26,098] Trial 241 finished with value: 6.308513916527069 and parameters: {
[I 2026-03-15 10:37:26,114] Trial 242 finished with value: 14.205975091592048 and parameters: {
[I 2026-03-15 10:37:26,129] Trial 243 finished with value: 1.6246260078212842 and parameters: {
[I 2026-03-15 10:37:26,144] Trial 244 finished with value: 17.41220457002565 and parameters: {
[I 2026-03-15 10:37:26,158] Trial 245 finished with value: 14.572890249445754 and parameters: {
[I 2026-03-15 10:37:26,173] Trial 246 finished with value: 22.58737064899416 and parameters: {
[I 2026-03-15 10:37:26,186] Trial 247 finished with value: 36.706099033940035 and parameters: {
[I 2026-03-15 10:37:26,198] Trial 248 finished with value: 8.84885012925965 and parameters: {'x
[I 2026-03-15 10:37:26,210] Trial 249 finished with value: 9.21221222201416 and parameters: {'x
[I 2026-03-15 10:37:26,226] Trial 250 finished with value: 27.572899530244214 and parameters: {
[I 2026-03-15 10:37:26,242] Trial 251 finished with value: 23.15132052869439 and parameters: {
[I 2026-03-15 10:37:26,258] Trial 252 finished with value: 1.3083032088223092 and parameters: {
[I 2026-03-15 10:37:26,275] Trial 253 finished with value: 45.363859196599414 and parameters: {
[I 2026-03-15 10:37:26,296] Trial 254 finished with value: 4.1382248742682 and parameters: {'x
[I 2026-03-15 10:37:26,328] Trial 255 finished with value: 8.910222405138187 and parameters: {
```

```
[I 2026-03-15 10:37:26,353] Trial 256 finished with value: 25.953369815673458 and parameters: {
[I 2026-03-15 10:37:26,375] Trial 257 finished with value: 30.45342226210133 and parameters: {
[I 2026-03-15 10:37:26,395] Trial 258 finished with value: 35.67375851351205 and parameters: {
[I 2026-03-15 10:37:26,428] Trial 259 finished with value: 16.549429836977456 and parameters: {
[I 2026-03-15 10:37:26,449] Trial 260 finished with value: 14.985781916816567 and parameters: {
[I 2026-03-15 10:37:26,473] Trial 261 finished with value: 19.42663468188599 and parameters: {
[I 2026-03-15 10:37:26,497] Trial 262 finished with value: 8.993993646693818 and parameters: {
[I 2026-03-15 10:37:26,517] Trial 263 finished with value: 25.973984509110807 and parameters: {
[I 2026-03-15 10:37:26,530] Trial 264 finished with value: 35.51988229416287 and parameters: {
[I 2026-03-15 10:37:26,544] Trial 265 finished with value: 16.25243220037805 and parameters: {
[I 2026-03-15 10:37:26,559] Trial 266 finished with value: 21.161885418910447 and parameters: {
[I 2026-03-15 10:37:26,571] Trial 267 finished with value: 5.74117621537534 and parameters: {'x
[I 2026-03-15 10:37:26,583] Trial 268 finished with value: 22.546833858974974 and parameters: {
[I 2026-03-15 10:37:26,594] Trial 269 finished with value: 20.45941879038726 and parameters: {
[I 2026-03-15 10:37:26,605] Trial 270 finished with value: 13.446737366871389 and parameters: {
[I 2026-03-15 10:37:26,620] Trial 271 finished with value: 13.694232255995624 and parameters: {
[I 2026-03-15 10:37:26,638] Trial 272 finished with value: 3.664721978866858 and parameters: {
[I 2026-03-15 10:37:26,650] Trial 273 finished with value: 29.68815051477471 and parameters: {
[I 2026-03-15 10:37:26,664] Trial 274 finished with value: 39.42025730263994 and parameters: {
[I 2026-03-15 10:37:26,675] Trial 275 finished with value: 15.589632884056547 and parameters: {
[I 2026-03-15 10:37:26,691] Trial 276 finished with value: 27.640804297593505 and parameters: {
[I 2026-03-15 10:37:26,709] Trial 277 finished with value: 17.315740431541805 and parameters: {
[I 2026-03-15 10:37:26,724] Trial 278 finished with value: 26.519829139538707 and parameters: {
[I 2026-03-15 10:37:26,740] Trial 279 finished with value: 19.550810174806344 and parameters: {
[I 2026-03-15 10:37:26,761] Trial 280 finished with value: 38.362044562433695 and parameters: {
[I 2026-03-15 10:37:26,778] Trial 281 finished with value: 19.501034935608555 and parameters: {
[I 2026-03-15 10:37:26,792] Trial 282 finished with value: 26.040735151253998 and parameters: {
[I 2026-03-15 10:37:26,807] Trial 283 finished with value: 21.400855813806274 and parameters: {
[I 2026-03-15 10:37:26,822] Trial 284 finished with value: 5.3197417970241325 and parameters: {
[I 2026-03-15 10:37:26,834] Trial 285 finished with value: 29.187477526148843 and parameters: {
[I 2026-03-15 10:37:26,849] Trial 286 finished with value: 32.24128702097344 and parameters: {
[I 2026-03-15 10:37:26,862] Trial 287 finished with value: 9.004866457301192 and parameters: {
[I 2026-03-15 10:37:26,873] Trial 288 finished with value: 31.464586956091267 and parameters: {
[I 2026-03-15 10:37:26,884] Trial 289 finished with value: 20.053633966717033 and parameters: {
[I 2026-03-15 10:37:26,896] Trial 290 finished with value: 14.82407836041262 and parameters: {
[I 2026-03-15 10:37:26,907] Trial 291 finished with value: 14.704879587073057 and parameters: {
[I 2026-03-15 10:37:26,919] Trial 292 finished with value: 18.961105886247775 and parameters: {
[I 2026-03-15 10:37:26,930] Trial 293 finished with value: 8.28173765573526 and parameters: {'x
[I 2026-03-15 10:37:26,941] Trial 294 finished with value: 6.593260310758705 and parameters: {
[I 2026-03-15 10:37:26,952] Trial 295 finished with value: 33.09246410685645 and parameters: {
[I 2026-03-15 10:37:26,963] Trial 296 finished with value: 16.47019953899086 and parameters: {
[I 2026-03-15 10:37:26,975] Trial 297 finished with value: 15.11740669949117 and parameters: {
[I 2026-03-15 10:37:26,989] Trial 298 finished with value: 51.98625882121457 and parameters: {
[I 2026-03-15 10:37:27,001] Trial 299 finished with value: 55.353661520414484 and parameters: {


Best parameters: {'x1': -0.017633284479625186, 'x2': -0.07645305423528395}
Best value: 1.199223
```

Then work through the exercise in four passes:

1. Run the two-dimensional version and record how close Optuna gets to the origin.
2. Increase the number of trials and compare whether the best value continues to improve materially.
3. Add a third dimension and observe how the difficulty changes.
4. Compare TPE with random search and note which one appears more sample-efficient.

Questions to answer after the run:

- Did the optimizer converge close to the global minimum or settle in a local basin?
- How much harder did the problem become after adding one more dimension?
- Did more trials help because the sampler improved its beliefs, or simply because brute-force coverage increased?
- Which aspects of this benchmark feel unrealistic compared with a noisy simulator or RL environment?

> ⚠ Common Mistake
>
> Do not assume success on benchmark functions automatically translates to success on real-world objectives. Benchmarks are useful because they isolate difficulty, but practical systems add noise, failures, constraints, and evaluation costs that can dominate optimizer behavior.

13.13. Recap

Optuna is a general-purpose optimizer because its interface is built around black-box objective evaluation rather than around machine-learning APIs.

- Mathematical functions are useful for understanding optimizer behavior directly.
- Simulations, solvers, and engineering systems fit naturally when they can be evaluated programmatically.
- Reinforcement learning and stochastic environments require extra care because evaluation noise can dominate the search.
- Benchmark functions are best used as controlled stress tests, not as proof that a workflow is production-ready.

13.14. Decision Guide

Choose the objective design as carefully as you choose the sampler.

- Use Optuna for non-ML problems when the system can be evaluated as a black box and sample efficiency matters.
- Use benchmark functions when you want to study optimizer behavior in a controlled setting.
- Use domain knowledge aggressively to narrow search ranges for simulations and engineering systems.

- Treat noisy objectives, failed runs, and constraints as first-class design concerns rather than as edge cases.
- Reuse pruning, storage, and distributed execution patterns from earlier chapters whenever objective cost becomes significant.

The next chapter turns these ideas into operational practice. In Production Workflows, we will look at study persistence, experiment tracking, reproducibility, and the habits that make Optuna reliable in real systems rather than just interesting in notebooks.

14. Production Workflows

This chapter focuses on using Optuna reliably in production systems. By the end, you should be able to design studies that are tracked, reproducible, persistent, manageable at scale, and cleanly integrated into larger machine-learning or optimization pipelines.

14.1. Why This Matters

Running optimization in real environments introduces additional constraints that do not appear in small local experiments.

- experiments must be tracked
- results must be reproducible
- studies must be persistent
- experiments must be manageable at scale
- systems must integrate with broader training or deployment pipelines

That changes the question from "Can Optuna find a good configuration?" to "Can the workflow be trusted, resumed, inspected, and handed off safely?"

14.2. Fast First Win: Add Persistence and Logging

The quickest production improvement is to stop treating a study as disposable. The example below creates a temporary SQLite-backed study, adds a small callback for logging, and records basic metadata.

```
from pathlib import Path
from tempfile import TemporaryDirectory

import optuna


def objective(trial):
    x = trial.suggest_float("x", -10.0, 10.0)
    return (x - 2.0) ** 2


def log_callback(study, frozen_trial):
    print(
        f"trial={frozen_trial.number} "
```

```
        f"value={frozen_trial.value:.5f} "
        f"best={study.best_value:.5f}"
    )


with TemporaryDirectory() as tmpdir:
    db_path = Path(tmpdir) / "experiment.db"
    storage = f"sqlite:///{db_path}"

    sampler = optuna.samplers.TPESampler(seed=42)
    study = optuna.create_study(
        study_name="production-demo",
        storage=storage,
        load_if_exists=True,
        sampler=sampler,
        direction="minimize",
    )
    study.set_user_attr("dataset_version", "demo-v1")
    study.set_user_attr("pipeline_version", "baseline")

    study.optimize(objective, n_trials=10, callbacks=[log_callback])

    print("Best params:", study.best_params)
    print("Trials completed:", len(study.trials))
```

```
[I 2026-03-15 10:37:35,995] A new study created in RDB with name: production-demo
[I 2026-03-15 10:37:36,078] Trial 0 finished with value: 20.332863203744573 and parameters: {'x
[I 2026-03-15 10:37:36,129] Trial 1 finished with value: 49.2002098882354 and parameters: {'x'
[I 2026-03-15 10:37:36,185] Trial 2 finished with value: 6.968960269965035 and parameters: {'x
[I 2026-03-15 10:37:36,238] Trial 3 finished with value: 0.0007198658598402143 and parameters:
[I 2026-03-15 10:37:36,287] Trial 4 finished with value: 78.847779053833 and parameters: {'x':

trial=0 value=20.33286 best=20.33286
trial=1 value=49.20021 best=20.33286
trial=2 value=6.96896 best=6.96896
trial=3 value=0.00072 best=0.00072
trial=4 value=78.84778 best=0.00072

[I 2026-03-15 10:37:36,329] Trial 5 finished with value: 78.8563463885915 and parameters: {'x'
[I 2026-03-15 10:37:36,371] Trial 6 finished with value: 117.46934856026657 and parameters: {'x
[I 2026-03-15 10:37:36,422] Trial 7 finished with value: 28.339896231839813 and parameters: {'x
[I 2026-03-15 10:37:36,474] Trial 8 finished with value: 0.0004973004749973951 and parameters:
[I 2026-03-15 10:37:36,521] Trial 9 finished with value: 4.671872828592918 and parameters: {'x

trial=5 value=78.85635 best=0.00072
trial=6 value=117.46935 best=0.00072
```

```
trial=7 value=28.33990 best=0.00072
trial=8 value=0.00050 best=0.00050
trial=9 value=4.67187 best=0.00050
Best params: {'x': 2.0223002348641756}
Trials completed: 10
```

Even this small example already adds several production properties at once: persistence, visibility, metadata, and resumability.

14.3. Experiment Tracking

Hyperparameter optimization produces many trials, each with parameter configurations, evaluation metrics, intermediate values, and runtime information.

Tracking experiments is essential if you want to:

- compare studies
- reproduce results
- debug failures
- audit model development

Example trial record:

Trial	Params	Score	Time
1	`lr=0.01 depth=5`	0.82	20s
2	`lr=0.005 depth=8`	0.84	25s

Optuna automatically records core study information such as:

- parameters
- objective values
- timestamps
- trial states

These are stored in the study storage backend. That is one reason persistent storage matters even when distributed execution is not yet required.

14.4. Reproducibility

Reproducibility means more than being able to rerun the script once. It means having enough control over data, code, environment, and randomness that the result remains interpretable and repeatable.

14.4.1. Fix Random Seeds

```
sampler = optuna.samplers.TPESampler(seed=42)

study = optuna.create_study(
    sampler=sampler,
    direction="minimize",
)
```

It is equally important to fix seeds in the libraries used by the objective function.

```
import numpy as np
import torch

np.random.seed(42)
torch.manual_seed(42)
```

14.4.2. Version Control

Track at least:

- dataset versions
- model code
- training pipeline definitions

The practical pattern is simple:

```
git commit + experiment metadata
```

14.4.3. Environment Reproducibility

The environment is part of the experiment. Useful tools include:

- Docker
- conda environments
- `requirements.txt`

```
pip freeze > requirements.txt
```

Fixing the Optuna seed is necessary, but it is not enough by itself. Reproducibility is a system property, not a single argument.

14.5. Study Persistence

Persistent storage ensures that experiments are not lost when processes stop or machines fail.

An in-memory study is convenient for quick exploration:

```
study = optuna.create_study()
```

But production workflows should use persistent storage instead:

```
study = optuna.create_study(
    storage="sqlite:///experiment.db",
    study_name="rf-optimization",
    load_if_exists=True,
)
```

Benefits include:

- experiments survive crashes
- experiments can resume
- distributed workers can collaborate
- dashboards and monitoring tools can inspect the study live

14.6. Experiment Management

Once teams begin running multiple studies across several datasets and model families, organization becomes part of optimization quality.

Large experiments often require managing:

- multiple studies
- multiple datasets
- multiple models

Example directory structure:

```
experiments/
   dataset_A/
      study_rf.db
      study_xgb.db
   dataset_B/
      study_nn.db
```

Each study should represent one well-defined optimization problem rather than an ambiguous collection of trials.

14.6.1. Naming Studies

Use meaningful names.

```
study = optuna.create_study(
    study_name="xgboost-credit-risk",
    storage="sqlite:///credit-risk.db",
    load_if_exists=True,
)
```

Clear names reduce confusion when several studies coexist in the same storage system or dashboard.

14.7. Integration With ML Pipelines

In production ML systems, optimization becomes one stage inside a broader training pipeline.

Typical structure:

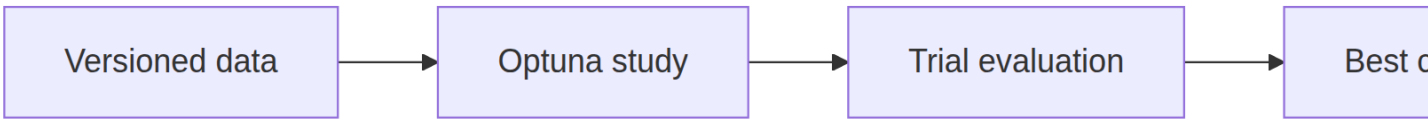
```
data ingestion
   -> feature engineering
   -> hyperparameter optimization
   -> model training
   -> evaluation
   -> model deployment
```

Optuna typically runs during the training and model-selection phase.

```
def train_pipeline():
    study = optuna.create_study(direction="maximize")
    study.optimize(objective, n_trials=200)

    best_params = study.best_params
    model = train_model(best_params)
    return model
```

In practice, the most important rule is to keep optimization and final training separate. The model fitted during search is usually not the final artifact you should deploy.



14.8. Logging Experiments

Logging is critical in production systems because failures, regressions, and unusual trial behavior rarely become obvious from final metrics alone.

Typical logs include:

- trial parameters
- training metrics
- runtime duration
- errors and failures

```
import logging

logging.info(f"Trial {trial.number}")
logging.info(f"Params {trial.params}")
logging.info(f"Score {score}")
```

External experiment trackers can also be integrated.

Tool	Use
MLflow	experiment tracking
Weights & Biases	experiment monitoring
TensorBoard	deep-learning visualization

Optuna handles search state well, but broader run lineage and artifact management may still belong in a dedicated tracking system.

14.9. Optuna Dashboard

Optuna provides an interactive dashboard for monitoring studies.

```
optuna-dashboard sqlite:///experiment.db
```

The dashboard is useful because it exposes:

- trial history
- parameter values
- optimization curves
- Pareto fronts
- trial filtering

That becomes especially valuable for long-running experiments or collaborative workflows where several people need visibility into the same study.

14.10. Monitoring Production Experiments

Production monitoring helps detect issues such as:

- stalled optimization
- failing trials
- unstable objective values
- exhausted compute resources

Minimal programmatic monitoring can be as simple as:

```
print("Trials completed:", len(study.trials))
print("Best value:", study.best_value)
```

More mature systems may add alerts when:

- trials fail repeatedly
- performance degrades sharply
- resource usage exceeds safe thresholds

14.11. Best Practices for Production Systems

The raw mechanics of Optuna are not enough. Real systems need operating rules.

14.11.1. Use Persistent Storage

Always store important studies in a database. Avoid in-memory studies for production-facing work.

14.11.2. Limit Trial Runtime

Use pruning or timeouts when objectives can run too long.

```
study.optimize(objective, timeout=3600)
```

14.11.3. Monitor Resource Usage

Optimization may consume:

- CPU
- GPU
- memory

Ensure trials do not overload shared infrastructure.

14.11.4. Separate Optimization and Training

Use the following mental model:

```
hyperparameter search
   -> best configuration
   -> final training on full dataset
```

Do not deploy a model trained only inside the optimization loop.

14.11.5. Store Best Model Configuration

Save the winning parameters explicitly.

```
best_params = study.best_params
```

Persist them for downstream use.

```
import json

with open("best_params.json", "w", encoding="utf-8") as file:
    json.dump(best_params, file)
```

14.12. Example Production Workflow

At a high level, a production optimization pipeline often looks like this:

```
dataset versioned
   -> Optuna study created
   -> distributed optimization
   -> best hyperparameters found
   -> final model training
   -> model stored in registry
   -> model deployed
```

This sequence creates:

- reproducibility
- traceability
- safer deployment handoff

> **Common Mistake**
>
> Do not treat `study.best_params` as the end of the workflow. Without persistent storage, metadata, logging, and a separate final-training step, the result may be difficult to reproduce and unsafe to operationalize.

14.13. Practical Exercise: Turn A Study Into A Production Artifact

Take a simple Optuna objective and upgrade it into a small but production-style workflow.

```
from pathlib import Path
from tempfile import TemporaryDirectory

import optuna


def objective(trial):
    x = trial.suggest_float("x", -10.0, 10.0)
    return (x - 2.0) ** 2
```

Work through the exercise in four passes:

1. Run the study in memory and record the best parameters.
2. Move it to SQLite storage with `study_name` and `load_if_exists=True`.
3. Add metadata for dataset version, code version, or experiment tag.
4. Write the post-optimization handoff: how the best configuration would be retrained, validated, logged, and persisted.

Questions to answer after the run:

- Which information is lost if the study remains in memory only?
- Which metadata fields are necessary to interpret the result later?
- Where should logging, dashboarding, or experiment tracking live in your stack?
- What must happen after selecting `study.best_params` before deployment is even considered?

14.14. Recap

Production optimization requires more than running many trials.

- Experiment tracking makes studies comparable and auditable.
- Reproducibility depends on seeds, versions, environments, and explicit metadata.
- Persistent storage is the foundation of resumability and collaboration.
- Experiment management keeps multiple studies understandable at scale.
- Logging, dashboards, and clear separation between search and final training make the workflow operationally credible.

14.15. Decision Guide

Treat production readiness as part of optimization quality.

- Use persistent storage as soon as a study matters beyond one local session.
- Name studies clearly and organize them so that the optimization problem is obvious.
- Record metadata during optimization rather than reconstructing it later.
- Separate hyperparameter search from final training and deployment.
- Add logging, dashboarding, and external experiment tracking when studies become long-running or collaborative.

The next chapter, Final Project, uses these production rules alongside the rest of the book to assemble complete end-to-end optimization workflows.

15. Final Project

This final chapter turns the book into a capstone portfolio. By the end, you should be able to build complete optimization workflows in three distinct settings: machine-learning pipelines, benchmark black-box functions, and stochastic simulation systems. Together, these projects are meant to prove not only that you can call Optuna effectively, but that you can design a search problem well, evaluate it honestly, and justify your optimization choices.

15.1. Why This Matters

The earlier chapters separated the key ideas: search-space design, samplers, pruning, diagnostics, distributed execution, ML evaluation, general black-box optimization, and production discipline.

The capstone is where those parts must work together. A strong final project does not merely show that Optuna found a decent parameter set. It shows that you can:

- define the objective correctly
- choose a search space that reflects the problem structure
- select or justify a sampler
- evaluate results honestly
- interpret optimization behavior
- communicate why the final configuration is credible

That is the difference between learning the API and mastering the workflow.

15.2. How to Use This Chapter

This module contains three project tracks:

- Project A: ML pipeline optimization
- Project B: complex function optimization and sampler comparison
- Project C: simulation optimization under uncertainty

You asked for all three, so the chapter is structured as a capstone portfolio rather than a single narrow assignment. Each project emphasizes a different skill:

Project	Primary lesson
A	rigorous ML optimization with leakage-safe evaluation
B	sampler behavior on difficult black-box landscapes
C	Optuna as a general-purpose optimizer for noisy simulations

If you complete all three well, you have covered the most important practical uses of Optuna in one portfolio.

15.3. Fast First Win: A Small Capstone Preview

Before committing to the larger projects, run one small end-to-end study on a benchmark function. This gives you a concrete baseline and a quick reminder of the basic study loop.

```
import math

import optuna


def rastrigin(values):
    amplitude = 10.0
    dimension = len(values)
    return amplitude * dimension + sum(
        value**2 - amplitude * math.cos(2.0 * math.pi * value)
        for value in values
    )


def objective(trial):
    x1 = trial.suggest_float("x1", -5.12, 5.12)
    x2 = trial.suggest_float("x2", -5.12, 5.12)
    return rastrigin([x1, x2])


study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=80)

print("Best value:", round(study.best_value, 6))
print("Best params:", study.best_params)
```

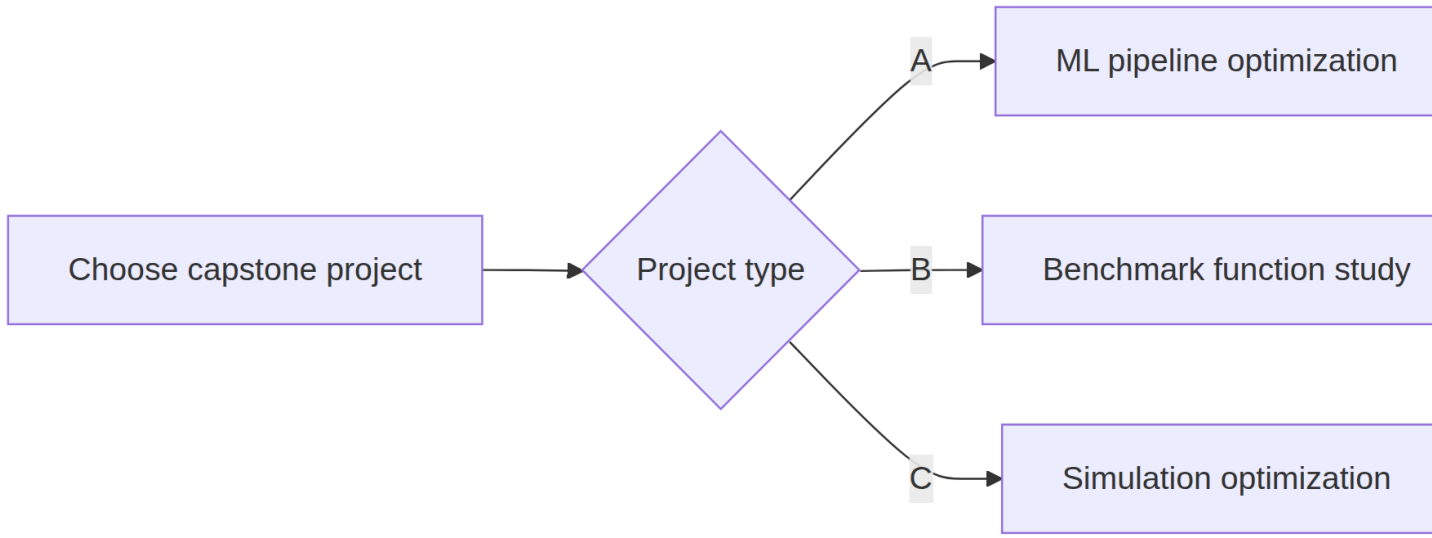
```
[I 2026-03-15 10:37:42,611] A new study created in memory with name: no-name-fa53f949-956f-4ed
[I 2026-03-15 10:37:42,621] Trial 0 finished with value: 62.19017009336859 and parameters: {'x
[I 2026-03-15 10:37:42,622] Trial 1 finished with value: 16.866694564014622 and parameters: {'
[I 2026-03-15 10:37:42,623] Trial 2 finished with value: 24.361772921777927 and parameters: {'
[I 2026-03-15 10:37:42,625] Trial 3 finished with value: 28.319649774866036 and parameters: {'
[I 2026-03-15 10:37:42,626] Trial 4 finished with value: 47.774297005206336 and parameters: {'
[I 2026-03-15 10:37:42,627] Trial 5 finished with value: 27.23980343317651 and parameters: {'x
[I 2026-03-15 10:37:42,628] Trial 6 finished with value: 23.733238919977403 and parameters: {'
[I 2026-03-15 10:37:42,629] Trial 7 finished with value: 35.20671599670328 and parameters: {'x
[I 2026-03-15 10:37:42,630] Trial 8 finished with value: 34.13847110964937 and parameters: {'x
[I 2026-03-15 10:37:42,632] Trial 9 finished with value: 20.55324473891625 and parameters: {'x
[I 2026-03-15 10:37:42,640] Trial 10 finished with value: 40.11555618663701 and parameters: {'
```

```
[I 2026-03-15 10:37:42,644] Trial 11 finished with value: 31.049441750125368 and parameters: {
[I 2026-03-15 10:37:42,650] Trial 12 finished with value: 29.13382775573151 and parameters: {'x
[I 2026-03-15 10:37:42,655] Trial 13 finished with value: 34.586263461374315 and parameters: {
[I 2026-03-15 10:37:42,659] Trial 14 finished with value: 28.860577629168883 and parameters: {
[I 2026-03-15 10:37:42,664] Trial 15 finished with value: 32.97192471161619 and parameters: {'x
[I 2026-03-15 10:37:42,668] Trial 16 finished with value: 50.586603387214375 and parameters: {
[I 2026-03-15 10:37:42,672] Trial 17 finished with value: 18.919532454542694 and parameters: {
[I 2026-03-15 10:37:42,676] Trial 18 finished with value: 42.662779649827414 and parameters: {
[I 2026-03-15 10:37:42,680] Trial 19 finished with value: 23.06727759597969 and parameters: {'x
[I 2026-03-15 10:37:42,684] Trial 20 finished with value: 51.65199925081086 and parameters: {'x
[I 2026-03-15 10:37:42,687] Trial 21 finished with value: 22.676800020867297 and parameters: {
[I 2026-03-15 10:37:42,692] Trial 22 finished with value: 19.39643718035711 and parameters: {'x
[I 2026-03-15 10:37:42,697] Trial 23 finished with value: 38.18169360541074 and parameters: {'x
[I 2026-03-15 10:37:42,702] Trial 24 finished with value: 27.08062739451967 and parameters: {'x
[I 2026-03-15 10:37:42,706] Trial 25 finished with value: 33.00895772226858 and parameters: {'x
[I 2026-03-15 10:37:42,710] Trial 26 finished with value: 31.946891901795862 and parameters: {
[I 2026-03-15 10:37:42,715] Trial 27 finished with value: 33.810448748351824 and parameters: {
[I 2026-03-15 10:37:42,719] Trial 28 finished with value: 20.446467861061898 and parameters: {
[I 2026-03-15 10:37:42,723] Trial 29 finished with value: 34.899216315220336 and parameters: {
[I 2026-03-15 10:37:42,729] Trial 30 finished with value: 39.818948856435206 and parameters: {
[I 2026-03-15 10:37:42,733] Trial 31 finished with value: 17.13634920704436 and parameters: {'x
[I 2026-03-15 10:37:42,737] Trial 32 finished with value: 14.914238668933088 and parameters: {
[I 2026-03-15 10:37:42,742] Trial 33 finished with value: 26.18809450003777 and parameters: {'x
[I 2026-03-15 10:37:42,747] Trial 34 finished with value: 18.840244183057266 and parameters: {
[I 2026-03-15 10:37:42,752] Trial 35 finished with value: 38.656150858155 and parameters: {'x1
[I 2026-03-15 10:37:42,756] Trial 36 finished with value: 29.70654825576908 and parameters: {'x
[I 2026-03-15 10:37:42,761] Trial 37 finished with value: 52.5493386834211 and parameters: {'x
[I 2026-03-15 10:37:42,765] Trial 38 finished with value: 30.367574508169653 and parameters: {
[I 2026-03-15 10:37:42,770] Trial 39 finished with value: 18.7726173033797 and parameters: {'x
[I 2026-03-15 10:37:42,774] Trial 40 finished with value: 24.246284886204197 and parameters: {
[I 2026-03-15 10:37:42,779] Trial 41 finished with value: 27.241859882904045 and parameters: {
[I 2026-03-15 10:37:42,784] Trial 42 finished with value: 29.79079412091786 and parameters: {'x
[I 2026-03-15 10:37:42,788] Trial 43 finished with value: 23.554417958564418 and parameters: {
[I 2026-03-15 10:37:42,794] Trial 44 finished with value: 47.41948761318258 and parameters: {'x
[I 2026-03-15 10:37:42,798] Trial 45 finished with value: 10.414302039296219 and parameters: {
[I 2026-03-15 10:37:42,803] Trial 46 finished with value: 10.147935288937393 and parameters: {
[I 2026-03-15 10:37:42,807] Trial 47 finished with value: 17.04383389315955 and parameters: {'x
[I 2026-03-15 10:37:42,812] Trial 48 finished with value: 30.47005259644181 and parameters: {'x
[I 2026-03-15 10:37:42,817] Trial 49 finished with value: 44.73561094492052 and parameters: {'x
[I 2026-03-15 10:37:42,821] Trial 50 finished with value: 6.867591878831295 and parameters: {'x
[I 2026-03-15 10:37:42,826] Trial 51 finished with value: 26.57777652793112 and parameters: {'x
[I 2026-03-15 10:37:42,830] Trial 52 finished with value: 16.980209678565128 and parameters: {
[I 2026-03-15 10:37:42,835] Trial 53 finished with value: 16.423481848973314 and parameters: {
[I 2026-03-15 10:37:42,840] Trial 54 finished with value: 10.949626618582062 and parameters: {
[I 2026-03-15 10:37:42,844] Trial 55 finished with value: 40.63084205300618 and parameters: {'x
[I 2026-03-15 10:37:42,849] Trial 56 finished with value: 30.818133466071036 and parameters: {
[I 2026-03-15 10:37:42,855] Trial 57 finished with value: 34.711843861199114 and parameters: {
[I 2026-03-15 10:37:42,860] Trial 58 finished with value: 7.95058339678274 and parameters: {'x
```

```
[I 2026-03-15 10:37:42,864] Trial 59 finished with value: 4.0137352872408805 and parameters: {
[I 2026-03-15 10:37:42,869] Trial 60 finished with value: 1.3006988769235441 and parameters: {
[I 2026-03-15 10:37:42,874] Trial 61 finished with value: 2.375742219993107 and parameters: {'x
[I 2026-03-15 10:37:42,879] Trial 62 finished with value: 2.565915409145255 and parameters: {'x
[I 2026-03-15 10:37:42,883] Trial 63 finished with value: 3.118656572841349 and parameters: {'x
[I 2026-03-15 10:37:42,888] Trial 64 finished with value: 2.73579743506847 and parameters: {'x1
[I 2026-03-15 10:37:42,892] Trial 65 finished with value: 9.408467491425657 and parameters: {'x
[I 2026-03-15 10:37:42,897] Trial 66 finished with value: 25.52268639266382 and parameters: {'x
[I 2026-03-15 10:37:42,901] Trial 67 finished with value: 27.487896425878258 and parameters: {
[I 2026-03-15 10:37:42,906] Trial 68 finished with value: 20.76191542487588 and parameters: {'x
[I 2026-03-15 10:37:42,911] Trial 69 finished with value: 20.357564378897354 and parameters: {
[I 2026-03-15 10:37:42,916] Trial 70 finished with value: 18.157797012888935 and parameters: {
[I 2026-03-15 10:37:42,922] Trial 71 finished with value: 4.291538327376328 and parameters: {'x
[I 2026-03-15 10:37:42,926] Trial 72 finished with value: 34.03704403189592 and parameters: {'x
[I 2026-03-15 10:37:42,931] Trial 73 finished with value: 25.2452170593264 and parameters: {'x1
[I 2026-03-15 10:37:42,935] Trial 74 finished with value: 23.226625166251292 and parameters: {
[I 2026-03-15 10:37:42,940] Trial 75 finished with value: 35.133683009626125 and parameters: {
[I 2026-03-15 10:37:42,945] Trial 76 finished with value: 20.201997999025448 and parameters: {
[I 2026-03-15 10:37:42,951] Trial 77 finished with value: 18.85074823023991 and parameters: {'x
[I 2026-03-15 10:37:42,956] Trial 78 finished with value: 23.91845334940226 and parameters: {'x
[I 2026-03-15 10:37:42,962] Trial 79 finished with value: 20.108636504716696 and parameters: {
```

```
Best value: 1.300699
Best params: {'x1': 1.0030981620030865, 'x2': -0.03849706719827871}
```

This is not the full capstone yet. It is just a compact reminder that every larger project still reduces to the same fundamental structure: define a black-box objective, search efficiently, and interpret the result critically.



15.4. Project A: ML Pipeline Optimization

Project A is the closest to a real applied ML workflow. Its purpose is not merely to optimize a classifier, but to show that you can build a leakage-safe, model-comparison-capable search procedure with a hierarchical search space.

15.4.1. Objective

Build an end-to-end supervised-learning optimization workflow that includes:

- preprocessing choices
- model-family selection
- model-specific hyperparameters
- robust cross-validation
- result analysis and interpretation

This project should look like something you would actually keep and extend in a production ML setting.

15.4.2. Problem Setup

Use a structured tabular dataset such as `breast_cancer` from scikit-learn, or an equivalent real dataset if you want a more domain-specific problem.

The pipeline structure is intentionally broad:

```
raw features
   -> optional scaling
   -> optional dimensionality reduction
   -> model family
   -> cross-validation score
```

That gives Optuna several kinds of choices to manage at once: preprocessing, model selection, and model-specific hyperparameters.

15.4.3. What the Search Space Should Express

Project A should force you to use a conditional, hierarchical search space. That is one of the most important skills in practical Optuna usage.

Recommended decision structure:

- scaler: `StandardScaler`, `MinMaxScaler`, or no scaler
- PCA usage: yes or no
- number of PCA components when PCA is enabled
- model family: logistic regression, random forest, or SVC

- model-specific hyperparameters depending on the selected branch

This means the search space is not flat. Different model families expose different parameter subspaces, which is exactly the kind of conditional structure Optuna handles well.

15.4.4. Full Project Code

```
import optuna
import numpy as np

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC

X, y = load_breast_cancer(return_X_y=True)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)


def objective(trial):
    steps = []

    scaler_name = trial.suggest_categorical(
        "scaler", ["none", "standard", "minmax"]
    )

    if scaler_name == "standard":
        steps.append(("scaler", StandardScaler()))
    elif scaler_name == "minmax":
        steps.append(("scaler", MinMaxScaler()))

    use_pca = trial.suggest_categorical("use_pca", [False, True])
    if use_pca:
        n_components = trial.suggest_int("pca_components", 5, min(X.shape[1],
 ↪  25))
        steps.append(("pca", PCA(n_components=n_components)))

    model_name = trial.suggest_categorical("model", ["logreg", "rf", "svc"])

    if model_name == "logreg":
        c = trial.suggest_float("logreg_C", 1e-3, 10.0, log=True)
        solver = trial.suggest_categorical("logreg_solver", ["lbfgs",
 ↪  "liblinear"])
```

```
        model = LogisticRegression(
            C=c,
            solver=solver,
            max_iter=2000,
            random_state=42,
        )

    elif model_name == "rf":
        n_estimators = trial.suggest_int("rf_n_estimators", 50, 300)
        max_depth = trial.suggest_int("rf_max_depth", 2, 20)
        min_samples_split = trial.suggest_int("rf_min_samples_split", 2, 10)
        model = RandomForestClassifier(
            n_estimators=n_estimators,
            max_depth=max_depth,
            min_samples_split=min_samples_split,
            random_state=42,
            n_jobs=-1,
        )

    else:
        c = trial.suggest_float("svc_C", 1e-3, 10.0, log=True)
        kernel = trial.suggest_categorical("svc_kernel", ["linear", "rbf"])
        if kernel == "rbf":
            gamma = trial.suggest_float("svc_gamma", 1e-4, 1.0, log=True)
            model = SVC(C=c, kernel=kernel, gamma=gamma, probability=False)
        else:
            model = SVC(C=c, kernel=kernel, probability=False)

    steps.append(("model", model))
    pipeline = Pipeline(steps)

    scores = cross_val_score(
        pipeline,
        X,
        y,
        cv=cv,
        scoring="accuracy",
        n_jobs=-1,
    )
    return scores.mean()


study = optuna.create_study(
    direction="maximize",
    sampler=optuna.samplers.TPESampler(seed=42),
)

study.optimize(objective, n_trials=100)
```

```
print("Best score:", study.best_value)
print("Best params:")
for key, value in study.best_params.items():
    print(f"  {key}: {value}")
```

15.4.5. What Project A Should Teach You

This project is successful if it teaches you how to:

- optimize whole pipelines rather than isolated models
- structure conditional search spaces cleanly
- avoid leakage by keeping preprocessing inside `Pipeline`
- compare multiple model families in one study
- reason about whether search-space breadth actually improved the workflow

15.4.6. Recommended Extensions

If you want to strengthen Project A, add one or more of the following:

- nested cross-validation for unbiased workflow assessment
- persistent study storage with SQLite or PostgreSQL
- Optuna visualizations for diagnostics
- a final retraining stage on a full training split
- parameter-importance and optimization-history analysis

15.4.7. Deliverables for Project A

At the end of Project A, you should have:

- the best pipeline configuration
- the best cross-validation score
- the top trials or a short leaderboard
- at least one diagnostic plot
- a short explanation of why the winning pipeline is credible

15.5. Project B: Complex Function Optimization

Project B is the most controlled of the three tracks. Its main purpose is to help you understand optimizer behavior rather than application-specific evaluation protocol.

15.5.1. Objective

Optimize one or more difficult non-convex benchmark functions and compare multiple samplers under a fair evaluation procedure.

Recommended samplers:

- random search
- TPE
- CMA-ES

This project is where you should become more precise about sample efficiency, robustness across seeds, and the relationship between search-space geometry and sampler behavior.

15.5.2. Benchmark Function Choices

Use at least one of the following:

- Rastrigin
- Ackley
- Rosenbrock

If you want stronger analysis, run more than one function and compare how the samplers behave across different landscape types.

15.5.3. Benchmark Definitions

```
import numpy as np


def rastrigin(x):
    x = np.asarray(x)
    amplitude = 10
    return amplitude * len(x) + np.sum(x**2 - amplitude * np.cos(2 * np.pi * x))


def ackley(x):
    x = np.asarray(x)
    n = len(x)
    term1 = -20 * np.exp(-0.2 * np.sqrt(np.sum(x**2) / n))
    term2 = -np.exp(np.sum(np.cos(2 * np.pi * x)) / n)
    return term1 + term2 + 20 + np.e


def rosenbrock(x):
    x = np.asarray(x)
    return np.sum(100 * (x[1:] - x[:-1] ** 2) ** 2 + (1 - x[:-1]) ** 2)
```

15.5.4. Objective Wrapper

```
import optuna

FUNCTION_NAME = "rastrigin"  # or "ackley" / "rosenbrock"


def evaluate_function(x):
    if FUNCTION_NAME == "rastrigin":
        return rastrigin(x)
    if FUNCTION_NAME == "ackley":
        return ackley(x)
    if FUNCTION_NAME == "rosenbrock":
        return rosenbrock(x)
    raise ValueError("Unknown function")


def objective(trial):
    x1 = trial.suggest_float("x1", -5.0, 5.0)
    x2 = trial.suggest_float("x2", -5.0, 5.0)
    return evaluate_function([x1, x2])
```

15.5.5. Comparing Samplers

```
def run_study(sampler, n_trials=200):
    study = optuna.create_study(direction="minimize", sampler=sampler)
    study.optimize(objective, n_trials=n_trials)
    return study


study_random = run_study(optuna.samplers.RandomSampler(seed=42), n_trials=200)
study_tpe = run_study(optuna.samplers.TPESampler(seed=42), n_trials=200)
study_cmaes = run_study(optuna.samplers.CmaEsSampler(seed=42), n_trials=200)

print("Random best:", study_random.best_value, study_random.best_params)
print("TPE best:", study_tpe.best_value, study_tpe.best_params)
print("CMA-ES best:", study_cmaes.best_value, study_cmaes.best_params)
```

15.5.6. Recommended Evaluation Protocol

Do not stop at one run per sampler. A serious comparison should repeat each sampler across several seeds.

```
def repeated_runs(sampler_factory, n_runs=5, n_trials=200):
    results = []
    for seed in range(n_runs):
        sampler = sampler_factory(seed)
        study = optuna.create_study(direction="minimize", sampler=sampler)
        study.optimize(objective, n_trials=n_trials)
        results.append(study.best_value)
    return results


random_results = repeated_runs(
    lambda seed: optuna.samplers.RandomSampler(seed=seed)
)
tpe_results = repeated_runs(
    lambda seed: optuna.samplers.TPESampler(seed=seed)
)
cmaes_results = repeated_runs(
    lambda seed: optuna.samplers.CmaEsSampler(seed=seed)
)

print("Random:", random_results)
print("TPE:", tpe_results)
print("CMA-ES:", cmaes_results)
```

15.5.7. What Project B Should Teach You

This project is successful if it teaches you:

- how samplers differ in exploration and exploitation behavior
- when TPE is a strong default and when it is not
- where CMA-ES is competitive, especially on continuous spaces
- how problem geometry changes optimizer performance
- why one good run is not enough for a fair comparison

15.5.8. Deliverables for Project B

At the end of Project B, you should have:

- the best result for each sampler
- repeated-run statistics across seeds
- a convergence comparison
- a short interpretation of sampler behavior
- a conclusion about which sampler suits which kind of landscape

15.6. Project C: Simulation Optimization

Project C is where Optuna looks least like a hyperparameter tuner and most like a general optimization engine.

15.6.1. Objective

Tune parameters of a stochastic simulation in order to optimize an expected performance measure.

This is a strong capstone because it introduces several real difficulties at once:

- noisy objectives
- repeated evaluations for robustness
- integer decision variables
- no gradients
- objective cost driven by simulation runtime

15.6.2. Example Problem: Inventory Optimization

Use a simple inventory-control simulation. The system has two decision variables:

- reorder point
- reorder quantity

The simulation returns measures such as total cost and stockouts under stochastic demand. The goal is to minimize expected cost.

15.6.3. Simulation Model

```
import numpy as np


def simulate_inventory(reorder_point, reorder_quantity, days=180, seed=None):
    rng = np.random.default_rng(seed)

    inventory = reorder_quantity
    total_cost = 0.0
    stockouts = 0
    pending_order = 0
    lead_time_remaining = 0

    holding_cost_per_unit = 0.1
    order_cost = 20.0
    stockout_penalty = 15.0

    for _ in range(days):
```

```
        demand = rng.poisson(8)

        if lead_time_remaining > 0:
            lead_time_remaining -= 1
            if lead_time_remaining == 0:
                inventory += pending_order
                pending_order = 0

        sold = min(inventory, demand)
        unmet = demand - sold
        inventory -= sold

        total_cost += inventory * holding_cost_per_unit
        total_cost += unmet * stockout_penalty
        stockouts += unmet

        if inventory <= reorder_point and pending_order == 0:
            pending_order = reorder_quantity
            lead_time_remaining = 3
            total_cost += order_cost

    return {
        "total_cost": total_cost,
        "stockouts": stockouts,
    }
```

15.6.4. Objective Function

Because the simulation is noisy, the objective should average multiple replications.

```
import optuna


def objective(trial):
    reorder_point = trial.suggest_int("reorder_point", 0, 50)
    reorder_quantity = trial.suggest_int("reorder_quantity", 5, 100)

    costs = []
    for rep in range(10):
        result = simulate_inventory(
            reorder_point=reorder_point,
            reorder_quantity=reorder_quantity,
            days=180,
            seed=rep,
        )
        costs.append(result["total_cost"])
```

```
    return float(np.mean(costs))


study = optuna.create_study(
    direction="minimize",
    sampler=optuna.samplers.TPESampler(seed=42),
)

study.optimize(objective, n_trials=100)

print("Best cost:", study.best_value)
print("Best params:", study.best_params)
```

15.6.5. Why Project C Is Valuable

Project C should convince you that Optuna is not limited to ML. It is a strong fit for decision problems where evaluation requires stochastic simulation and robustness matters more than analytic convenience.

15.6.6. Possible Extensions

You can deepen Project C by adding:

- multi-objective optimization over cost and stockouts
- explicit service-level constraints
- parallel execution of simulation replications
- pruning for long-horizon simulations with intermediate signals
- a financial or control-systems variant instead of inventory management

15.6.7. Deliverables for Project C

At the end of Project C, you should have:

- the best parameter configuration
- expected simulated performance
- variability across replications
- a short discussion of objective noise and robustness
- optional sensitivity analysis or multi-objective trade-offs

> ⚠ Common Mistake
>
> Do not treat the final project as a contest to produce the single highest score at any cost. A weak capstone often over-optimizes one metric while ignoring evaluation quality, robustness across seeds, or the interpretability of the search procedure. The goal is not just to win a

benchmark, but to justify the workflow.

15.7. How the Three Projects Fit Together

The three projects are complementary rather than redundant.

Project	Main skill
A	structured ML optimization with honest evaluation
B	sampler comparison and optimization theory in practice
C	stochastic black-box optimization beyond ML

Taken together, they form a practical definition of Optuna mastery: you can use the library on real ML tasks, controlled benchmark landscapes, and noisy simulation systems without changing the underlying optimization mindset.

15.8. Recommended Capstone Order

If you plan to complete all three, the most effective order is:

1. Start with Project B to sharpen your intuition about samplers.
2. Continue with Project A to practice structured ML evaluation and leakage-safe workflow design.
3. Finish with Project C to generalize your understanding to noisy, non-ML black-box systems.

That order moves from controlled settings to more realistic, operationally messy ones.

15.9. Final Report Structure

A strong capstone report should include the following for each project:

- problem statement
- search-space design and justification
- sampler choice and rationale
- evaluation protocol
- best results and diagnostics
- brief lessons learned

The strongest final synthesis answers questions such as:

1. When is TPE the right default?
2. When is CMA-ES a better choice?
3. How should search-space design differ between ML, benchmark, and simulation settings?
4. How does objective noise change the optimization strategy?

15.10. Completion Checklist

Before considering the capstone complete, verify that you can show all of the following:

- at least one full ML optimization workflow
- at least one controlled sampler comparison on a benchmark function
- at least one noisy black-box optimization problem outside ML
- written justification for the main search-space and sampler decisions
- some form of diagnostic or interpretive analysis, not only final numbers
- a clear explanation of what you would improve in the next iteration

15.11. Recap

The final project is successful when it demonstrates synthesis rather than repetition.

- Project A proves that you can optimize a realistic ML pipeline responsibly.
- Project B proves that you can reason about optimizer behavior rather than only accept defaults.
- Project C proves that you understand Optuna as a general-purpose black-box optimization engine.
- The combined portfolio proves that you can carry Optuna from theory to practice across very different problem classes.

15.12. Course Completion Summary

By finishing this capstone portfolio, you will have used Optuna for:

- hierarchical ML pipelines
- benchmark black-box optimization
- stochastic simulation tuning

That is a strong and well-rounded mastery path. From here, the natural next step is to build your own compact Optuna reference sheet or internal playbook covering core APIs, search-space rules, sampler selection, pruning patterns, and production checklists.

Summary

This book set out to present Optuna as more than a convenient hyperparameter-tuning tool. Across the full set of chapters, the central idea has been consistent: Optuna is a practical framework for black-box optimization problems where evaluations are expensive, noisy, structured, or operationally constrained.

The early chapters established the optimization setting itself. We started from the limits of grid search, random search, and gradient-based thinking in black-box problems, then moved into Bayesian optimization concepts and the TPE algorithm that gives Optuna much of its practical strength. That foundation matters because good Optuna usage depends on understanding what kind of search problem you are solving, not just which API call to make.

The middle chapters focused on workflow design. You saw how studies and trials are structured, how search spaces should be defined, how samplers differ, and how pruning changes the economics of expensive evaluations. Those chapters all point to the same practical truth: optimization performance is shaped as much by search-space design and evaluation logic as by the optimizer itself.

The later chapters broadened the picture. We moved into multi-objective optimization, diagnostics, distributed execution, machine-learning workflows, general-purpose black-box optimization, and production concerns such as persistence, reproducibility, and operational handoff. By that point, Optuna stopped looking like a tuning helper and started looking like part of a broader optimization system.

The Main Lessons To Keep

Several ideas should remain after the details of the individual examples fade.

- Optimization begins with problem formulation. A bad search space or a weak evaluation protocol will undermine even a strong sampler.
- TPE is a strong default for many real Optuna workloads, but defaults should still be justified against problem structure.
- Pruning is valuable when intermediate signals are meaningful, not merely because early stopping sounds efficient.
- Diagnostics matter. Optimization history, parameter importance, and interaction plots are part of understanding the search, not optional polish.
- Persistent studies, metadata, and final-training separation are essential once optimization moves beyond local experiments.
- Optuna applies well beyond machine learning whenever the objective can be evaluated as a black box and sample efficiency matters.

How The Parts Fit Together

The structure of the book was cumulative by design.

- The foundations explain why Optuna exists.
- The core usage chapters explain how to express problems in Optuna.
- The advanced chapters explain how to interpret, scale, and extend optimization workflows.
- The application chapters show how the same ideas transfer across ML, benchmark optimization, simulation, and production systems.
- The capstone chapter turns those ideas into a portfolio of complete workflows.

If you have worked through the full book carefully, you should now be able to move between theory and practice without treating them as separate worlds.

What Competence Should Look Like Now

At this point, you should be able to answer questions such as:

- Is this problem a good fit for Optuna?
- Is the search space reflecting the real structure of the decision problem?
- Is the objective measuring what actually matters?
- Is the evaluation protocol stable and leakage-free?
- Is the chosen sampler appropriate for the space and budget?
- Are the results reproducible, inspectable, and operationally usable?

Those are better indicators of mastery than simply knowing how to launch a study.

Where To Go Next

There are several strong next steps depending on your goals.

- If you care most about applied machine learning, extend the pipeline and production chapters into your own project templates.
- If you care about optimization research, use the benchmark and sampler-comparison material as a starting point for more systematic experiments.
- If you work with simulations or decision systems, adapt the general-optimization and production chapters into reproducible black-box optimization workflows.
- If you lead teams, turn the practical rules from this book into an internal playbook covering search spaces, evaluation design, storage, diagnostics, and deployment handoff.

Final Takeaway

The real value of Optuna is not that it automates search. Many tools can automate search. Its value is that it lets you express complex optimization problems cleanly, reuse information efficiently, and build workflows that scale from quick experiments to reproducible systems.

Used well, Optuna is not just a tuning library. It is a disciplined way to think about black-box optimization in practice.

References

A. Optuna Expert Cheat Sheet

Use this appendix as a compact reference for the patterns, APIs, and decision rules that matter most in day-to-day Optuna work. It is meant to complement the main chapters, not replace them. When you need the full reasoning behind a recommendation, follow the cross-links back into the corresponding module.

A.1. How To Use This Appendix

This appendix is organized around practical tasks:

- defining an objective
- choosing parameter types
- selecting samplers and pruners
- persisting and scaling studies
- diagnosing search behavior
- applying Optuna in machine-learning and production settings

If you only need a quick reminder, jump to One-Screen Summary. If you need a slightly more complete operational reference, scan the sections in order.

A.2. 1. Basic Optimization Workflow

Minimal Optuna workflow:

```
import optuna


def objective(trial):
    x = trial.suggest_float("x", -10, 10)
    return (x - 2) ** 2


study = optuna.create_study(direction="minimize")
study.optimize(objective, n_trials=100)

print(study.best_params)
print(study.best_value)
```

Core workflow:

```
define objective
create study
run optimization
inspect results
```

For the deeper conceptual picture, see Basic Optuna Usage and Optuna Architecture.

A.3. 2. Parameter Suggestion API

A.3.1. Continuous parameters

```
trial.suggest_float("lr", 1e-5, 1e-1)
```

A.3.2. Log-scale parameters

Use log scaling for multiplicative parameters such as learning rate, regularization, tolerances, and other quantities that span several orders of magnitude.

```
trial.suggest_float("lr", 1e-5, 1e-1, log=True)
```

A.3.3. Integer parameters

```
trial.suggest_int("depth", 2, 10)
```

A.3.4. Categorical parameters

```
trial.suggest_categorical("optimizer", ["adam", "sgd"])
```

For search-space design guidance, see Designing Search Spaces.

A.4. 3. Conditional Search Spaces

Optuna supports dynamic, define-by-run spaces.

```
model = trial.suggest_categorical("model", ["rf", "svm"])

if model == "rf":
    depth = trial.suggest_int("depth", 3, 10)
else:
    c_value = trial.suggest_float("C", 1e-3, 10, log=True)
```

This creates hierarchical parameter spaces where later decisions depend on earlier choices.

That pattern becomes essential once model families, pipeline branches, or architectural decisions vary across trials.

A.5. 4. Study Configuration

A.5.1. Default study

```
study = optuna.create_study(direction="minimize")
```

A.5.2. Custom sampler

```
study = optuna.create_study(
    sampler=optuna.samplers.TPESampler()
)
```

A.5.3. Persistent storage

```
study = optuna.create_study(
    storage="sqlite:///study.db",
    study_name="experiment",
    load_if_exists=True,
)
```

Persistent storage is the default choice once a study matters beyond one local session. See Distributed and Scalable Optimization and Production Workflows.

A.6. 5. Common Samplers

Sampler	Use case
`TPESampler`	General hyperparameter tuning and mixed spaces
`RandomSampler`	Baselines and debugging
`CmaEsSampler`	Continuous optimization
`GridSampler`	Very small discrete search spaces
`NSGAIISampler`	Multi-objective optimization

Example:

```
sampler = optuna.samplers.CmaEsSampler()

study = optuna.create_study(
    sampler=sampler,
    direction="minimize",
)
```

The practical default for many problems remains TPE. Use other samplers when the geometry or objective structure clearly justifies it. See Samplers.

A.7. 6. Pruning Pattern

Pruning is useful for iterative training loops.

```
for epoch in range(100):
    loss = train()
    trial.report(loss, epoch)

    if trial.should_prune():
        raise optuna.TrialPruned()
```

Create the study with a pruner:

```
study = optuna.create_study(
    pruner=optuna.pruners.MedianPruner()
)
```

Use pruning only when intermediate values are genuinely meaningful. See Pruning.

A.8. 7. Best Trial Retrieval

After optimization:

```
study.best_value
study.best_params
study.best_trial
```

Those are the first values you usually inspect, but they are not the whole story. Diagnostics and trial history still matter.

A.9. 8. Access All Trials

```
for trial in study.trials:
    print(trial.number)
    print(trial.params)
    print(trial.value)
```

The full trial list is useful for debugging, visualization, reporting, and understanding whether the study is converging or drifting.

A.10. 9. Visualization Tools

Requires:

```
pip install plotly
```

A.10.1. Optimization history

```
optuna.visualization.plot_optimization_history(study)
```

A.10.2. Parameter importance

```
optuna.visualization.plot_param_importances(study)
```

A.10.3. Slice plot

```
optuna.visualization.plot_slice(study)
```

A.10.4. Parallel coordinate

```
optuna.visualization.plot_parallel_coordinate(study)
```

A.10.5. Contour plots

```
optuna.visualization.plot_contour(study)
```

These plots are most useful when treated as diagnostic tools rather than presentation extras. See Visualization and Diagnostics.

A.11. 10. Dashboard

Interactive monitoring:

```
optuna-dashboard sqlite:///study.db
```

It helps inspect:

- trials
- parameters
- performance history
- Pareto frontiers

A.12. 11. Distributed Optimization

Shared database example:

```
study = optuna.create_study(
    storage="postgresql://user:pass@host/db",
    load_if_exists=True,
)
```

Run multiple workers:

```
python worker.py
python worker.py
python worker.py
```

All workers collaborate on the same study. For scaling trade-offs and storage choices, see Distributed and Scalable Optimization.

A.13. 12. Parallel Execution

Single-machine parallelization:

```
study.optimize(objective, n_trials=200, n_jobs=4)
```

This is usually the first scaling step before moving to shared-storage distributed execution.

A.14. 13. Multi-objective Optimization

Create the study:

```
study = optuna.create_study(
    directions=["minimize", "maximize"],
    sampler=optuna.samplers.NSGAIISampler(),
)
```

Objective:

```
return loss, model_size
```

Get the Pareto front:

```
study.best_trials
```

See Advanced Optimization Techniques.

A.15. 14. Reproducibility

Set seeds in Optuna:

```
sampler = optuna.samplers.TPESampler(seed=42)
```

Also control randomness in:

- `numpy`
- `torch`
- `tensorflow`

Reproducibility also depends on data versioning, environment control, and explicit evaluation logic.

A.16. 15. Production Setup

One reasonable architecture is:

```
ML pipeline
   -> Optuna optimization
   -> PostgreSQL storage
   -> Optuna dashboard
   -> experiment tracking (MLflow)
```

See Production Workflows.

A.17. 16. Common Best Practices

A.17.1. Use log scales when scale matters

Good:

```
trial.suggest_float("lr", 1e-5, 1e-1, log=True)
```

A.17.2. Avoid huge ranges without justification

Bad:

```
learning rate: [1e-6, 10]
```

A.17.3. Use cross-validation when appropriate

```
cross_val_score(model, X, y, cv=5)
```

A.17.4. Use pipelines to avoid leakage

```
Pipeline([
    ("scaler", StandardScaler()),
    ("model", RandomForest()),
])
```

A.17.5. Use pruning for iterative workloads

Pruning is often valuable in deep learning, boosting, and long-running iterative objectives.

A.18. 17. Common Optimization Pattern

Recommended workflow:

```
design search space
run 100-300 trials
visualize results
refine search space
run second optimization
```

Optimization is usually iterative. The first study often teaches you how to design the second one better.

A.19. 18. Debugging Optimization

Signs of a weak search space or unstable objective:

Symptom	Likely cause
no convergence	space too large
flat slice plot	parameter irrelevant or poorly explored
boundary solutions	range too narrow or badly centered
unstable results	noisy objective

A.20. 19. Typical Trial Budgets

Problem	Trials
small models	50-100
ML pipelines	100-300
deep learning	200-1000
simulation optimization	500+

These are practical heuristics, not rules. Objective cost and noise matter as much as problem type.

A.21. 20. Mental Model Of Optuna

Optuna optimization loop:

```
Sampler proposes parameters
        ↓
Objective evaluates
        ↓
Result stored
        ↓
Sampler updates strategy
        ↓
Next trial
```

This continues until one of the stopping criteria is met:

- `n_trials`
- timeout
- manual stop

A.22. One-Screen Summary

```
trial.suggest_* -> define search space
objective()     -> compute metric
study.optimize  -> run trials
sampler         -> choose parameters
pruner          -> stop bad trials
visualization   -> analyze results
storage         -> persist experiments
```

A.23. Final Note

This cheat sheet is a quick reference, not a substitute for workflow judgment. The strongest Optuna users are not the ones who memorize the most methods. They are the ones who design the objective carefully, shape the search space thoughtfully, and interpret optimization results with discipline.